How To Use the Notebooks

The notebooks use various packages (which you need to install) and also some local modules (`FinEcmt_OLS`, etc.) which are located in the `src` subfolder.

The next fell cells illustrate how to work with these notebooks.

Load Packages and Extra Functions

The next two cells puts the `src` subfolder in the `LOAD_PATH` (if it isn't already) and then loads some modules and packages.

1. The `FinEcmt_OLS` module is defined in the `src` subfolder. It uses many of the .jl files in that subfolder and exports the key functions. The cells below show two *different* ways of loading this module. Sometimes VS code protests about precompiling (or rather about how to print the compilation message), but running the cell again typically works.
2. Other local modules (`FinEcmt_TimeSeries.jl` etc.) are also defined in the `src` subfolder and are called on in the relevant notebooks.
3. The `Statistics` package is part of the Julia distribution, so you don't have to install it (this may change in the future).
4. The `Distributions.jl` package needs to be installed. Do `import Pkg;Pkg.add("Distributions")`.

```
#approach 1 to load the FinEcmt_OLS module, precompiles so is quicker

MyModulePath = joinpath(pwd(),"src")     #add /src to module path
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath);

using FinEcmt_OLS: OlsGM, printmat, @doc2   #to load a few functions from module
#using FinEcmt_OLS                         #to load all functions in module
```

```
#approach 2 to load the FinEcmt_OLS module, works also on Colab

#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS: OlsGM, printmat, @doc2   #to load a few functions from module
using .FinEcmt_OLS                          #to load all functions in module
=#
```

```
using Statistics, Distributions
```

See the Documentation

The next cell shows the documentation of the `OLSGM` function.

```
@doc2 OlsGM                             #`doc OlsGM` does not work in VS Code
#println(@doc OlsGM)                    #plain text printing instead
```

```
OlsGM(Y,X)
```

LS of Y on X; for one dependent variable, Gauss-Markov assumptions

Input

- `Y::Vector`: T-vector, the dependent variable
- `X::Matrix`: Txk matrix of regressors (including deterministic ones)

Output

- `b::Vector`: k-vector, regression coefficients
- `u::Vector`: T-vector, residuals Y - yhat
- `Yhat::Vector`: T-vector, fitted values X*b
- `V::Matrix`: kxk matrix, covariance matrix of b
- `R²::Number`: scalar, R² value

See the Source Code

To see the source code, either open the correct file in the src subfolder, or use the CodeTracking.jl package to print the source code here in the notebook. The latter approach requires "calling" on the function with some valid inputs (in this case we use [1],[1]). In many notebooks, such printing is coded but commented out.

```
using CodeTracking
println(@code_string OlsGM([1],[1]))   #println gets the line breaks right
```

```
function OlsGM(Y,X)

    T    = size(Y,1)

    b    = X\Y
    Yhat = X*b
    u    = Y - Yhat

    σ²   = var(u)
    V    = inv(X'X)*σ²
    R²   = 1 - σ²/var(Y)

    return b, u, Yhat, V, R²

end
```

Use the Function

(here with some artificial inputs)

```
Y = [1,2,3]
X = [1 2;1 1;1 3]
b, = OlsGM(Y,X);
printmat(b)
```

```
     1.000
     0.500
```

Required External Packages

for the different local modules. These packages need to be installed:

1. FinEcmt_OLS: Distributions, StatsBase, FiniteDiff
2. FinEcmt_Lasso: OSQP
3. FinEcmt_TimeSeries: none
4. FinEcmt_MLEGMM: Optim, FiniteDiff
5. FinEcmt_KernelRegression: none

We also use a number of standard libraries (eg. Printf), but they are typically shipped with the Julia binary.

For the notebooks, some more external packages are needed, for instance, CodeTracking for printing the functions. See the different notebooks for more information.

In case a package is missing, Julia will give an error message (and often tell you what you need to do).

List of Files

```
using Printf
include("src/printmat.jl");
```

```
files = filter(x -> endswith(lowercase(x),".ipynb"), readdir())  #all .ipynb files
filter!(x -> x != "Ch00_ListOfFiles.ipynb",files)

printmat(files)
```

```
Ch00_HowToUse.ipynb
Ch01_StatsReview.ipynb
Ch02a_OLS.ipynb
Ch02b_FrischWaugh.ipynb
Ch03_OLS_Distribution.ipynb
Ch04a_OLS_Testing.ipynb
Ch04b_DeltaMethod.ipynb
Ch05_OLS_non_iid.ipynb
Ch06_OLS_System.ipynb
Ch08a_BinScatter.ipynb
Ch08c_NLSProbit.ipynb
Ch09_Asymptotics.ipynb
Ch10a_MonteCarlo.ipynb
Ch10b_Bootstrap.ipynb
Ch11_PortfolioSorts.ipynb
Ch12a_PanelRegr.ipynb
Ch12b_EventStudy.ipynb
Ch13_IV.ipynb
Ch14_GMM.ipynb
Ch15_TimeSeries.ipynb
Ch16b_LassoElasticNet.ipynb
Ch17_ReturnPredictability.ipynb
Ch18_Distributions.ipynb
```

Ch19_MLE.ipynb
Ch20a_Garch.ipynb
Ch20b_DCC.ipynb
Ch21_RiskMeasures.ipynb
Ch22_NonparametricRegression.ipynb
Ch23_QuantileRegression.ipynb

Review of Statistics

This notebook shows some basic statistics needed for this course in financial econometrics. Details are in the first chapter.

It uses the `Statistics` package (built in) for descriptive statistics (averages, autocorrelations, etc) and the Distributions.jl package for statistical distributions (pdf, cdf, etc). For more stat functions, see the StatsBase.jl package (not used here).

Formatted printing is done with the `printmat()` function from the `FinEcmt_OLS` module (which is part of this repository). The source file is found in the `src` subfolder.

Load Packages and Extra Functions

```
MyModulePath = joinpath(pwd(),"src")      #add /src to module path
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))    #alternative way
using .FinEcmt_OLS
=#
```

```
using Statistics, DelimitedFiles, Distributions, QuadGK
```

```
using Plots, LaTeXStrings         #packages for plotting and LaTeX

default(size = (480,320),fmt = :png)  # :svg gives prettier plots, :png works better on GitHub
```

Distributions

Probability Density Function (pdf)

The cells below calculate and plot pdfs of some distributions often used in econometrics. The `Distributions.jl` package has many more distributions.

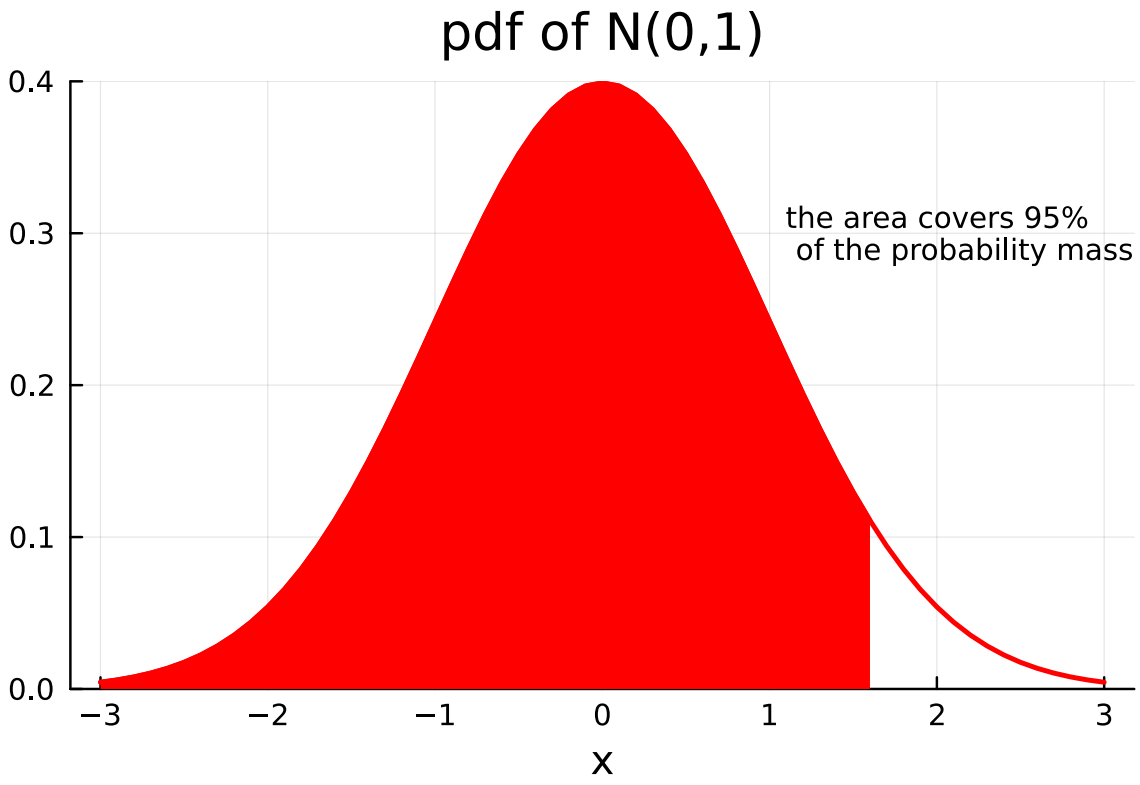
A Remark on the Code

- Notice that the `Distributions.jl` package wants `Normal(μ,σ)`, where σ is the standard deviation. However, the notation in the lecture notes is $N(\mu, \sigma^2)$. For instance, $N(0, 2)$ from the lectures is coded as `Normal(0,sqrt(2))`.
- `pdf.(Normal(0,1),x)` calculates the pdf of a standard normal variable at each value in the array `x`. Notice the dot (`.`).

```
x  = -3:0.1:3
xb = x[x.<=1.645]                 #pick out x values <= 1.645

pdfx  = pdf.(Normal(0,1),x)       #calculate the pdf of a N(0,1) variable
pdfxb = pdf.(Normal(0,1),xb)

p1 = plot( x,pdfx,                          #plot pdf
           linecolor = :red,
           linewidth = 2,
           legend = nothing,
           ylims = (0,0.4),
           title = "pdf of N(0,1)",
           xlabel = "x",
       annotation = (1.1,0.3,text("the area covers 95%\n of the probability mass",:left,8)) )
plot!(xb,pdfxb,linecolor=:red,linewidth=2,legend=nothing,fill=(0,:red))   #plot area under pdf
display(p1)
```



Cumulative Distribution Function (cdf)

The cdf calculates the probability for the random variable (here denoted y) to be below or at a value x, for instance, $\text{cdf}(x) = \Pr(y \leq x)$.

Also, we can calculate the probability that y exceeds x, $\Pr(x < y)$ as $1 - \text{cdf}(x)$ and $\Pr(x_1 < y \leq x_2)$ as $\text{cdf}(x_2) - \text{cdf}(x_1)$.

A Remark on the Code

$1 - \text{cdf}(x)$ can be coded as `1-cdf(dist,x)` where `dist` is, for instance, `Chisq(2)`. Alternatively, we could also use `ccdf(dist,x)`. (The extra `c` stands for the complement.) We use the latter approach below.

```
printblue("Probability of:\n")
println∩s("y<=-1.645 when y is N(0,1) ",cdf(Normal(0,1),-1.645))
println∩s("y<=0 when y is N(0,1)      ",cdf(Normal(0,1),0))
println∩s("2<y<=3 when y is N(0,2)   ",cdf(Normal(0,sqrt(2)),3)-cdf(Normal(0,sqrt(2)),2))
```

```
printlnPs("2<y<=3 when y is N(1,2)   ",cdf(Normal(1,sqrt(2)),3)-cdf(Normal(1,sqrt(2)),2))

printlnPs("\ny>4.61 when y is Chisq(2)  ",ccdf(Chisq(2),4.61))
printlnPs("y>9.24 when y is Chisq(5)  ",ccdf(Chisq(5),9.24))
```

```
Probability of:

y<=-1.645 when y is N(0,1)      0.050
y<=0 when y is N(0,1)           0.500
2<y<=3 when y is N(0,2)         0.062
2<y<=3 when y is N(1,2)         0.161

y>4.61 when y is Chisq(2)       0.100
y>9.24 when y is Chisq(5)       0.100
```

Quantiles (percentiles)

…are just about inverting the cdf. For instance, the 5th percentile is the value q such that cdf(q) = 0.05.

```
N_q     = quantile.(Normal(0,1),[0.025,0.05])
Chisq_q = quantile(Chisq(5),0.9)

printblue("\npercentiles:")
printlnPs("2.5th and 5th percentiles of a N(0,1) ",N_q)
printlnPs("90th percentile of a Chisquare(5)     ",Chisq_q)
```

```
percentiles:
2.5th and 5th percentiles of a N(0,1)     -1.960    -1.645
90th percentile of a Chisquare(5)          9.236
```

Numerical Integration to Find Theoretical Moments

In this section we apply a numerical integration of a function $g(x)$ to calculate its expectation by

$$\mathrm{E}g(x) = \int_{-\infty}^{\infty} f(x)g(x)dx,$$

where $f(x)$ is the pdf.

We use the `QuadGK.jl` package for the numerical integration.

```
(a,b) = (3,5)                                    #we study a+b*x

fg(x) = (a + b*x)*pdf(Normal(1,2),x)             #integrand for mean of a+b*x
Ex_num = QuadGK.quadgk(fg,-15,15)

fg2(x) = (a+b*x-(a+b*1))^2*pdf(Normal(1,2),x)  #integrand for variance of a+b*x
Var_num = QuadGK.quadgk(fg2,-15,15)

println("Correct and numerical mean&variance of  $a+$b*x, where x~N(1,2^2)")
xut = hcat([a+b*1;Ex_num[1]],[b^2*2^2;Var_num[1]])
printmat(xut;colNames=["mean","variance"],rowNames=["correct","numerical"])
```

```
Correct and numerical mean&variance of  3+5*x, where x~N(1,2^2)
               mean  variance
correct       8.000   100.000
numerical     8.000   100.000
```

Load Data from a csv File

```
x = readdlm("Data/FFmFactorsPs.csv",',',skipstart=1)

                #yearmonth, market, small minus big, high minus low
(ym,Rme,RSMB,RHML) = [x[:,i] for i=1:4]

println("Sample period: ",ym[1]," to ",ym[end])   #just numbers, not converted to Dates
```

```
Sample period: 197001.0 to 202412.0
```

Estimating Means and Standard Deviations

If $\text{std}(x)$ equals σ and x is iid, then $\text{std}(\bar{x})$ equals $\sigma/\sqrt{T}$. This can be used to test hypotheses, for instance, that the mean is zero.

```
x = [Rme RHML]

x̄  = mean(x,dims=1)    #,dims=1 to calculate average along a column
```

```
σ  = std(x,dims=1)     #add `;corrected=false` to divide by T instead of T-1
T  = length(Rme)
printmat([x̄;σ],colNames=["Rme","HML"],rowNames=["average","std"])
```

```
             Rme       HML
average     0.608     0.287
std         4.599     3.099
```

```
x̄_b  = sum(x,dims=1)/T                        #'manual' calculations
σ²_b = sum((x.-x̄_b).^2,dims=1)/(T-1)          #variance, using /(T-1)
printmat([x̄_b;sqrt.(σ²_b)],colNames=["Rme","HML"],rowNames=["average alt","std alt"])
```

```
                 Rme       HML
average alt     0.608     0.287
std alt         4.599     3.099
```

```
printblue("std of sample average (assuming iid data):\n")

Stdx̄  = σ/sqrt(T)
tstat = x̄./Stdx̄    #testing if the means are zero

printmat([x̄;Stdx̄;tstat],colNames=["Rme","HML"],rowNames=["average","std(average)","t-stat"])
```

```
std of sample average (assuming iid data):

                  Rme       HML
average          0.608     0.287
std(average)     0.179     0.121
t-stat           3.397     2.380
```

Estimating Covariances and Correlations

```
println("\ncov(x) matrix: ")
printmat(cov(x))

println("\ncor(x) matrix: ")
printmat(cor(x))
```

```
cov(x) matrix:
    21.149    -3.032
    -3.032     9.604


cor(x) matrix:
     1.000    -0.213
    -0.213     1.000
```

```
Ω = (x.-x̄)'*(x.-x̄)/(T-1)   #'manual' calculations, use T or T-1
printmat(Ω)

x̄_b = x̄'                   #need a column vector for this calculation
Ω_b = zeros(2,2)           #very 'manual' calculations
for t in 1:T
    global Ω_b
    Ω_b = Ω_b + (x[t,:]-x̄_b)*(x[t,:]-x̄_b)'/(T-1)
end
printmat(Ω_b)
```

```
    21.149    -3.032
    -3.032     9.604

    21.149    -3.032
    -3.032     9.604
```

For Data with Missing Values (indicated by `NaN` or `missing`)

We can use either the `excise()` or the `FindNN()` functions from the `FinEcmt_OLS` module.

Alternatively, consider the NaNStatistics.jl package.

```
x = [1 11;NaN 12;3 33]

μ₁ = mean(excise(x),dims=1)       #excise(x) cuts rows with any NaN/missing

vv = FindNN(x)                    #finds rows without any NaN/missing
μ₂ = mean(x[vv,:],dims=1)

printblue("averages estimated in different ways (to handle NaN/missing):")
printmat([μ₁;μ₂];rowNames=["excise","FindNN"],colNames=["x₁","x₂"])
```

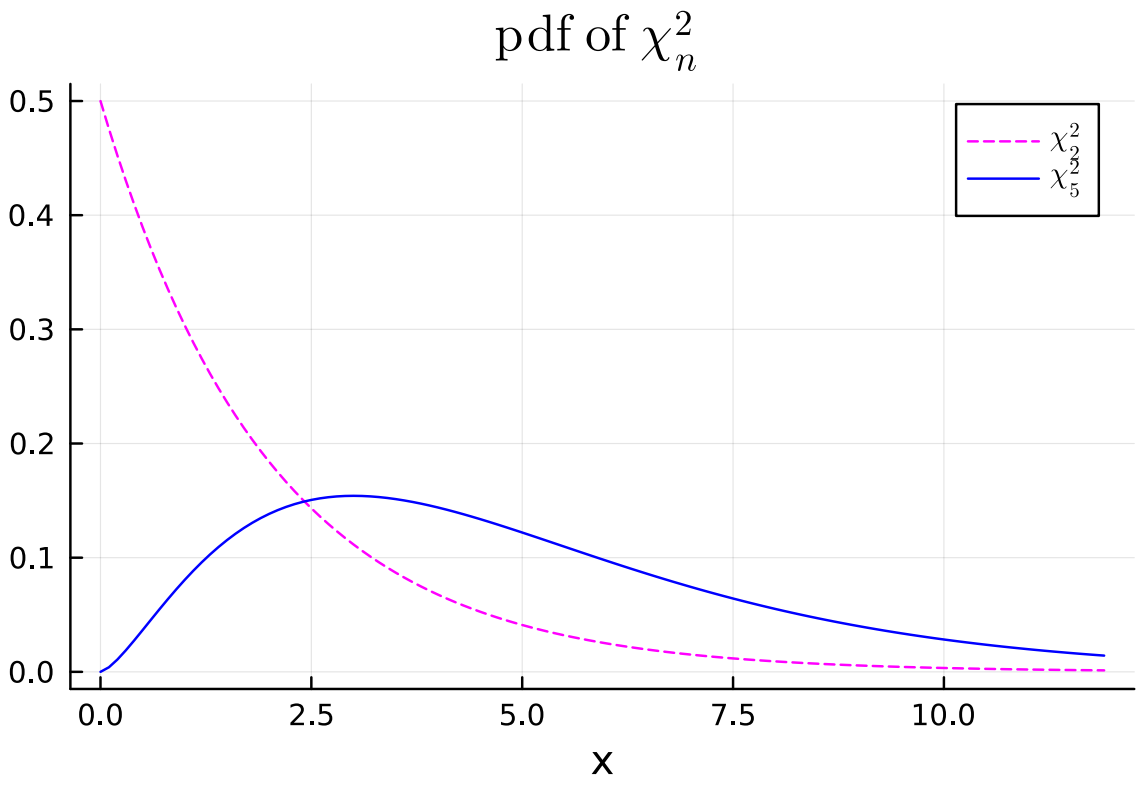
```
averages estimated in different ways (to handle NaN/missing):
              x₁        x₂
excise     2.000    22.000
FindNN     2.000    22.000
```

Distributions Commonly Used in Tests

```
x  = 0.0001:0.1:12

pdf2  = pdf.(Chisq(2),x)         #pdf of Chisq(2)
pdf5 =  pdf.(Chisq(5),x)

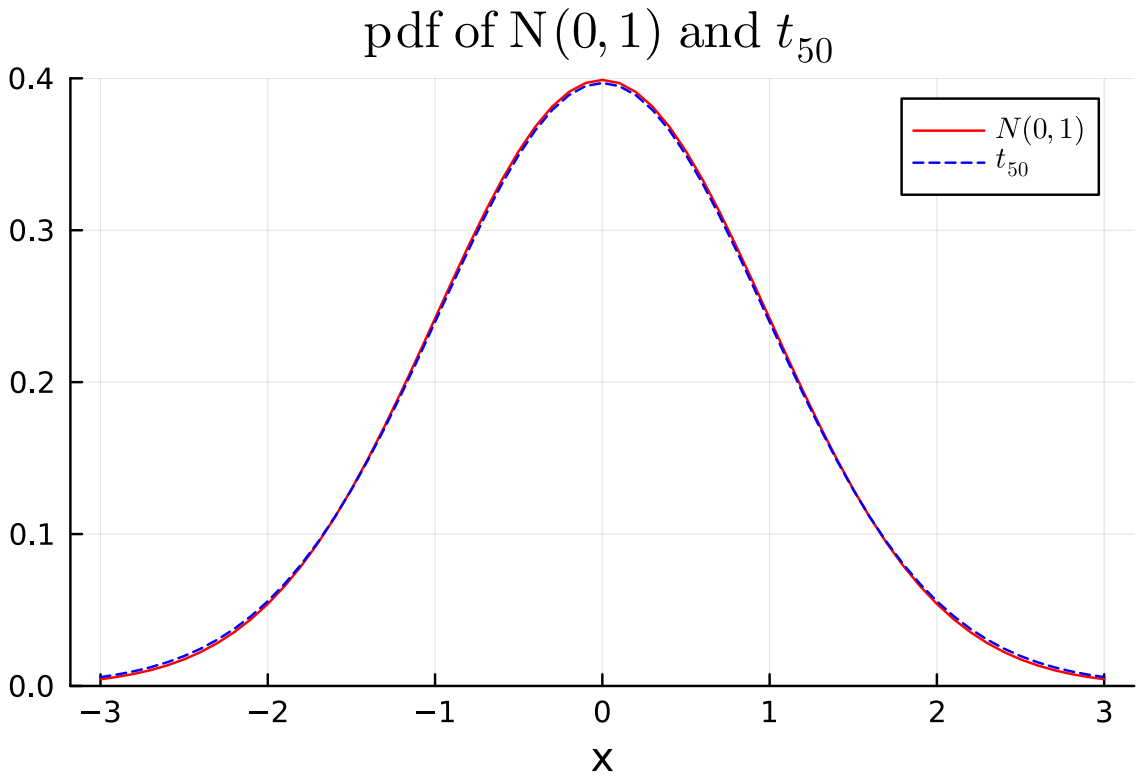
p1 = plot( x,[pdf2 pdf5],
           linecolor = [:magenta :blue],
           linestyle = [:dash :solid],
           label = [L"\chi_{2}^{2}" L"\chi_{5}^{2}"],
           title = L"\mathrm{pdf\ of\ } \chi_{n}^{2}",  #use \ to get spacing
           xlabel = "x" )
display(p1)
```




```
x  = -3:0.1:3

pdfN   = pdf.(Normal(0,1),x)
pdft50 = pdf.(TDist(50),x)            #pdf of t-dist with 50 df

p1 = plot( x,[pdfN pdft50],
           linecolor = [:red :blue],
           linestyle = [:solid :dash],
           label = [L"N(0,1)" L"t_{50}"],
           ylims = (0,0.4),
           title = L"\mathrm{pdf \ of \ N(0,1) \ and \ } t_{50}",
           xlabel = "x" )
display(p1)
```


pdf of N(0, 1) and t_{50}
0.4
0.3
0.2
0.1
0.0
−3
−2
−1
0
1
2
3
x
$N(0,1)$
t_{50}

Basic OLS

This notebook estimates a linear regression and reports traditional standard errors (assuming iid residuals).

For a package, consider GLM.jl or LinearRegression.jl (not used here).

Load Packages and Extra Functions

The key functions are from the `FinEcmt_OLS` module found in the `src` subfolder.

The `DelimitedFiles` package is used for importing the csv data file and the `LinearAlgebra` package for some matrix operations (eg. `diag()`, which extracts the diagonal of a matrix.)

```
MyModulePath = joinpath(pwd(),"src")     #add /src to module path
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using Statistics, DelimitedFiles, LinearAlgebra
```

Loading Data

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

                #yearmonth, 2 industries, factors
(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float
```

```
x = nothing
printlnPs("Re: ",size(Re),"\n","F: ",size(F))

y = Re[:,1]                    #to get standard OLS notation, here one return series
T = size(y,1)
x = [ones(T) F]
k = size(x,2)

println("\nT and k: $T $k")
```

```
      Re:   (660, 2)
       F:   (660, 3)

T and k: 660 4
```

OLS Estimates and Their Distribution

Consider the linear regression

$y_t = \beta' x_t + u_t,$

When x_t and u_t are independent and u_t is iid (Gauss-Markov assumptions), then the distribution of the estimates is (typically)

$\hat{\beta} \sim N(\beta_0, S_{xx}^{-1}\sigma^2),$

where σ^2 is the variance of the residual and $S_{xx} = \sum_{t=1}^{T} x_t x_t'$. Whether σ^2 is estimated using $1/T$ or $1/(T-1)$ affects the results slightly.

In matrix form, these expressions are $Y = Xb + u$, and $S_{xx} = X'X$, where X is defined below.

Matrix Form

To calculate the estimates it is often convenient to work with matrices. Define $X_{T\times k}$ by letting x_t' and be the tth row

$$X_{T\times k} = \begin{bmatrix} x_1' \\ \vdots \\ x_T' \end{bmatrix}$$

In contrast Y is a just a vector with T elements (or possibly a $T \times 1$ matrix).

This is implemented in the `OlsGM()` function from the `FinEcmt_OLS` module. The source file is in the `src` subfolder. The next cells print the documentation and source code of the function. In particular, notice the `b = X\Y` and `V = inv(X'X)*σ²`.

```
@doc2 OlsGM
```

```
OlsGM(Y,X)
```

LS of Y on X; for one dependent variable, Gauss-Markov assumptions

Input

- `Y::Vector`: T-vector, the dependent variable
- `X::Matrix`: Txk matrix of regressors (including deterministic ones)

Output

- `b::Vector`: k-vector, regression coefficients
- `u::Vector`: T-vector, residuals Y - yhat
- `Yhat::Vector`: T-vector, fitted values X*b
- `V::Matrix`: kxk matrix, covariance matrix of b
- `R²::Number`: scalar, R² value

```
using CodeTracking
println(@code_string OlsGM([1],[1]))    #print the source code
```

```
function OlsGM(Y,X)

    T    = size(Y,1)

    b    = X\Y
    Yhat = X*b
    u    = Y - Yhat

    σ²   = var(u;corrected=false)        #using 1/T, not 1/(T-1)
    V    = inv(X'X)*σ²
    R²   = 1 - var(u)/var(Y)

    return b, u, Yhat, V, R²

end
```

OLS Regression

```
(b,_,_,V,R²) = OlsGM(y,x)
Stdb = sqrt.(diag(V))        #standard errors

printblue("OLS Results:\n")
xNames = ["c","Market","SMB","HML"]
printmat(b,Stdb,colNames=["b","std"],rowNames=xNames)

printlnPs("R²: ",R²)
```

```
OLS Results:

               b       std
c          0.142     0.109
Market     1.130     0.025
SMB        0.180     0.036
HML       -0.510     0.036

      R²:      0.823
```

```
RegressionTable(b,V,xNames)     #a function for printing regression results
```

```
            coef    stderr    t-stat   p-value
c          0.142     0.109     1.303     0.193
Market     1.130     0.025    45.854     0.000
SMB        0.180     0.036     4.939     0.000
HML       -0.510     0.036   -14.350     0.000
```

Missing Values (extra)

The next cells use a simple function (`excise()`) to remove observations (t) where y_t and/or some of the x_t variables are NaN/missing. We illustrate the usage by a very simple example.

An alternative approach is to fill *both* y_t and x_t with zeros (if any of them contains NaN/missing) by using the `OLSyxReplaceNaN` function and then do the regression. This is illustrated in the subsequent cell.

```
(y0,x0) = (copy(y),copy(x))    #so we can can change some values
x0[2,2] = NaN                  #set a value to NaN

(y1,x1) = excise(y0,x0)      #gettting rid of obs t if any in (y_t,y_t) is Nan/missing
println("obs 1-3 before")
printmat(y0[1:3],x0[1:3,:];colNames=vcat("y",xNames))

println("after")
printmat(y1[1:3],x1[1:3,:];colNames=vcat("y",xNames))

printblue("OLS using only observations without any NaN/missing:")
b = x1\y1
printmat(b)
```

```
obs 1-3 before
         y         c    Market       SMB       HML
    -9.660     1.000    -8.100     2.930     3.130
     2.320     1.000       NaN    -2.580     3.930
    -3.150     1.000    -1.060    -2.320     3.990

after
         y         c    Market       SMB       HML
    -9.660     1.000    -8.100     2.930     3.130
    -3.150     1.000    -1.060    -2.320     3.990
   -14.840     1.000   -11.000    -6.150     6.180

OLS using only observations without any NaN/missing:
     0.143
     1.131
     0.179
    -0.509
```

```
(vv,y2,x2) = OLSyxReplaceNaN(y0,x0)      #alternative approach: fill (y_t,x_t) with zeros

println("after")
printmat(y2[1:3],x2[1:3,:];colNames=vcat("y",xNames))

printblue("OLS from setting observations with any NaN/missing to 0:")
b = x2\y2
printmat(b)
```

```
after
```

```
         y          c     Market        SMB        HML
    -9.660      1.000     -8.100      2.930      3.130
     0.000      0.000      0.000      0.000      0.000
    -3.150      1.000     -1.060     -2.320      3.990

OLS from setting observations with any NaN/missing to 0:
     0.143
     1.131
     0.179
    -0.509
```

Different Ways to Calculate OLS Estimates (extra)

Recall that OLS can be calculated as

$\hat{\beta} = S_{xx}^{-1} S_{xy}$, where $S_{xx} = \sum_{t=1}^{T} x_t x_t'$ and $S_{xy} = \sum_{t=1}^{T} x_t y_t$.

The next cell calculates the OLS estimates in three different ways: *(1)* a loop to create S_{xx} and S_{xy} followed by $S_{xx}^{-1} S_{xy}$; *(2)* $(X'X)^{-1}X'Y$; *(3)* and `X\Y`. They should give the same result in well-behaved data sets, but (3) is probably the most stable version.

The point of this is to get familiar with different approaches, and see that they represent the same.

```
printblue("Three different ways to calculate OLS estimates:")

k    = size(x,2)
Sxx = zeros(k,k)
Sxy = zeros(k,1)
for t = 1:T
    #local x_t, y_t            #local/global is needed in script
    #global Sxx, Sxy
    x_t = x[t,:]               #a vector
    y_t = y[t]
    Sxx = Sxx + x_t*x_t'     #kxk, same as Sxx += x_t*x_t'
    Sxy = Sxy + x_t*y_t      #kx1, same as Sxy += x_t*y_t
end
b1 = inv(Sxx)*Sxy            #OLS coeffs, version 1

b2 = inv(x'x)*x'y            #OLS coeffs, version 2

b3 = x\y                     #OLS coeffs, version 3

printmat(b1,b2,b3,colNames=["1","2","3"],rowNames=xNames)
```

Three different ways to calculate OLS estimates:

	1	2	3
c	0.142	0.142	0.142
Market	1.130	1.130	1.130
SMB	0.180	0.180	0.180
HML	-0.510	-0.510	-0.510

OLS, The Frisch-Waugh Theorem

This notebook illustrates the Frish-Waugh theorem.

In particular, it shows the following. First, we regress

$y = x_1'\beta_1 + x_2'\beta_2 + u$

Second, we run three regressions

1. $y = x_1'\gamma_1 + e_y$
2. $x_2 = x_1'\delta + e_2$
3. $e_y = e_2'\theta + v$, where (e_y, e_2) are from the regressions in 1. and 2.

Then, the estimates of β_2 and θ will be the same (as will their standard errors). This is used in, for instance, fixed effects panel regressions (where x_1 are dummies indicating different cross-sectional units).

Load Packages and Extra Functions

The `OlsGM()` function was used in ch. 2. It is from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))    #alternative way
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, LinearAlgebra
```

Loading Data

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float
x = nothing
println("Re: ",size(Re),"\n","F: ",size(F))

y = Re[:,1]                    #to get standard OLS notation, here one return series
T = size(y,1)
```

```
Re:   (660, 2)
 F:   (660, 3)
```

```
660
```

```
x₁ = [ones(T) F[:,1]]              #1st set of regressors (2)
x₂ = F[:,2:3];                     #2nd set of regressors
```

Regress y on Both x and x

```
(b,_,_,V,) = OlsGM(y,[x₁ x₂])
std_b = sqrt.(diag(V))

printblue("OLS Results from y regressed on x:\n")
rowNames=["x₁ (c)","x₁ (Rme)","x₂ (SMB)","x₂ (HML)"]
printmat([b std_b];colNames=["b","std"],rowNames)
```

```
OLS Results from y regressed on x:

                 b       std
x₁ (c)       0.142     0.109
x₁ (Rme)     1.130     0.025
x₂ (SMB)     0.180     0.036
x₂ (HML)    -0.510     0.036
```

The Three Steps in Frisch-Waugh

1. Regress y on x_1 and save the residuals as e_y. (Sorry, cannot create a symbol like e_y.)
2. Regress x_2 on x_1 and save the residuals as e_2.
3. Regress e_y on e_2.

```
(_,e_y,) = OlsGM(y,x₁)                #step 1

(_,e₂,) = OlsGM(x₂,x₁)                #step 2

(b,_,_,V,) = OlsGM(e_y,e₂)            #step 3
std_b = isa(V,Number) ? sqrt(V) : sqrt.(diag(V)) #diag() fails if V is a number (not a matrix)

printblue("OLS Results from e_y regressed on e₂:\n")
printmat([b std_b],colNames=["b","std"],rowNames=["e₂ (SMB)","e₂ (HML)"])
printred("Should be same coeff and std as in multiple regression (above)")
```

```
OLS Results from e_y regressed on e₂:

                b       std
e₂ (SMB)     0.180     0.036
e₂ (HML)    -0.510     0.036

Should be same coeff and std as in multiple regression (above)
```

A Partial Frisch-Waugh Approach (extra)

Regress y (not e_y) on e_2. This gives the same point estimate, but wrong standard error.

```
(b,_,_,V,) = OlsGM(y,e₂)              #step 3, adjusted
std_b = isa(V,Number) ? sqrt(V) : sqrt.(diag(V)) #diag() fails if V is a number (not a matrix)

printblue("OLS Results from y regressed on e₂:\n")
printmat([b std_b],colNames=["b","std"],rowNames=["e₂ (SMB)","e₂ (HML)"])
printred("Should be same coeff (but different std) as in multiple regression (above)")
```

```
OLS Results from y regressed on e₂:

                b       std
```

e_2 (SMB)	0.180	0.084
e_2 (HML)	-0.510	0.082

Should be same coeff (but different std) as in multiple regression (above)

OLS Distribution (iid) and Some Diagnostics

This notebook studies key properties of the model (distribution under iid assumptions, measures of fit, normality, multicollinearity).

You may also consider the HypothesisTests.jl package (not used here).

Load Packages and Extra Functions

The key functions for the diagnostic tests are from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics, LinearAlgebra
```

Loading Data

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float
x = nothing
println("Re: ",size(Re),"\n","F: ",size(F))

y = Re[:,1]                   #to get standard OLS notation, here one return series
```

```
T = size(y,1)
x = [ones(T) F]
k = size(x,2)

println("\nT and k: $T $k")
```

```
      Re:   (660, 2)
       F:   (660, 3)

T and k: 660 4
```

```
(b,u,_,V,R²) = OlsGM(y,x)    #do OLS
Stdb = sqrt.(diag(V))

printblue("OLS with traditional standard errors:\n")
xNames = ["c","market","SMB","HML"]
printmat([b Stdb],colNames=["coef","std"],rowNames=xNames)
```

```
OLS with traditional standard errors:

            coef       std
c          0.142     0.109
market     1.130     0.025
SMB        0.180     0.036
HML       -0.510     0.036
```

```
σ² = var(u;corrected=false) #variance of (fitted) residual, perhaps drop `;corrected=false`
Sxx = x'*x
V_alt = inv(Sxx)*σ²    #just double checking the VCV

Stdb_alt = sqrt.(diag(V_alt))
printmat(Stdb,Stdb_alt;colNames=["Std (earlier)","Std (new)"],rowNames=xNames)
```

```
      Std (earlier) Std (new)
c          0.109     0.109
market     0.025     0.025
SMB        0.036     0.036
HML        0.036     0.036
```

Measures of Fit

Adjusted R², AIC, BIC (the two latter is discussed in more detail in another chapter)

```
@doc2 RegressionFit
#using CodeTracking
#println(@code_string RegressionFit([1],0.0,3))    #print the source code
```

```
RegressionFit(u,R²,k)
```

Calculate adjusted R², AIC and BIC from regression residuals.

Input

- `u::Vector`: T-vector of residuals
- `R²::Float`: the R² value
- `k::Int`: number of regressors

```
(R²adj,AIC,BIC) = RegressionFit(u,R²,k)

printblue("Measures of fit")
printmat([R²,R²adj,AIC,BIC];rowNames=["R²","R²adj","AIC","BIC"])
```

```
Measures of fit
R²        0.823
R²adj     0.822
AIC       2.035
BIC       2.062
```

Test of Normality

of the residuals, applying the Jarque-Bera test.

```
@doc2 JarqueBeraTest
#println(@code_string JarqueBeraTest([1]))    #print the source code
```

```
JarqueBeraTest(x)
```

Calculate the JB test for each column in a matrix. Reports `(skewness,kurtosis,JB)`.

```
(skewness,kurtosis,JB,pvals) = JarqueBeraTest(u)

printblue("Test of normality")
xut = vcat(skewness,kurtosis,JB)
printmat(xut,collect(pvals);rowNames=["skewness","kurtosis","Jarque-Bera"],colNames=["stat","p-valu
```

```
Test of normality
                 stat   p-value
skewness        0.119     0.211
kurtosis        3.895     0.000
Jarque-Bera    23.598     0.000
```

Multicollinearity

by studying the correlation matrix and the variance inflation factor (VIF). A high VIF (5 to 10) might indicate issues with multicollinearity.

```
@doc2 VIF
#println(@code_string VIF([1]))    #print the source code
```

```
VIF(X)
```

Calculate the variance inflation factor

Input

- `x::Matrix`: Txk matrix with regressors

Output

- `maxVIF::Float`: highest VIF value
- `allVIF::Vector`: a k VIF values

```
printblue("Correlation matrix (checking multicollinearity)")
printmat(cor(x);colNames=xNames,rowNames=xNames)
```

```
Correlation matrix (checking multicollinearity)
                c    market       SMB       HML
c           1.000       NaN       NaN       NaN
market        NaN     1.000     0.281    -0.213
SMB           NaN     0.281     1.000    -0.155
HML           NaN    -0.213    -0.155     1.000
```

```
(maxVIF,allVIF) = VIF(x)
printblue("VIF (checking multicollinearity)")
printmat(allVIF;rowNames=xNames)
```

```
VIF (checking multicollinearity)
c           1.000
market      1.121
SMB         1.097
HML         1.058
```

A Convenience Function for Printing All These Tests (extra)

```
@doc2 DiagnosticsTable
#println(@code_string DiagnosticsTable([1],[1],0.0))    #print the source code
```

```
DiagnosticsTable(X,u,R²,nlags,xNames="")
```

Compute and print a number of regression diagnostic tests.

Input

- `X::Matrix`: Txk matrix of regressors
- `u::Vector`: T-vector of residuals
- `R²::Float`: the R² value
- `xNames::Vector`: of strings, regressor names

```
DiagnosticsTable(x,u,R²,xNames)
```

```
Test of all slopes = 0
stat   3072.554
p-val     0.000

Measures of fit
R²        0.823
R²adj     0.822
AIC       2.035
BIC       2.062

Test of normality
                 stat   p-value
skewness        0.119     0.211
kurtosis        3.895     0.000
Jarque-Bera    23.598     0.000

Correlation matrix (checking multicollinearity)
               c    market       SMB       HML
c          1.000       NaN       NaN       NaN
market       NaN     1.000     0.281    -0.213
SMB          NaN     0.281     1.000    -0.155
HML          NaN    -0.213    -0.155     1.000

VIF (checking multicollinearity)
c          1.000
market     1.121
SMB        1.097
HML        1.058
```

OLS, Testing

This notebook estimates a linear regression and tests various hypotheses using standard errors assuming iid residuals (Gauss-Markov assumptions).

You may also consider the HypothesisTests.jl package (not used here).

Load Packages and Extra Functions

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics, LinearAlgebra, Distributions
```

Loading Data

```
(x,header) = readdlm("Data/TwoIndustries.csv",',',header=true)

(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float
x = nothing
printlnPs("Re: ",size(Re),"\n","F: ",size(F))

y = Re[:,1]              #to get standard OLS notation, here only one return serprintlies
T = size(y,1)
```

```
x = [ones(T) F]
k = size(x,2)

println("Dependent variable: $(header[2])")
println("Regressors:")
printmat("c",header[4:end]...)
println("\nT and k: $T $k")
```

```
      Re:   (660, 2)
       F:   (660, 3)
Dependent variable: HiTec
Regressors:
         c    Market       SMB       HML


T and k: 660 4
```

OLS

(using a heteroskedasticity robust variance-covariance matrix, discussed in another notebook)

```
(b,u,_,V,R²) = OlsNW(y,x,0)
std_w = sqrt.(diag(V))

printblue("OLS Results (robust to heteroskedasticity):\n")
xNames = ["c","Market","SMB","HML"]
printmat(b,std_w;colNames=["b","std"],rowNames=xNames)

printblue("VCV*10_000 (robust to heteroskedasticity):\n")
printmat(V*10_000;colNames=xNames,rowNames=xNames)
```

```
OLS Results (robust to heteroskedasticity):

                b       std
c           0.142     0.107
Market      1.130     0.030
SMB         0.180     0.048
HML        -0.510     0.046

VCV*10_000 (robust to heteroskedasticity):
```

```
                c    Market       SMB       HML
c         114.875    -2.129    -0.095    -3.874
Market     -2.129     8.750    -5.490    -0.899
SMB        -0.095    -5.490    22.815     1.512
HML        -3.874    -0.899     1.512    20.871
```

Testing a Linear Combination

Since the estimator $\hat{\beta}_{k\times 1}$ satisfies

$\hat{\beta} - \beta \sim N(0, V_{k\times k})$,

we can easily apply various tests.

For instance, consider

$H_0 : R\beta = q$,

where R is a $1 \times k$ row vector and q is a scalar. In this case, we can apply a t-test as

$(R\hat{\beta} - q)/\omega$, where $\omega = \sqrt{RVR'}$.

```
println("The regressors are: ",xNames)
R = [0 0 1 1]                   #see xNames for the order of regressors
q = 0
ω = sqrt(only(R*V*R'))          #only() to create scalar from 1x1 matrix
t = (only(R*b) - q)/ω

printblue("Testing b₂+b₃=0, t-stat:")
println1Ps(t)
```

```
The regressors are: ["c", "Market", "SMB", "HML"]
Testing b₂+b₃=0, t-stat:
    -4.827
```

Testing a Joint Hypothesis

Consider a joint linear hypothesis of the form

$H_0 : R\beta = q$,

where R is a $J \times k$ matrix and q is a J-vector. To test this, use

$(R\beta - q)'(RVR')^{-1}(R\beta - q) \xrightarrow{d} \chi^2_J$.

How we estimate V depends on whether there is heteroskedasticity and/or autocorrelation.

```
R = [0 1 0 0;               #testing if b₂=0, b₃=0 and b₄=0
     0 0 1 0;
     0 0 0 1]
q = [0,0,0]
test_stat = (R*b-q)'inv(R*V*R')*(R*b-q)    #R*V*R' is 2x2

printblue("Testing Rb = [0,0,0] (χ²):")
printmat([test_stat,quantile(Chisq(2),0.9)];rowNames=["test statistic","10% critical value"])
```

```
Testing Rb = [0,0,0] (χ²):
test statistic      1959.030
10% critical value     4.605
```

Regression Diagnostics: Testing All Slope Coefficients

The `OlsR2Test()` function tests all slope coefficients (or equivalently, the R^2) of a regression, under the assumption of *iid residuals*. Notice that the regression must contain an intercept for R² to be useful. The test is to previous joint test, except that the current test assumes iid residuals, so the results can differ.

```
@doc2 OlsR2Test
```

```
OlsR2Test(R²,T,df)
```

Test of all slope coefficients. Notice that the regression must contain an intercept for R² to be useful.

Input

- `R²::Number`: R² value
- `T::Int`: number of observations
- `df::Number`: number of (non-constant) regressors

Output

- `RegrStat::Number`: test statistic
- `pval::Number`: p-value

```
using CodeTracking
println(@code_string OlsR2Test(1.0,1,25))    #print the source code
```

```
function OlsR2Test(R²,T,df)
    RegrStat = T*R²/(1-R²)           #R\^2[TAB]
    pval     = ccdf(Chisq(df),RegrStat)    #same as 1-cdf()
    return RegrStat, pval
end
```

```
df = size(x,2) - 1              #number of slope coefficients
(RegrStat,pval) = OlsR2Test(R²,T,df)

printblue("Test of all slopes = 0:\n")
printmat([RegrStat,pval],rowNames=["stat","p-val"])

println("The result looks more significant than before, since it assumes iid")
```

```
Test of all slopes = 0:

stat   3072.554
p-val     0.000

The result looks more significant than before, since it assumes iid
```

The Delta Method

Load Packages and Extra Functions

The notebook first implements the delta method step-by-step. At the end it also presents a the function `DeltaMethod()` from the (local) `FinEcmt_OLS` module that wraps those calculations.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics
```

Load Data

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float

T = size(F,1)
```

```
660
```

The Inputs: Parameters and their VCV

```
Rme = F[:,1]           #equity market excess return

μ   = mean(Rme)                  #estimates of the mean and variance
σ²  = var(Rme,corrected=false)

printblue("mean and variance:")
momNames = ["μ","σ²"]
printmat([μ,σ²];colNames=momNames)
```

```
mean and variance:
     0.608
    21.117
```

The variance-covariance matrix (called V) of the point estimates depends on the distribution of the data. Since data is unlikely to be iid N(), we abstain from using the known VCV for that case. Instead, we estimate V from the moment conditions (see GMM for details), which allows for a non-normal distribution (and more).

```
println("V*100, from moment conditions")
g = hcat(Rme .- μ,(Rme .- μ).^2 .- σ²)   #VCV from moment conditions
V = cov(g;corrected=false)/T
printmat(V*100;rowNames=momNames,colNames=momNames)
```

```
V*100, from moment conditions
            μ         σ²
μ       3.200     -7.526
σ²     -7.526    247.285
```

The Sharpe Ratio and Its Derivatives

The Sharpe ratio and its derivatives (with respect to the parameters of the Sharpe ratio) are

$SR(\beta) = \mu/\sigma$, where $\beta = (\mu, \sigma^2)$

Let $f(\beta)$ denote the Sharpe ratio where β is a vector of parameters consisting of the mean and the variance (μ, σ^2). The derivatives are then

$\frac{\partial f(\beta)}{\partial \beta'} = \begin{bmatrix} \frac{1}{\sigma} & \frac{-\mu}{2\sigma^3} \end{bmatrix}$

We will refer to the matrix of derivatives as D.

```
"""
    SRFn(μ,σ²)

Calculate the Sharpe ratio from the mean μ and variance σ²

"""
function SRFn(μ,σ²)
  σ  = sqrt(σ²)
  SR = μ/σ
  D = hcat(1/σ, -μ/(2*σ^3))     #Jacobian of SR, 1x2
  return SR, D
end
```

SRFn

```
(SR,D) = SRFn(μ,σ²)

printlnPs("Sharpe ratio: ",SR)

printblue("\nDerivatives*100 (the Jacobian) of Sharpe ratio function wrt:")
printmat(D*100,colNames=momNames)
```

```
Sharpe ratio:      0.132

Derivatives*100 (the Jacobian) of Sharpe ratio function wrt:
         μ        σ²
    21.761    -0.313
```

Applying the Delta Method

Recall that if

$\hat{\beta} \sim N(\beta, V)$,

then the distribution of the function $f(\hat{\beta})$ is asymptotically

$f(\hat{\beta}) \sim N(f(\beta), DVD')$

where D are the derivatives of $f(\beta)$.

```
Std_SR = sqrt(only(D*V*D'))  #only() to convert from 1x1 matrix to scalar
tstat = SR/Std_SR

printblue("Results from the delta method:")
printmat([SR Std_SR tstat],colNames=["SR","Std(SR)","t-stat"])

printblue("annualised:")

printblue("Results from the delta method:")
printmat([sqrt(12)*SR sqrt(12)*Std_SR tstat],colNames=["SR","Std(SR)","t-stat"])
```

```
Results from the delta method:
        SR   Std(SR)    t-stat
     0.132     0.041     3.265

annualised:
Results from the delta method:
        SR   Std(SR)    t-stat
     0.458     0.140     3.265
```

A Function for the Delta Method (extra)

is included below. It uses numerical derivatives from the `FiniteDiff.jl` package.

To use this, first write a function that takes `(β,x)` as inputs (see `SRFn2(β,x)` below), where `β` is a vector of the parameters and `x` any data needed (for the Sharpe ratio, no data is needed).

```
@doc2 DeltaMethod
```

```
DeltaMethod(fn::Function,β,V,x=NaN)
```

Apply the delta method on the function `fn(β,x)`

Input

- `fn::Function`: of the type fn(β,x)
- `β::Vector`: with parameters
- `V::Matrix`: Cov(β)
- `x::VecOrMat`: data (if any is needed)

Requires

- using FiniteDiff: finite_difference_jacobian as jacobian

```
using CodeTracking
println(@code_string  DeltaMethod(cos,[1],[1]))
```

```
function DeltaMethod(fn::Function,β,V,x=NaN)
    P = jacobian(b->fn(b,x),β)        #numerical Jacobian
    Cov_fn = P*V*P'
    return Cov_fn
end
```

```
"""
    SRFn2(β,x)

Function for Sharpe ratio in terms of the vector β. No derivatives
"""
function SRFn2(β,x=NaN)
  (μ,σ²) = β
  σ  = sqrt(σ²)
  SR = μ/σ
  return SR
end;
```

```
Var_SR = DeltaMethod(SRFn2,[μ,σ²],V)

printblue("Std of SR from DeltaMethod():")
printmat(sqrt(only(Var_SR)))

printblue("Compare with the earlier results where we used analytical derivatives")
```

```
Std of SR from DeltaMethod():
     0.041

Compare with the earlier results where we used analytical derivatives
```

OLS, Testing with Non-iid Residuals

This notebook tests for autocorrelation and heteroskedasticity and then tests hypotheses using standard errors from White's and Newey-West's methods.

You may also consider the HypothesisTests.jl package (not used here).

Load Packages and Extra Functions

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics, LinearAlgebra, Distributions, CodeTracking
```

Loading Data

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float
x = nothing
println∏s("Re: ",size(Re),"\n","F: ",size(F))

y = Re[:,1]                    #HiTec, standard OLS notation
T = size(y,1)
```

```
x = [ones(T) F[:,1]]           #constant, market excess return
k = size(x,2)

println("\nT and k: $T $k")
```

```
      Re:   (660, 2)
       F:   (660, 3)

T and k: 660 2
```

OLS under the Gauss-Markov Assumptions

(assuming iid residuals)

```
(b,u,_,V,R²) = OlsGM(y,x)
std_iid = sqrt.(diag(V))

printblue("OLS Results (assuming iid residuals):\n")
xNames = ["c","Market"]
printmat(b,b./std_iid;colNames=["b","t-stat"],rowNames=xNames)
```

```
OLS Results (assuming iid residuals):

                b    t-stat
c          -0.052    -0.413
Market      1.237    45.336
```

Distribution of OLS Estimates without the Gauss-Markov Assumptions

The distribution of the OLS estimates is (typically)

$(\hat{\beta} - \beta) \xrightarrow{d} N(0, V)$ where $V = S_{xx}^{-1} S S_{xx}^{-1}$.

and where $S_{xx} = \sum_{t=1}^{T} x_t x_t'$ and S is the covariance matrix of $\sum_{t=1}^{T} u_t x_t$.

When the Gauss-Markov assumptions hold, then S can be simplified to $S_{xx}\sigma^2$, where σ^2 is the variance of u_t, so $V = S_{xx}^{-1}\sigma^2$.

In contrast, with heteroskedasticity and/or autocorrelation, S must be estimated differently.

Heteroskedasticity

Regression Diagnostics: Heteroskedasticity

The `OlsWhitesTest()` function does White's test for heteroskedasticity. Again, the regression must have an intercept for this test to be useful.

```
@doc2 OlsWhitesTest
```

```
OlsWhitesTest(u,x)
```

Test of heteroskedasticity. Notice that the regression must contain an intercept for the test to be useful.

Input

- `u::Vector`: T-vector, residuals
- `x::Matrix`: Txk, regressors

Output

- `RegrStat::Number`: test statistic
- `pval::Number`: p-value
- `b::Vector`: coefficients in regressing `u^2` on w
- `tstat:Vector`: t-stats for b

```
#println(@code_string OlsWhitesTest([1],[1]))    #print the source code
```

```
(WhiteStat,pval,γ,γtstat) = OlsWhitesTest(u,x)

printblue("White's test (H₀: heteroskedasticity is not correlated with regressors):\n")
printmat([WhiteStat,pval],rowNames=["stat","p-val"])

printmat([γ γtstat][2:end,:],rowNames=["γ","t-stat"])
```

```
0.009228390534838815
White's test (H₀: heteroskedasticity is not correlated with regressors):

stat      6.147
p-val     0.046
```

```
γ          0.071     0.405
t-stat     0.051     2.478
```

White's Covariance Matrix

If $u_t x_t$ is not autocorrelated, then S simplifies to $\sum_{t=1}^{T} x_t x_t' \sigma_t^2$. White's method replaces σ_t^2 by $\hat{u}_t^2$. This estimate is robust to heteroskedasticity (in particular, time variation in σ_t^2 that is related to x_t).

A Remark on the Code

S_{xx} can be calculated as `Sxx = X'X` and S as `S = (X.*u)'*(X.*u)`.

Clearly, these calculations can also be done in a loop like

```
for t = 1:T
   Sxx = Sxx + X[t,:]*X[t,:]'
   S   = S   + X[t,:]*X[t,:]'*u[t]^2
end
```

```
Sxx = x'x

S     = (x.*u)'*(x.*u)                #S according to White's method
V     = inv(Sxx)'S*inv(Sxx)           #Cov(b), White
std_W = sqrt.(diag(V))

printblue("Coefficients and standard errors (from different methods):\n")
xx = hcat(b,b./std_iid,b./std_W)
printmat(xx;colNames=["b","t (iid)","t (White)"],rowNames=xNames,width=12)
```

```
Coefficients and standard errors (from different methods):

                b     t (iid)   t (White)
c          -0.052      -0.413      -0.412
Market      1.237      45.336      39.016
```

```
V_alt = OlsNW(y,x,0)[4]           #comparison with another OLS function, used later
printmat(b./sqrt.(diag(V_alt)))   #notice: lag order 0
```

```
    -0.412
    39.016
```

Autocorrelation

Regression Diagnostics: Autocorrelation

The `OlsAutoCorr()` function estimates autocorrelations, calculates the DW and Box-Pierce statistics for the input (often, the residual).

```
@doc2 OlsAutoCorr
```

```
OlsAutoCorr(u,L=1)
```

Test the autocorrelation of OLS residuals

Input

- `u::Vector`: T-vector, residuals
- `L::Int`: scalar, number of lags in autocorrelation and Box-Pierce test

Output

- `AutoCorr::Matrix`: Lx3, autocorrelation, t-stat and p-value
- `BoxPierce::Matrix`: 1x2, Box-Pierce statistic and p-value
- `DW::Number`: DW statistic

Requires

- StatsBase, Distributions

```
#println(@code_string OlsAutoCorr([1],5))    #print the source code
```

```
L = 3     #number of autocorrs to test

(ρStats,BoxPierce,DW) = OlsAutoCorr(u,L)

printmagenta("Testing autocorrelation of residuals\n")

printblue("Autocorrelations (lag 1 to $L):\n")
printmat(ρStats,colNames=["autocorr","t-stat","p-val"],rowNames=1:L,cell00="lag")
```

```
printblue("\nBoxPierce ($L lags): ")
printmat(BoxPierce',rowNames=["stat","p-val"])

printblue("DW statistic:")
printlnPs(DW)
```

```
Testing autocorrelation of residuals

Autocorrelations (lag 1 to 3):

lag  autocorr    t-stat     p-val
1      -0.039    -0.994     0.320
2       0.016     0.413     0.680
3       0.023     0.600     0.548


BoxPierce (3 lags):
stat      1.519
p-val     0.678

DW statistic:
     2.074
```

Autocorrelation of of `x.*u`

What matters most for the uncertainty about a slope coefficient is not the autocorrelation of the residual itself, but of the residual times the regressor. This is tested below.

```
k = size(x,2)
for i in 1:k         #iterate over different regressors
    #local pStats
    ρStats, = OlsAutoCorr(x[:,i].*u,L)
    printblue("Autocorrelations of $(xNames[i])*u  (lag 1 to $L):")
   printmat(ρStats,colNames=["autocorr","t-stat","p-val"],rowNames=1:L,cell00="lag")
end
```

```
Autocorrelations of c*u  (lag 1 to 3):
lag  autocorr    t-stat     p-val
1      -0.039    -0.994     0.320
2       0.016     0.413     0.680
```

```
3       0.023     0.600     0.548

Autocorrelations of Market*u  (lag 1 to 3):
lag  autocorr    t-stat     p-val
1       0.190     4.882     0.000
2       0.194     4.972     0.000
3       0.186     4.773     0.000
```

Newey-West's Covariance Matrix

Let $g_t = u_t x_t$ be a k-vector of data.

To calculate the Newey-West covariance matrix, we first need

$\Lambda_s = \sum_{t=s+1}^{T} (g_t - \bar{g})(g_{t-s} - \bar{g})'$,

which is proportional to the sth autocovariance matrices.

Then we form a linear combination (with tent-shaped weights) of those autocovariance matrices (from lag $-m$ to m) as in

$S = \text{Cov}(\sum_t g_t) = \Lambda_0 + \sum_{s=1}^{m} (1 - \frac{s}{m+1})(\Lambda_s + \Lambda_s')$.

With $m = 0$ this is the same as White's method.

If we divide S by T, then we get an estimate of $\text{Cov}(\sqrt{T}\bar{g})$, and if we instead divide by T^2 then we get an estimate of $\text{Cov}(\bar{g})$.

The `CovNW()` function implements this.

```
@doc2 CovNW
```

```
CovNW(g0,m=0,DivideByT=0)
```

Calculates covariance matrix of sample sum (DivideByT=0), √T*(sample average) (DivideByT=1) or sample average (DivideByT=2).

Input

- `g0::Matrix`: Txq matrix of data
- `m::Int`: number of lags to use
- `DivideByT::Int`: divide the result by T^DivideByT

Output

- `S::Matrix`: qxq covariance matrix

Remark

- `DivideByT=0`: Var(g +g +…), variance of sample sum
- `DivideByT=1`: Var(g +g +…)/T = Var(√T gbar), where gbar is the sample average. This is the same as Var(g) if data is iid
- `DivideByT=2`: Var(g +g +…)/T^2 = Var(gbar)

```
#println(@code_string CovNW([1],2))    #print the source code
```

```
S      = CovNW(x.*u,2)          #S acccording to Newey-West, 2 lags
V      = inv(Sxx)'S*inv(Sxx)    #Cov(b), Newey-West
std_NW = sqrt.(diag(V))

S       = CovNW(x.*u,0)         #S acccording to Newey-West, 0 lags
V       = inv(Sxx)'S*inv(Sxx)
std_NW0 = sqrt.(diag(V))

printblue("Coefficients and standard errors (from different methods):\n")
xx = hcat(b,b./std_iid,b./std_W,b./std_NW,b./std_NW0)
printmat(xx,colNames=["b","t (iid)","t (White)","t (NW 2 lags)","t (NW 0 lags)"],rowNames=xNames,width

printred("Remark: NW with 0 lags should be the same as White's method")
```

```
Coefficients and standard errors (from different methods):

                   b        t (iid)      t (White)   t (NW 2 lags)   t (NW 0 lags)
c             -0.052         -0.413         -0.412         -0.414         -0.412
Market         1.237         45.336         39.016         32.944         39.016

Remark: NW with 0 lags should be the same as White's method
```

```
V_alt = OlsNW(y,x,2)[4]          #comparison with another OLS function, used later
printmat(b./sqrt.(diag(V_alt))) #notice the lag order (2)
```

```
   -0.414
   32.944
```

A Convenience Function for Printing the Tests (extra)

```
DiagnosticsNoniidTable(x,u,3,xNames)
```

```
White's test (H₀: heteroskedasticity is not correlated with regressors)
0.009228390534838815
stat      6.147
p-val     0.046

Testing autocorrelation of residuals (lag 1 to 3)
lag  autocorr    t-stat     p-val
1      -0.039    -0.994     0.320
2       0.016     0.413     0.680
3       0.023     0.600     0.548

BoxPierce (3 lags)
stat      1.519
p-val     0.678

DW statistic
     2.074

Autocorrelations of c*u  (lag 1 to 3)
lag  autocorr    t-stat     p-val
1      -0.039    -0.994     0.320
2       0.016     0.413     0.680
3       0.023     0.600     0.548

Autocorrelations of Market*u  (lag 1 to 3)
lag  autocorr    t-stat     p-val
1       0.190     4.882     0.000
2       0.194     4.972     0.000
3       0.186     4.773     0.000
```

OLS on a System of Regressions

This notebook illustrates how to estimate a system of regressions with OLS - and to test (coefficients) across the regressions.

Load Packages and Extra Functions

The key function `OlsSure()` is from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, LinearAlgebra, Distributions
```

Loading Data

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float
x = nothing
println("Re: ",size(Re),"\n","F: ",size(F))

y = copy(Re)                    #to get standard OLS notation, both
(T,n) = size(y)                 #time periods, no. assets
```

```
x = [ones(T) F[:,1]]           #constant, market excess return
k = size(x,2)

println("\nT and k: $T $k")
```

```
      Re:   (660, 2)
       F:   (660, 3)

T and k: 660 2
```

A Function for Joint Estimation of Several Regressions (OLS)

Consider the linear regressions

$y_{it} = x_t' \beta_i + u_{it}$,

where $i = 1, 2, .., n$ indicates n different dependent variables. The K regressors are the *same* across the n regressions. (This is often called SURE, Seemingly Unrelated Regression Equations.)

For the case of two regressions ($n = 2$), the variance-covariance matrix has the following structure. Stack the β coefficients into a vector (from equation 1 first, then from equation 2.) Then, the variance-covariance matrix is

$$\text{Var}\left(\begin{bmatrix}\hat{\beta}_1 \\ \hat{\beta}_2\end{bmatrix}\right) = \begin{bmatrix}S_{xx}^{-1} & \mathbf{0} \\ \mathbf{0} & S_{xx}^{-1}\end{bmatrix} \Omega \begin{bmatrix}S_{xx}^{-1} & \mathbf{0} \\ \mathbf{0} & S_{xx}^{-1}\end{bmatrix}$$

where

$$\Omega = \text{Var}\left(\sum_{t=1}^{T} \begin{bmatrix}x_t u_{1t} \\ x_t u_{2t}\end{bmatrix}\right)$$

Notice that $x_t u_{1t}$ is a vector with K elements (as many as there are regressors) and $x_t u_{2t}$ is similar. The Ω matrix is thus $2K \times 2K$. The `OlsSure` can do White's/Newey-West.

The case of n regressions (rather than 2) involves creating similar matrices. This is implemented in the `OlsSure()` function.

```
@doc2 OlsSure
```

```
OlsSure(Y,X,NWQ=true,m=0)
```

LS of `Y` on `X`; where `Y` is Txn, and `X` is the same for all regressions

Input

- `Y::Matrix`: Txn, the n dependent variables
- `X::Matrix`: Txk matrix of regressors (including deterministic ones)
- `NWQ:Bool`: if true, then Newey-West's covariance matrix is used, otherwise Gauss-Markov
- `m::Int`: scalar, bandwidth in Newey-West

Output

- `b::Matrix`: kxn, regression coefficients (one column for each `Y[:,i]`)
- `u::Matrix`: Txn, residuals Y - Yhat
- `Yhat::Matrix`: Txn, fitted values X*b
- `V::Matrix`: covariance matrix of θ=vec(b)
- `R²::Matrix`: 1xn matrix, R² values

```
#using CodeTracking
#println(@code_string OlsSure([1],[1]))   #print the source code
```

Using the Function

```
(b,u,yhat,V,R²) = OlsSure(y,x,true)
Stdb  = sqrt.(reshape(diag(V),2,n))    #V = Cov(vec(b)), in vec(b) 1:2 are for asset 1, 3:4 for asset 2,
tstat  = b./Stdb

printblue("CAPM regressions: α is the intecept, β the coeff on Rme\n")
assetNames = [string("asset ",i) for i=1:n]
xNames      = ["c","Market"]

println("coeffs")
printmat(b;colNames=assetNames,rowNames=["α","β"])

println("t-stats")
printmat(tstat;colNames=assetNames,rowNames=["α","β"])
```

```
CAPM regressions: α is the intecept, β the coeff on Rme

coeffs
    asset 1   asset 2
α    -0.052     0.231
```

```
β     1.237     0.520

t-stats
    asset 1   asset 2
α    -0.412     1.701
β    39.016    14.566
```

Testing Across Regressions

To test across regressions, we first stack the point estimates into a vector by `θ = vec(b)`.

The test below applies the usual χ^2 test, where

$H_0 : R\theta = q$,

where R is a $J \times k$ matrix and q is a J-vector. To test this, use

$(R\theta - q)'(RVR')^{-1}(R\theta - q) \xrightarrow{d} \chi^2_J$.

The R matrix clearly depends on which hypotheses that we want to test.

The next cell creates a matrix of coefficient names that will help us see how the results are organised.

```
bNames = fill("",2,n)       #matrix of coef names, subscript for the asset number
for i in 1:n
    bNames[:,i] = [string("α",'₀'+i),string("β",'₀'+i)]        #'₀'+1 to get ₁
end
printmat(bNames)
```

```
α₁        α₂
β₁        β₂
```

```
θ = vec(b)

printblue("stacking the coeffs into a vector:")
printmat(θ;rowNames=vec(bNames))

printblue("The VCV*10_000 of the stacked vector of coeffs:")
printmat(V*10_000;colNames=vec(bNames),rowNames=vec(bNames))
```

```
stacking the coeffs into a vector:
α₁    -0.052
β₁     1.237
```

```
α₂      0.231
β₂      0.520

The VCV*10_000 of the stacked vector of coeffs:
           α₁         β₁         α₂         β₂
α₁    160.636    -5.909   -71.306     3.245
β₁     -5.909    10.049     3.245    -4.382
α₂    -71.306     3.245   184.552   -12.496
β₂      3.245    -4.382   -12.496    12.759
```

Testing if All Intercepts Are 0

CAPM implies that all intercepts should be zero. The next cell sets up a joint test.

```
R = zeros(n,2*n)                          #are all intercepts == 0?
for i in 1:n
    R[i,(i-1)*2+1] = 1
end

printblue("The R matrix:")
hypNames = string.("hypothesis ",1:size(R,1),":")
printmat(R;colNames=bNames,rowNames=hypNames,width=4,prec=0)

J = size(R,1)
println("The number of hypotheses that we test: $J \n")
q = zeros(J)
printblue("R*vec(b) - q:")
printmat(R*θ-q;rowNames=hypNames)
```

```
The R matrix:
               α₁  β₁  α₂  β₂
hypothesis 1:   1   0   0   0
hypothesis 2:   0   0   1   0

The number of hypotheses that we test: 2

R*vec(b) - q:
hypothesis 1:    -0.052
hypothesis 2:     0.231
```

```
println("Joint test of all hypotheses")

Γ = R*V*R'
test_stat = (R*θ - q)'inv(Γ)*(R*θ - q)
critval = quantile(Chisq(J),0.9)          #10% critical value
printmat([test_stat,critval];rowNames=["test statistic","10% crit value"])
```

```
Joint test of all hypotheses
test statistic     2.997
10% crit value     4.605
```

Bin Scatter

This notebook illustrates bin scatter plots, which aim to investigate possible non-linearities.

Load Packages and Extra Functions

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics, Plots

default(size = (480,320),fmt = :png)
```

Loading Data

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)   #Date,HiTec,Utils,Market,SMB,HML

(Re,F) = (Float64.(x[:,2:3]),Float64.(x[:,4:end]))   #make sure data is Float
x = nothing

y  = Re[:,1]                    #to the notation used in comment, use HiTec
x₁ = F[:,1:2]                   #1st set of regressors (2), no intercept
x₂ = F[:,3];                #2nd set of regressors, which we focus on in the binscatter
```

Bin Scatter

A binscatter is a method studying whether a linear model is a reasonable approximation.

We want to assess whether y_t and a single variable x_{2t} are linearly related, controlling for the vector x_{1t}.

Consider the regression

$y_t = x_{1t}'\gamma + d_t'\beta + u_t$

where x_{1t} is a vector (possibly empty) of control variables and d_t is an $N \times 1$ dummy vector indicating whether the single variable x_{2t} belongs to each the bins. (Clearly, only one element in d_t is one and the rest are zero). A common choice is to use the (minimum, 10th percentile, 20th percentile,..., maximum) as bin boundaries.

If the β estimates form a linear pattern when plotted against the *bin midpoints*, then a linear regression is a reasonable choice.

```
@doc2 BinScatter
```

```
BinScatter(y,x₁,x₂,L=[],U=[],N=20,critval=1.645)
```

Do a regression `y = x₁'γ + d'β + u`, where `d` is an N-vector indicating membership in a certain `x₂` bin. Plotting `β` against those bins is a binscatter plot.

Input:

- `y::Vector`: dependent variable
- `x₁::VecOrMat`: control variables
- `x₂::Vector`: main regressor of interest
- `L::Vector`: lower bin boundaries, if [] then quantiles (see N)
- `U::Vector`: upper bin boundaries, if [] then quantiles (see N)
- `N::Vector`: number of quantiles, giving N+1 bins. Used if `L=U=[]`
- `critval::Vector`: for calculation of confidence band

Output

- `β::Vector`: N-vector of coeffs on the bin (x) dummies
- `stdβ::Vector`: N-vector of std of β
- `fnO::NamedTuple`: with (LU,confBand)

```
#using CodeTracking                #uncomment to see source code
#println(@code_string BinScatter([1],[1],[1]))
```

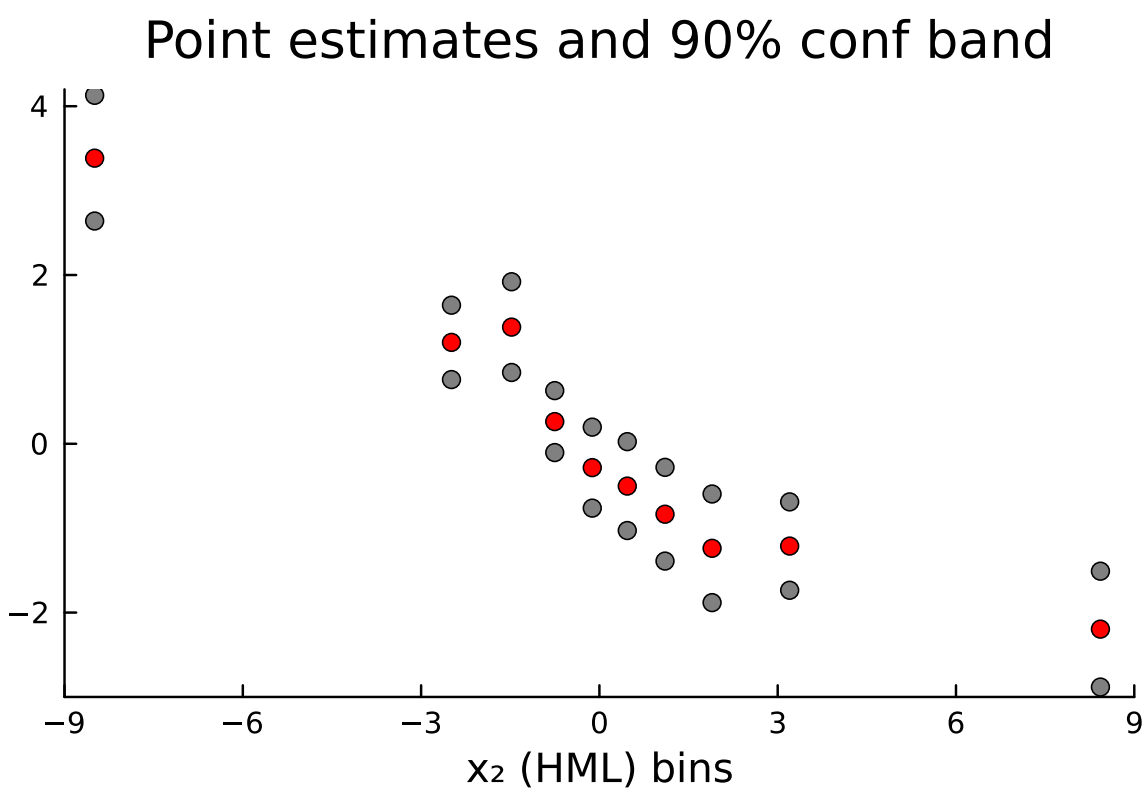
Using the Function

The next few cells calls on the function and plots the results

```
(β,stdβ,fnO) = BinScatter(y,x₁,x₂,[],[],10)  #use min,10th,20th,... perctiles as bins

(LU,cfB) = (fnO.LU,fnO.confBand)
LUmid = mean(LU,dims=2);                      #mid point of the bins
```

```
p1 = scatter( LUmid,β,
              xlims = (-9,9),
              ylims = (-3,4.2),
              title = "Point estimates and 90% conf band",
              xlabel = "x₂ (HML) bins",
              markercolor = :red,
              legend = false,
              grid = false )
scatter!( LUmid,cfB[:,1],markercolor = :grey )
scatter!( LUmid,cfB[:,2],markercolor = :grey )
#vline!(vcat(LU[1,1],LU[:,2]),linecolor=:black,line=(:dash,0.5))
display(p1)
```


Point estimates and 90% conf band
4
2
0
−2
−9
−6
−3
0
3
6
9
x₂ (HML) bins

Binary Choice (using NLS)

This notebook shows a simple example binary choice models, estimating by OLS/NLS.

Load Packages and Extra Functions

The general NLS functions are from the (local) `FinEcmt_MLEGMM` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
using FinEcmt_MLEGMM: NLS
using FinEcmt_ProbitTobit
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_MLEGMM.jl"))
using .FinEcmt_OLS
using .FinEcmt_MLEGMM: NLS
using FinEcmt_ProbitTobit
=#
```

```
using DelimitedFiles, Statistics
```

Loading Data

```
yx = readdlm("Data/Recessions.csv",',',skipstart=1)   #start on line 2, column 1
(y,x) = (yx[:,1],yx[:,2:end])
(T,k) = size(x)
```

```
(659, 4)
```

Linear Probability Model

```
xNames = ["yield curve slope(t-1)","market return(t-1)","recession(t-1)","c"]

bOls = x\y
(b1,StdErr1,V1,yhat1) = NLS(LpmRegrF,LpmdRegrF,y,x,bOls,0)

printmat(bOls,b1,b1./StdErr1;colNames=["OLS","NLS","t"],rowNames=xNames)
```

```
                            OLS       NLS         t
yield curve slope(t-1)    -0.017    -0.017    -3.099
market return(t-1)        -0.549    -0.549    -3.004
recession(t-1)             0.879     0.879    26.377
c                          0.038     0.038     3.424
```

Probit

```
xMean = mean(x,dims=1)           #sample means, for marginal effects

(bProb,StdProb,V3,yhatProb) = NLS(ProbitRegrF,ProbitdRegrF,y,x,bOls,0)
meProb = ProbitMeF(bProb,xMean,[3])

(R2Prob,cTab) = BinaryChoiceR2pred(y.>0.5,yhatProb.>0.5)

printmat(bProb,bProb./StdProb,meProb;colNames=["b","t","marg. eff"],rowNames=xNames,width=14)
println("R2: ",R2Prob,"\n")
```

```
                                 b             t     marg. eff
yield curve slope(t-1)        -0.346        -2.113        -0.020
market return(t-1)            -6.153        -2.160        -0.347
recession(t-1)                 3.802         8.232         0.902
c                             -2.025        -8.858        -0.114

      R2:      0.821
```

```
(bLogit,StdLogit,V4,yhatLogit) = NLS(LogitRegrF,LogitdRegrF,y,x,bOls,0)
(R2Logit,cTab) = BinaryChoiceR2pred(y.>0.5,yhatLogit.>0.5)
meLogit = LogitMeF(bLogit,xMean,[3])

printmat(bLogit,bLogit./StdLogit,meLogit;colNames=["b","t","marg. eff"],rowNames=xNames,width=14)
printlnPs("R2: ",R2Logit,"\n")
```

```
                                    b             t     marg. eff
yield curve slope(t-1)         -0.665        -2.122        -0.017
market return(t-1)            -11.099        -1.939        -0.278
recession(t-1)                  6.861         7.491         0.901
c                              -3.673        -7.951        -0.092

      R2:      0.821
```

Asymptotics

…or rather: simulations to illustrate the properties of OLS as the sample size increases.

This notebook uses Monte Carlo simulations to illustrate the properties of OLS. This is *not* asymptotics, but it does hint at what happens in longer and longer samples.

Load Packages and Extra Functions

The key functions are from the (local) `FinEcmt_OLS` module. We also use one function from the `FinEcmt_TimeSeries` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
using FinEcmt_TimeSeries: ARMAFilter
```

```
using Statistics, Random, Distributions, LaTeXStrings, Plots
Random.seed!(123);   #random number generator starts on this point
```

A Function for a Monte Carlo Simulation

`SimulateOLS(CreateYXFn,T,NSim)` makes `NSim` simulations of samples `T` long, where data is created in a function `CreateYXFn(T)` (see below for examples).

```
"""
    SimulateOLS(CreateYXFn,T,NSim)

Simple function for Monte Carlo simulations of OLS. Data is created
by the `CreateYXFn(T)` function.

"""
```

```
function SimulateOLS(CreateYXFn::Function,T,NSim)
  (y,x) = CreateYXFn(T)
  k = size(x,2)
  b = fill(NaN,NSim,k)           #to put the results in
  for i in 1:NSim
    (y,x) = CreateYXFn(T)
    b[i,:] = x\y                 #OLS estimates
  end
  return b
end
```

```
SimulateOLS
```

Estimating an AR(1)

```
"""
    CreateYXFn_ARMA11(T,ρ,θ)

Create `y,x` for an ARMA(1,1). Set θ=0 to simulate an AR(1)

"""
function CreateYXFn_ARMA11(T,ρ,θ)       #to generate iid (y,x)
  Tb = T + 100                          #100 extra observations, will be cut
  y = ARMAFilter(randn(Tb),[ρ],[θ])     #simulates an ARMA(1)
  x = [ones(T) y[100:end-1]]            #x starts at t=100, y at t=101
  y = y[101:end]
  return y,x
end
```

```
CreateYXFn_ARMA11
```

```
NSim = 3000      #number of simulations, increase to get more precise results

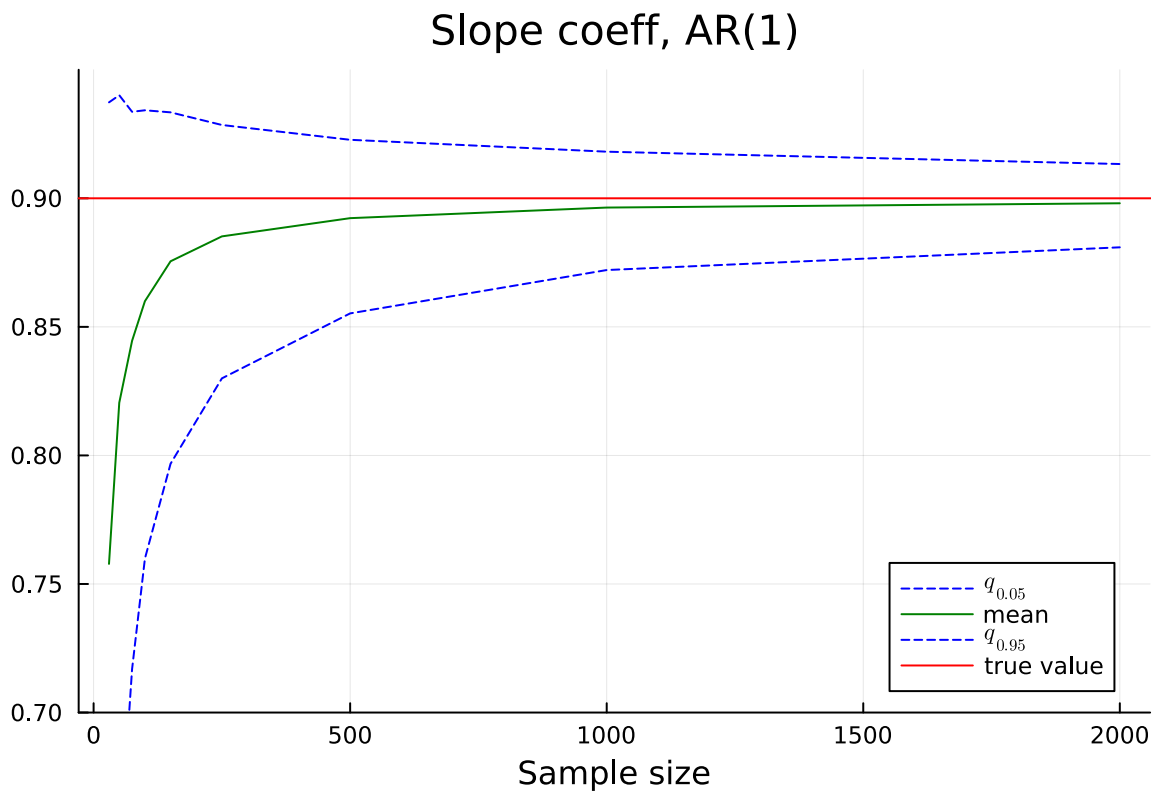
ρ = 0.9
TM = [30,50,75,100,150,250,500,1000,2000]       #calculate for different sample sizes

bM = fill(NaN,NSim,length(TM))
for (i,T) in enumerate(TM)
  local bHat
```

```
  bHat = SimulateOLS(z -> CreateYXFn_ARMA11(z,ρ,0),T,NSim)    #0=0 to get AR(1)
  bM[:,i] = bHat[:,2]                    #save the slope coef
end

q05 = [quantile(z,0.05) for z in eachcol(bM)]     #quantiles and mean of estimates
q95 = [quantile(z,0.95) for z in eachcol(bM)]
meanb = mean(bM,dims=1)'

p1 = plot( TM,[q05 meanb q95],
           ylims = [0.7,0.95],
           label = [L"q_{0.05}" "mean" L"q_{0.95}"],
           linecolor = [:blue :green :blue],
           linestyle = [:dash :solid :dash],
           title = "Slope coeff, AR(1)",
           xlabel = "Sample size" )
    hline!([0.9],linecolor=:red,line=:solid,label="true value")
display(p1)
```



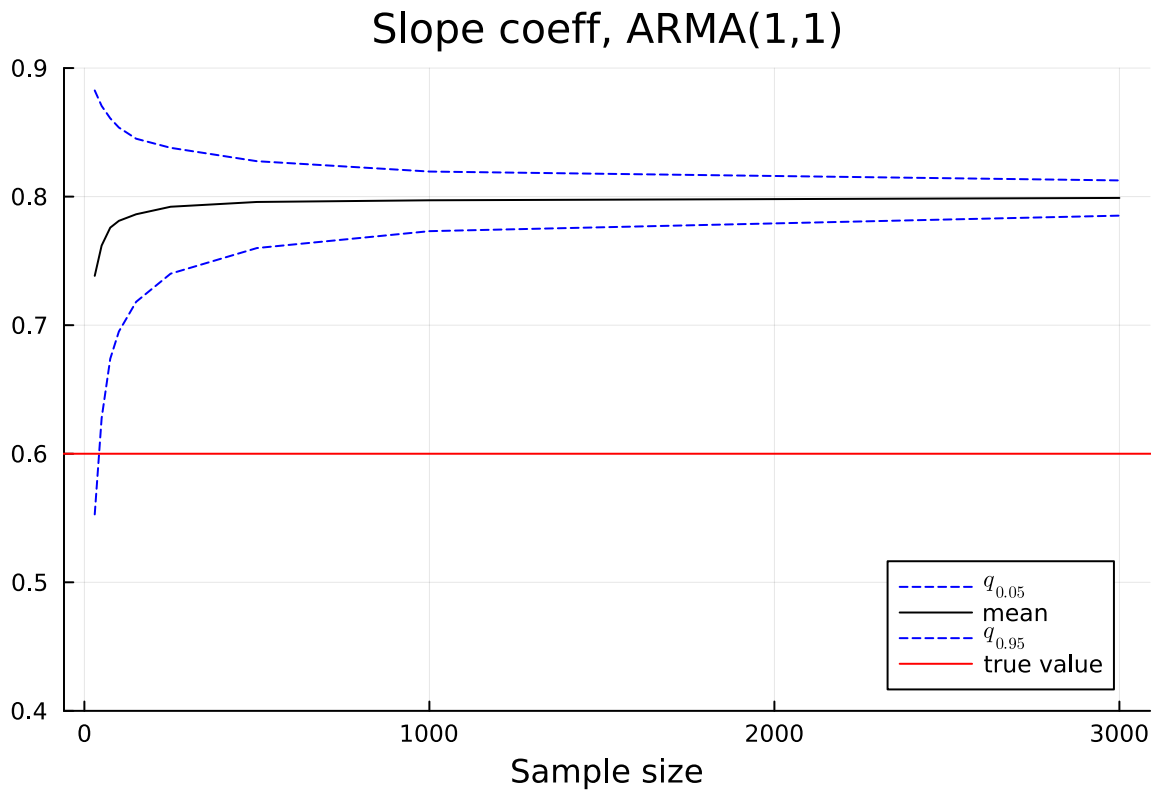
Estimating an AR(1) when Data is follows an ARMA(1,1)

```
(ρ,θ) = (0.6,0.9)
TM = [30,50,75,100,150,250,500,1000,3000]       #calculate for different sample sizes

bM = fill(NaN,NSim,length(TM))
for (i,T) in enumerate(TM)
  local bHat
  bHat = SimulateOLS(z -> CreateYXFn_ARMA11(z,ρ,θ),T,NSim)
  bM[:,i] = bHat[:,2]                   #save the slope coef
end

q05 = [quantile(z,0.05) for z in eachcol(bM)]
q95 = [quantile(z,0.95) for z in eachcol(bM)]
meanb = mean(bM,dims=1)'

p1 = plot( TM,[q05 meanb q95],
           ylims = [0.4,0.9],
           label = [L"q_{0.05}" "mean" L"q_{0.95}"],
           linecolor = [:blue :black :blue],
           linestyle = [:dash :solid :dash],
           title = "Slope coeff, ARMA(1,1)",
           xlabel = "Sample size")
     hline!([0.6],linecolor=:red,line=:solid,label="true value")
display(p1)
```



Omitted Variables

These simulations illustrate what happens to the estimate slop coefficient on x when the variable h is omitted. The results depend on the correlation, or rather the regression coefficient θ in $h = \theta x + e$.

```
"""
    CreateYXFn_omitted(T,b)

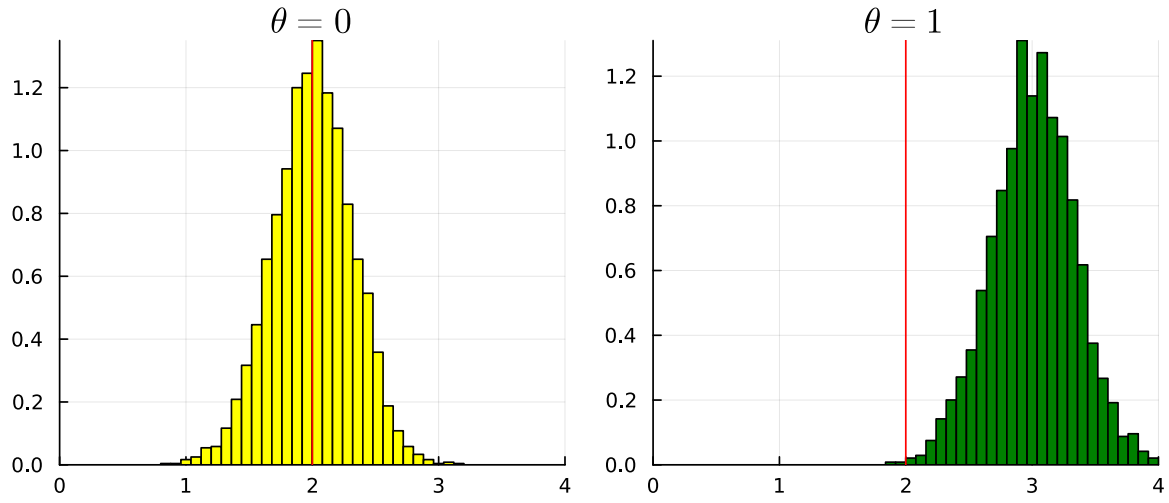
Create `y,x` when there is an omitted variable `h`

"""
function CreateYXFn_omitted(T,b,θ)
  x = [ones(T) randn(T)*sqrt(0.5)]
  h = θ*x[:,2] + randn(T)
  v = randn(T)*sqrt(4)
  y = x*b + h + v
  return y,x
end
```

CreateYXFn_omitted

```
(T,b) = (100,[1,2])
bM0 = SimulateOLS(z -> CreateYXFn_omitted(T,b,0),T,NSim)[:,2]
bM1 = SimulateOLS(z -> CreateYXFn_omitted(T,b,1),T,NSim)[:,2]

p1 = histogram( bM0,bins=range(0,4,length=51),
                normalized = true,
                legend = false,
                color = :yellow,
                xlim = (0,4),
                title = L"\theta=0",
                guidefontsize = 8 )
     vline!([2],linecolor=:red,line=:solid)
p2 = histogram( bM1,bins=range(0,4,length=51),
                normalized = true,
                legend = false,
                color = :green,
                xlim = (0,4),
                title = L"\theta=1",
                guidefontsize = 8 )
     vline!([2],linecolor=:red,line=:solid)
pAll = plot(p1,p2,layout=(1,2),size=(700,300))
display(pAll)
```



Measurement Errors in Regressors

These simulations illustrate what happens to the estimate slop coefficient on x when it is just proxy (with errors) of the correct regressor.

The results depend on the measurement error in x, which has the standard deviation σ_e.

```
"""
    CreateYXFn_measurement(T,b)

Create `y,x` when there is an omitted variable

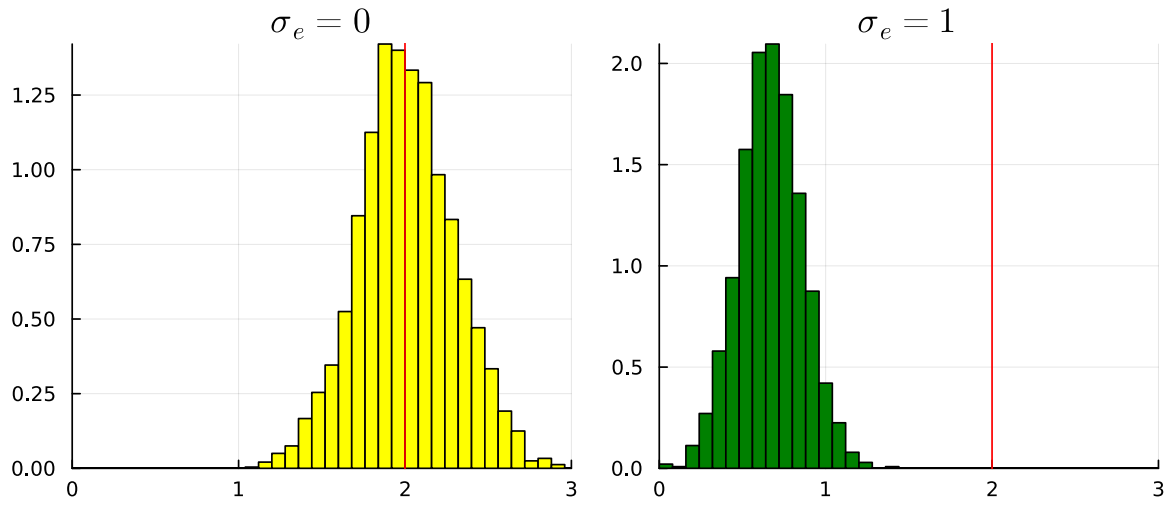
"""
function CreateYXFn_measurement(T,b,σₑ)
  w = [ones(T) randn(T)*sqrt(0.5)]
  v = randn(T)*sqrt(4)
  y = w*b + v
  x = w .+ [zeros(T) randn(T)*σₑ]
  return y,x
end
```

```
CreateYXFn_measurement
```

```
(T,b) = (100,[1,2])
bM0 = SimulateOLS(z -> CreateYXFn_measurement(T,b,0),T,NSim)[:,2]
bM1 = SimulateOLS(z -> CreateYXFn_measurement(T,b,1),T,NSim)[:,2]

p1 = histogram( bM0,bins=range(0,4,length=51),
                normalized = true,
                legend = false,
                color=:yellow,
                xlim = (0,3),
                title = L"\sigma_e=0",
                guidefontsize = 8 )
     vline!([2],linecolor=:red,line=:solid)
p2 = histogram( bM1,bins=range(0,4,length=51),
                normalized = true,
                legend = false,
                color=:green,
                xlim = (0,3),
                title = L"\sigma_e=1",
                guidefontsize = 8 )
```

```
    vline!([2],linecolor=:red,line=:solid)
pAll = plot(p1,p2,layout=(1,2),size=(700,300))
display(pAll)
```



(CLT) Non-Normal Residuals

to get a sense of what the CLT does. The residuals are $u = v - 2$, where $v \sim \chi^2_2$.

```
"""
    CreateYXFn_Chisq(T,b)

Create `y,x` when the residual has a chisq(2) distribution

"""
function CreateYXFn_Chisq(T,b)          #to generate iid (y,x)
  k = length(b)
  x = [ones(T) 10*randn(T,k-1)]
  u = rand(Chisq(2),T) .- 2
  y = x*b + u
  return y,x
end
```

```
CreateYXFn_Chisq
```

```
b  = [0,0.9]
TM = [5,100]

bM = fill(NaN,NSim,length(TM))
for (i,T) in enumerate(TM)
  local bHat
  bHat = SimulateOLS(z -> CreateYXFn_Chisq(z,b),T,NSim)
  bM[:,i] = bHat[:,2]
end

xGrid = range(-1,1,length=51)
pdfX1 = pdf.(Normal(mean(bM[:,1].-0.9),sqrt(5)*std(bM[:,1])),xGrid)
pdfX2 = pdf.(Normal(mean(bM[:,2].-0.9),sqrt(100)*std(bM[:,2])),xGrid)

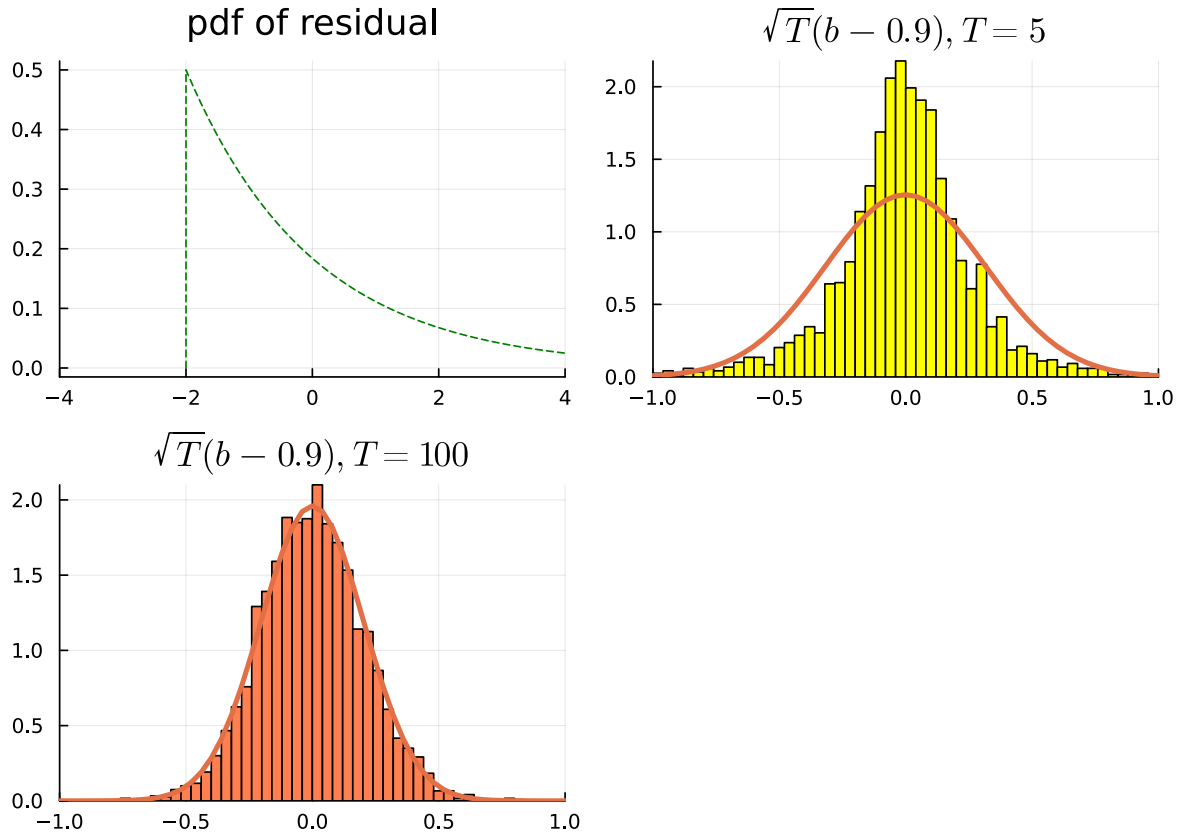
zGrid = range(0,6,length=51)
pdfu = pdf.(Chisq(2),zGrid)


p0 = plot( [-2;zGrid.-2],[0;pdfu],
           xlims = [-4,4],
           color = :green,
           linestyle = :dash,
           legend = false,
           title = "pdf of residual" )

p1 = histogram( sqrt(5)*(bM[:,1].-0.9),bins=range(-1,1,length=51),
                color = :yellow,
                normalized = true,
                legend = false,
                xlim = (-1,1),
                title = L"\sqrt{T}(b-0.9), T=5",
                guidefontsize = 8 )
plot!(xGrid,pdfX1,linewidth=3)

p2 = histogram( sqrt(100)*(bM[:,2].-0.9),bins=range(-1,1,length=51),
                color = :coral,
                normalized = true,
                legend = false,
                xlim = (-1,1),
                title = L"\sqrt{T}(b-0.9), T=100",
                guidefontsize = 8 )
plot!(xGrid,pdfX2,linewidth=3)
```

```
pAll = plot(p0,p1,p2,layout=(2,2),size=(700,500))                    #set up subplots
display(pAll)
```



Monte Carlo Simulations

This notebook shows examples of Monte Carlo simulations for: *(a)* the distribution from a GARCH process; *(b)* the effect of heteroskedasticity on OLS standard errors.

Load Packages and Extra Functions

The key functions for simulations are coded in this notebook. However, the function for OLS is from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using Random, LinearAlgebra, Distributions
```

```
using Plots, LaTeXStrings
default(size = (480,320),fmt = :png)
```

Simulating the Distribution of a GARCH(1,1) Variable

The `GarchSim(T,ω,α,β)` function (see below) generates T observations of a variable u_t whose conditional distribution (based on information in $t-1$) is $N(0,\sigma_t^2)$, where σ_t^2 follows a GARCH(1,1) process:

$\sigma_t^2 = \omega + \alpha u_{t-1}^2 + \beta\sigma_{t-1}^2$.

The subsequent cells call on the `GarchSim()` function and plots a histogram of the simulated values. This is repeated for two different values of α: a high value and a low value.

```
"""
    GarchSim(T,ω,α,β)

Simulate a time series of T residuals from a GARCH(1,1) process.

### Remark
- The vector of σ² values is not exported from the function.
If needed, this could easily be changed.

"""
function GarchSim(T,ω,α,β)
    (σ²,u) = [zeros(T) for i=1:2]
    σ²[1]  = ω/(1-α-β)                              #average σ² as starting value
    for t in 2:T
        σ²[t] = ω + α*u[t-1]^2 + β*σ²[t-1]
        u[t]  = sqrt(σ²[t])*randn()
    end
    return u
end
```

```
GarchSim
```

```
(T,ω,β) = (200_000,1,0.8)
(αH,αL) = (0.19,0.09)                             #fairly high and low α values

Random.seed!(123456)                             #to replicate the same random numbers
u = GarchSim(T,ω,αH,β)
uGrid = range(-30,30,length=101)                  #grid of values
pdfNu = pdf.(Normal(mean(u),std(u)),uGrid)        #best fitting N() as a comparison

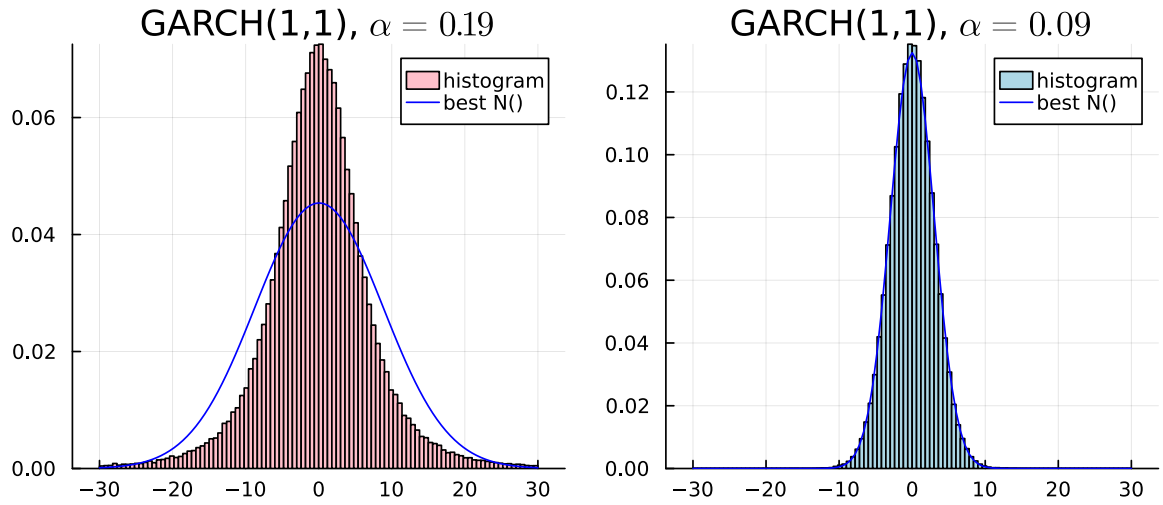
ub = GarchSim(T,ω,αL,β)
pdfNub = pdf.(Normal(mean(ub),std(ub)),uGrid)

p1 = histogram( u,
                bins = uGrid,
                fillcolor = :pink,
                normalized = true,
                label = "histogram",
                title = L"GARCH(1,1), $\alpha = %$αH$" )
```

```
plot!(uGrid,pdfNu,color=:blue,label="best N()")

p2 = histogram( ub,
                bins = uGrid,
                fillcolor = :lightblue,
                normalized = true,
                label = "histogram",
                title = L"GARCH(1,1), $\alpha = %$αL$" )
plot!(uGrid,pdfNub,color=:blue,label="best N()")

pAll = plot(p1,p2,layout=(1,2),size=(700,300))                #set up subplots
display(pAll)
```



Simulating (White's) Heteroskedasticity in OLS

The `SimOL(NSim,T,α)` function (see below) first simulates `T` observations where the residuals in an OLS regression are heteroskedastic, since they are generated as

$\epsilon_t = u_t \sigma_t$, where $\sigma_t^2 = (1 + \alpha |x_t - 2|)^2$

where u_t is $N(0,1)$ and x_t is a regressor distributed as $N(2,1)$.

The function then estimates the traditional OLS standard errors and also White's standard errors.

The subsequent cell compares the different standard errors in two cases: $\alpha = 0$ and $\alpha = 1$.

```
"""
    SimOLS(NSim,T,α)

Simulate data for a regression model with heteroskedastic errors and then estimate both point estimates
and standard errors according to traditional OLS (Gauss-Markov) and White.

### Input
- `NSim::Int`:    number of simulations (eg. 3000)
- `T::Int`:       sample length (eg. 200)
- `α::Number`:    degree of heteroskedasticity

"""
function SimOLS(NSim,T,α)

    (bLS,StdLS,StdWhite) = [fill(NaN,NSim,2) for i = 1:3]
    for i in 1:NSim
        x = randn(T) .+ 2                    #some random regressors
        σ² = (1 .+ α*abs.(x.-2)).^2
        ϵ = randn(T) .* sqrt.(σ²)            #heteroskedastic residuals
        y = 1 .+ 0.9*x + ϵ
        x = [x ones(T)]
        (b,u,) = OlsGM(y,x)                  #OLS, point estimates
        bLS[i,:] = b
        Sxx           = x'x
        S             = (x.*u)'*(x.*u)
        V_W           = inv(Sxx)'S*inv(Sxx)      #Cov(b), White
        StdWhite[i,:] = sqrt.(diag(V_W))
        V_iid         = inv(Sxx)*var(u)          #OLS, traditional
        StdLS[i,:]    = sqrt.(diag(V_iid))
    end

    return bLS, StdLS, StdWhite

end
```

SimOLS

```
(T,NSim) = (200,25000)           #number of simulated samples
Random.seed!(123456)

α = 0                            #no heteroskedasticity
(bLS,StdLS,StdWhite) = SimOLS(NSim,T,α)
```

```
αH = 1                          #lots of heteroskedasticity
(bLSb,StdLSb,StdWhiteb) = SimOLS(NSim,T,αH)


colNames = ["Simulated","OLS","White"]
rowNames=["slope","intercept"]
printblue("Standard errors when α=$(α):")
printmat(std(bLS,dims=1)',mean(StdLS,dims=1)',mean(StdWhite,dims=1)';colNames,rowNames)

printblue("Standard errors when α=$(αH):")
printmat(std(bLSb,dims=1)',mean(StdLSb,dims=1)',mean(StdWhiteb,dims=1)';colNames,rowNames)

printred("The standard errors from different methods are similar when α=0, but different otherwise")
```

```
Standard errors when α=0:
          Simulated       OLS     White
slope         0.071     0.071     0.070
intercept     0.159     0.158     0.157

Standard errors when α=1:
          Simulated       OLS     White
slope         0.190     0.134     0.185
intercept     0.403     0.299     0.392

The standard errors from different methods are similar when α=0, but different otherwise
```

Bootstrapping

This notebook implements both a traditional bootstrap and a block bootstrap in order to get more robust standard errors of OLS coefficients.

Load Packages and Extra Functions

The key functions (for doing OLS and block bootstrap) are from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics, LinearAlgebra, Random
```

Loading Data

The data are for

```
x = readdlm("Data/TwoIndustries.csv",',',skipstart=1)

(dN,Re,F) = (x[:,1],Float64.(x[:,2:3]),Float64.(x[:,4:end]))  #make sure data is Float
x = nothing
println1Ps("Re: ",size(Re),"\n","F: ",size(F))

y = Re[:,1]
```

```
T = length(y)
x = [ones(T) F[:,1]]           #constant, market excess return
k = size(x,2)

println("\nT and k: $T $k")
```

```
      Re:   (660, 2)
       F:   (660, 3)

T and k: 660 2
```

Point Estimates and Standard Errors

```
(bLS,u,yhat,Covb,) = OlsGM(y,x)            #OLS estimate and traditional std errors
t_iid = bLS./sqrt.(diag(Covb))

CovW = OlsNW(y,x,0)[4]                    #White's VCV
t_W  = bLS./sqrt.(diag(CovW))
CovNW = OlsNW(y,x,2)[4]                   #Newey-West VCV
t_NW  = bLS./sqrt.(diag(CovNW))

printblue("OLS estimates:\n")
#rowNames = [string("x",'₀'+i) for i=1:k]      #'₀'+1 to get ₁
rowNames = ["c","slope"]
printmat(bLS,t_iid,t_W,t_NW;colNames=["coeff","t (iid)","t (White)","t (NW, 2)"],rowNames=rowNames,w
```

```
OLS estimates:

               coeff        t (iid)      t (White)      t (NW, 2)
c             -0.052         -0.413         -0.412         -0.414
slope          1.237         45.336         39.016         32.944
```

Standard Bootstrap (1)

In each loop, a new series of residuals, $\tilde{u}_t$, is created by drawing (with replacement) values from the fitted residuals (from the estimates in earlier cells). Then, simulated values of the dependent variable are created as

$\tilde{y}_t = x_t'\beta + \tilde{u}_t$

and we redo the estimation on $(\tilde{y}_t, x_t)$. Notice that x_t is the same as in the data.

This is repeated `NSim` times.

```
@doc2 OlsBootstrap
#using CodeTracking
#println(@code_string OlsBootstrap([],[],[],1,1))    #print the source code
```

```
OlsBootstrap(y,x,bLS,BlockSize,NSim,ExciseIt=false;yxPairType=0)
```

Do a bootstrap of OLS regression, w/wo blocks and of residual/pairs for (y,x)

Input

- `y::Array`: Txn matrix of n dependent variables
- `x::Array`: TxK matrix of (common) regressors
- `bLS::Array`: OLS estimates (will be generated if not supplied)
- `BlockSize::Int`: scalar, size of blocks for block bootstrap (eg. 1 for no blocks, 20 for long blocks)
- `NSim::Int`: scalar, number of simulations
- `ExciseIt::Bool`: true for excise on [y,x]; default: false
- `yxPairType::Int`: 1 for drawing pairs of (y(s),x(s)); 2 for wild bootstrap; else residuals are drawn; default: 0

Output

- `CovbBoot::Array`: K*n x K*n covariance matrix of vec(b)
- `bBoot::Array`: NSim x K*n matrix, vec(b) in row i

Requires

- FindNN

```
NSim      = 3000                #no. of simulations
Random.seed!(123)

(CovbBoot,bBoot) = OlsBootstrap(y,x,[],1,NSim)  #resampling residuals, no adjustment
```

```
printblue("Coefficients:")
xx = [bLS  mean(bBoot,dims=1)']
printmat(xx;colNames=["OLS","avg. bootstr"],rowNames=rowNames,width=20)

printblue("t-stats:")
xx = [t_iid bLS./sqrt.(diag(CovbBoot))]
printmat(xx;colNames=["t (iid)","t (bootstrap 1)"],rowNames=rowNames,width=20)

printred("The results from these bootstrap are similar to standard iid OLS results, which is not good.")
```

```
Coefficients:
                       OLS        avg. bootstr
c                   -0.052              -0.047
slope                1.237               1.237

t-stats:
                   t (iid)     t (bootstrap 1)
c                   -0.413              -0.416
slope               45.336              45.772

The results from these bootstrap are similar to standard iid OLS results, which is not good.
```

Pairwise Bootstap (2)

to handle heteroskedasticity.

```
CovbBoot, = OlsBootstrap(y,x,[],1,NSim;yxPairType=1)    #pairwise resampling

printblue("t-stats:")
xx = [t_W  bLS./sqrt.(diag(CovbBoot))]
printmat(xx;colNames=["t (White's)","t (bootstrap 2)"],rowNames=rowNames,width=20)

printred("The results from these bootstrap are similar to White's standard error, which is better.")
```

```
t-stats:
              t (White's)     t (bootstrap 2)
c                   -0.412              -0.414
slope               39.016              39.116

The results from these bootstrap are similar to White's standard error, which is better.
```

Block Bootstrap & Pairwise (3)

To handle autocorrelations and heteroskedasticity, we do a block bootstrap of the (y_t, x_t) pairs.

```
CovbBoot, = OlsBootstrap(y,x,[],5,NSim;yxPairType=1)    #pairwise, blocks of 5

printblue("t-stats:")
xx = [t_NW  bLS./sqrt.(diag(CovbBoot))]
printmat(xx;colNames=["t (NW, 2)","t (bootstrap 3)"],rowNames=rowNames,width=20)

printred("The t-stats are even lower than from NW, which may suggest that NW needs more lags?")
```

```
t-stats:
                t (NW, 2)     t (bootstrap 3)
c                  -0.414              -0.403
slope              32.944              29.267

The t-stats are even lower than from NW, which may suggest that NW needs more lags?
```

Portfolio Sorts

This notebook implements univariate and bivariate portfolio sorts.

Load Packages and Extra Functions

The key functions used in this notebook are from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
using FinEcmt_TimeSeries: EMA
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_TimeSeries.jl"))
using .FinEcmt_OLS
using .FinEcmt_TimeSeries: EMA
=#
```

```
using Dates, DelimitedFiles, Statistics
```

Load Data

The data set contains daily data for "dates", the equity market return, riskfree rate and the the returns of the 25 Fama-French portfolios. All returns are in percent.

```
x    = readdlm("Data/MomentumSR.csv",',')
dN   = Date.(x[:,1],"yyyy-mm-dd")                  #Julia dates
x    = Float64.(x[:,2:end])
(Rm,Rf,R) = (x[:,1],x[:,2],x[:,3:end])
```

```
println("size of dN, Rm, Rf, R")
println(size(dN),"\n",size(Rm),"\n",size(Rf),"\n",size(R))

Re  = R .- Rf              #excess returns
Rme = Rm .- Rf

(T,n) = size(Re);          #number of periods and assets
```

```
size of dN, Rm, Rf, R
(11598,)
(11598,)
(11598,)
(11598, 25)
```

A Univariate Sort

on recent excess returns.

The next cells create a sorting variable `X` and does a portfolio sort to create a `Lo` portfolio and a `Hi` portfolio (corresponding to the 5 lowest and highest values of `X`). The sort is done for each trading day of the sample, so the portfolios are dynamic (has time-varying portfolio weights).

Since we are working with excess returns, all portfolio returns are themselves excess returns.

A Remark on the Code

- The `EMA(Re,22)` function (included above) calculates a moving average of `R[t-21:t,i]` for each column `i`. We then lag thus result using `lag()`. For `t<22` it instead calculates the MA of `R[1:t,i]`.
- With `x = [9,7,8]`, the `rankPs(x)` function (included above) gives the output `[3,1,2]`. This says, for instance, that 7 is the lowest number (rank 1).

```
m = 5                      #number of assets in each of Lo and Hi portfolio
q = 22
x = lag(EMA(Re,q));        #(lag of) moving average of R, to get a trading signal

#println("rows 2-4 of x*10:")
#printmat(x[2:4,:]*10,width=5,prec=2,colNames=string.(1:n))
println()
```

```
(RHi,RLo) = (fill(NaN,T),fill(NaN,T))
for t in 2:T         #loop over periods, save portfolio returns
    #local r, wHi, wLo          #local/global is needed in script
    r      = rankPs(x[t,:])       #x is lagged already
    wLo    = (r.<=m)/m            #low rank: in Lo portfortolio
    wHi    = ((n-m+1).<=r)/m      #high rank: in Hi portfolio
    RLo[t] = wLo'Re[t,:]
    RHi[t] = wHi'Re[t,:]
end

Rp = mean(Re,dims=2);         #'passive' portfolio, equal weight on all assets
```

Return Statistics

…comparing with a passive portfolio.

Excess returns for the `Lo`, `Hi` and passive (`p`) portfolios are calculated.

We then use the `ReturnStats(Re,Annfactor)` function (included above) to calculate the mean (excess) return, its standard deviation and the Sharpe ratio. Annualisation is done by assuming 252 trading days per year.

In calculating the return stats, we drop the first observation, since there is no trading signal for `t=1`.

```
Stats = ReturnStats([RLo RHi (RHi-RLo) Rp][2:end,:],252)    #return stats for obs 2 to T

printblue("Stats for the portfolio returns, annualized:\n")
printmat(Stats;colNames=["Lo","Hi","Hi-Lo","passive"],rowNames=["μ";"σ";"SR"])
```

```
Stats for the portfolio returns, annualized:

          Lo        Hi     Hi-Lo   passive
μ      4.085    13.714     9.628     9.746
σ     20.387    18.429    11.196    18.119
SR     0.200     0.744     0.860     0.538
```

A Univariate Sort (again)

on recent returns - but using another approach, which turns out to be easy to apply to the double sort as well.

A Remark on the Code

- The `sortLoHi(x,v,m)` function (included above) below create n-vectors `vL` and `vH` with trues/falses indicating membership of the Lo and Hi groups.
- As an example, `(vL,vH) = sortLoHi([3,1,2],[false,true,true],1)` gives `vL=[false,true,false]` and `vH=[false,false,true]`. This means that the 'Lo' portfolio consists of asset 2 and the 'Hi' portfolio of asset 3. Asset 1 is not assigned to any.
- The `EWportf(v)` function (also from `FinEcmt_OLS`) takes such a Bool vector (eg. `vL`) and calculates (equal) portfolio weights among those assets that are `true` in `v`. It handles the case of an empty portfolio (all elements in `v` are false) by setting all portfolio weights to `NaN`.

```
@doc2 sortLoHi
```

```
sortLoHi(x,v,m)
```

Create vectors `vL` and `vH` with trues/falses indicating membership of the Lo and Hi groups. It sorts according to `x[v]`, setting the m lowest (in `vL`) and m highest values (in `vH`) to `true`. All other elements (also those in `x[.!v]`) are set to false.

Input

- `x::Vector`: n-vector, sorting variable
- `v::Vector`: n-vector of true/false. Sorting is done within x[v]
- `m::Int`: number of assets in Lo/Hi portfolio

Output

- `vL::Vector`: n-vector of true/false, indicating membership of Lo portfolio
- `vH::Vector`: n-vector of true/false, indicating membership of Hi portfolio

```
#using CodeTracking
#println(@code_string sortLoHi([1],trues(1),1))    #print the source code
```

```
x = lag(EMA(Re,q))                        #sort on lag of MA(q) of R

(RH,RL) = (fill(NaN,T),fill(NaN,T))
for t in 2:T                    #loop over periods, save portfolio returns
    #local vL,vH,wL,wH          #local/global is needed in script
    (vL,vH) = sortLoHi(x[t,:],trues(n),m)
```

```
    (wL,wH) = (EWportf(vL),EWportf(vH))       #portfolio weights, EW
    RL[t]   = wL'Re[t,:]
    RH[t]   = wH'Re[t,:]
end

Stats = ReturnStats([RL RH][2:end,:],252)            #RL[1] = NaN, so start at t=2
printblue("Stats for the portfolio returns, annualized:\n")
printmat(Stats,colNames=["Lo","Hi"],rowNames=["μ";"σ";"SR"])

printred("Compare with the previous results to verify that they are the same")
```

```
Stats for the portfolio returns, annualized:

          Lo        Hi
μ      4.085    13.714
σ     20.387    18.429
SR     0.200     0.744

Compare with the previous results to verify that they are the same
```

An Independent Double Sort

on recent volatility (X) and and returns (Z).

This creates four portfolios: (Low X, Low Z), (Low X, High Z), (High X, Low Z) and (High X, High Z). Each of them is an intersection from independent sorts on X and Z.

The size of each of these portfolios can vary over time: sometimes the portfolio is empty. The portfolio weights created by the `EWportf()` function are then `NaN`.

A Remark on the Code

- `vXL .& vZL` tests if `vXL[i]` and `vZL[i]` are both true (and then repeats for each `i`).

```
X = lag(EMA(abs.(Re),q))                #(lag of) MA of |R|, first sort variable
Z = lag(EMA(Re,q));                     #(lag of) MA of R, second sort variable
```

```
(mX,mZ) = (10,10)

(RLL,RLH,RHL,RHH) = [fill(NaN,T) for i=1:4]
for t in 2:T                    #loop over periods, save portfolio returns
   #local vXL,vXH,vZL,vZH,vLL,vLH,vHL,vHH,wLL,wLH,wHL,wHH     #local/global is needed in script
    (vXL,vXH) = sortLoHi(X[t,:],trues(n),mX)       #in Lo/Hi according to X
    (vZL,vZH) = sortLoHi(Z[t,:],trues(n),mZ)       #in Lo/Hi according to Z
    vLL     = vXL .& vZL                                 #in Lo X,Low Z
    vLH     = vXL .& vZH                                 #in Lo X, Hi Z
    vHL     = vXH .& vZL                                 #in Hi X, Lo Z
    vHH     = vXH .& vZH                                 #in Hi X, Hi Z
   (wLL,wLH,wHL,wHH) = (EWportf(vLL),EWportf(vLH),EWportf(vHL),EWportf(vHH)) #portfolio weights, EW
   (RLL[t],RLH[t],RHL[t],RHH[t]) = (wLL'Re[t,:],wLH'Re[t,:],wHL'Re[t,:],wHH'Re[t,:])
end
```

Handling NaNs and Reporting Results

There are NaNs in the return series, since the portfolios are sometimes empty. We need to decide on how to handle those data points.

The assumption below is that an empty portfolio (when all weights are `NaNs`) means that the full investment is done in the riskfree asset and thus the excess return is zero. We thus replace the excess return of a `NaN` return with zero.

We typically want to study `RLH - RLL` and `RHH - RHL` since they show the "effect" of the second sort variable (`Z`) while controlling for the first (`X`).

```
ReAll = [RLL RLH RHL RHH]        #excess returns
replace!(ReAll,NaN⇒0)            #replace NaN by 0, assuming investment in riskfree
Stats = ReturnStats(ReAll[2:end,:],252)

printblue("Stats for the portfolio returns, annualized:\n")
colNames = ["LL","LH","HL","HH"]
printmat(Stats,colNames=colNames,rowNames=["μ";"σ";"SR"])

printblue("Study LH-LL and HH-HL to see the momentum effect (controlling for volatility):\n")
RLH_LL = ReAll[:,2] - ReAll[:,1]         #(L,H) minus (L,L)
RHH_HL = ReAll[:,4] - ReAll[:,3]         #(H,H) minus (H,L)

Stats = ReturnStats([RLH_LL RHH_HL][q+1:end,:],252)
printmat(Stats,colNames=["LH-LL","HH-HL"],rowNames=["μ";"σ";"SR"])
```

```
Stats for the portfolio returns, annualized:

          LL        LH        HL        HH
μ      8.287    12.181     4.976    11.987
σ     15.637    15.416    21.903    20.320
SR     0.530     0.790     0.227     0.590

Study LH-LL and HH-HL to see the momentum effect (controlling for volatility):

       LH-LL     HH-HL
μ      3.989     6.901
σ      8.614    12.387
SR     0.463     0.557
```

A Dependent Double Sort

on recent volatility (X) and and returns (Z).

This also creates four portfolios: (Low X, Low Z), (Low X, High Z), (High X, Low Z) and (High X, High Z). However, in this case, the Low X group is split up into: (Low X, Low Z) and (Low X, High Z), and similarly for the high X group.

The size of each of these portfolios is constant over time (unless there are missing values).

A Remark on the Code

- The `vXL` from `(vXL,vXH) = sortLoHi(X[t,:],trues(n),mX)` is an n-vector where element i is `true` when asset i belongs to 'low according to `X[t,:]`'.
- `sortLoHi(Z[t,:],vXL,mZ)` sorts `Z[t,vXL]` into low/high. The other elements in `Z[t,:]` will not belong to either.

```
(mX,mZ) = (10,5)

(RLL,RLH,RHL,RHH) = [fill(NaN,T) for i=1:4]
for t in 2:T                    #loop over periods, save portfolio returns
   #local vXL,vXH,vLL,vLH,vHL,vHH,wLL,wLH,wHL,wHH     #local/global is needed in script
    (vXL,vXH) = sortLoHi(X[t,:],trues(n),mX)       #Lo/Hi according to X
    (vLL,vLH) = sortLoHi(Z[t,:],vXL,mZ)           #within Lo X, Lo/Hi according to Z
    (vHL,vHH) = sortLoHi(Z[t,:],vXH,mZ)           #within Hi X, Lo/Hi according to Z
   (wLL,wLH,wHL,wHH) = (EWportf(vLL),EWportf(vLH),EWportf(vHL),EWportf(vHH)) #portfolio weights, EW
```

```
   (RLL[t],RLH[t],RHL[t],RHH[t]) = (wLL'Re[t,:],wLH'Re[t,:],wHL'Re[t,:],wHH'Re[t,:])
end
```

```
ReAll = [RLL RLH RHL RHH]
replace!(ReAll,NaN⇒0)         #replace NaN by 0, assuming investment in riskfree
Stats = ReturnStats(ReAll[2:end,:],252)

printblue("Stats for the portfolio returns, annualized:\n")
colNames = ["LL","LH","HL","HH"]
printmat(Stats,colNames=colNames,rowNames=["μ";"σ";"SR"])

printblue("Study LH-LL and HH-HL to see the momentum effect (controlling for volatility):\n")
RLH_LL = ReAll[:,2] - ReAll[:,1]         #(L,H) minus (L,L)
RHH_HL = ReAll[:,4] - ReAll[:,3]         #(H,H) minus (H,L)

Stats = ReturnStats([RLH_LL RHH_HL][q+1:end,:],252)
printmat(Stats,colNames=["LH-LL","HH-HL"],rowNames=["μ";"σ";"SR"])
```

```
Stats for the portfolio returns, annualized:

          LL        LH        HL        HH
μ      8.062    12.824     5.492    12.584
σ     16.118    15.506    22.364    21.200
SR     0.500     0.827     0.246     0.594

Study LH-LL and HH-HL to see the momentum effect (controlling for volatility):

       LH-LL     HH-HL
μ      4.743     7.078
σ      5.095     7.877
SR     0.931     0.899
```

Panel Regressions

This notebook uses functions to do panel regressions, handling autocorrelation, cross-sectional clustering and unbalanced panels.

Load Packages and Extra Functions

The key functions used in this notebook are from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics, LinearAlgebra
```

Loading Data and Reshuffling Data

```
(data,header) = readdlm("Data/nls_panelEd.txt",header=true)   #classical data set from Hill et al (200

d = PutDataInNT(data,header)                        #NamedTuple with d.id, d.lwage, etc
println(keys(d))

id  = convert.(Int,d.id)                             #id of individuals: 1,2,...,N
per = convert.(Int,d.year)
T = length(unique(per))                             #number of time periods
```

```
N = length(unique(id))                              #number of individuals

println("\nT=$T and N=$N")
```

```
(:id, :year, :lwage, :hours, :age, :educ, :collgrad, :msp, :nev_mar, :not_smsa, :c_city, :south, :black

T=5 and N=716
```

Creating Variables for the Regressions

The next cell creates a NT-vector y and a $NT \times K$ matrix x.

We then print the first few observations of (some of) the data. Notice the structure: the first 5 observations are for individual (`id`) 1 (period 1-5), the next 5 for individual 2.

```
y = d.lwage
c = ones(length(y))        #constant

xNames = ["exper/100","exper^2/100","tenure/100","tenure^2/100","south","union"]
x = [c d.exper/100 d.exper2/100 d.tenure/100 d.tenure2/100 d.south d.union]
K = size(x,2)                  #number of regressors

printblue("The first few lines of (some of) the data:\n")
printmat(id[1:11],y[1:11],x[1:11,1:2];colNames=["id","lnwage","c","exper/100"],rowNames=string.(1:1
```

```
The first few lines of (some of) the data:

obs        id    lnwage         c exper/100
1       1         1.808     1.000     0.077
2       1         1.863     1.000     0.086
3       1         1.789     1.000     0.102
4       1         1.847     1.000     0.122
5       1         1.856     1.000     0.136
6       2         1.281     1.000     0.076
7       2         1.516     1.000     0.084
8       2         1.930     1.000     0.104
9       2         1.919     1.000     0.120
10      2         2.201     1.000     0.132
11      3         1.815     1.000     0.114
```

Pooled OLS with `OlsNW()`

The next cell does pooled OLS estimation and reports White's standard errors. It is straightforward to used the `IndividualDemean()` or the `FirstDiff()` functions from the local `FinEcmt_OLS` module to also do fixed effects and first difference estimations.

However, the OLS code is cumbersome to extend to clustered standard errors and handling autocorrelation. For that reason, we later introduce the `PanelOls()` function which is better suited for these complications.

```
(b_LS,res,yhat,Covb,R2,) = OlsNW(y,x)            #pooled OLS, White's t-stats
tstat_LS = b_LS./sqrt.(diag(Covb))

printblue("results from Ols():\n")
printmat(b_LS,tstat_LS,colNames=["coef","t-stat"],rowNames=["c";xNames])
```

```
results from Ols():

                  coef    t-stat
c                1.285    28.513
exper/100        7.837     8.954
exper^2/100     -0.201    -5.264
tenure/100       1.206     2.346
tenure^2/100    -0.024    -0.828
south           -0.196   -13.247
union            0.110     6.928
```

Reshuffling the Data

to fit the convention in the `PanelOls()` function used below.

We use the `PanelReshuffle(y,x,[],id,per)` from the local `FinEcmt_OLS` module function to reshuffle the dependent variable into an $T \times N$ matrix `Y` and the regressors into a $T \times K \times N$ array `X`. This allows the `PanelOls()` function to handle autocorrelation and cross-sectional clustering.

```
#@doc2 PanelReshuffle
```

```
(Y,X) = PanelReshuffle(y,x,[],id,per)         #[] is not important here
(T,N,K) = (size(Y,1),size(Y,2),size(X,2))

println("The Y matrix is now $(T)x$(N), while X is $(T)x$(K)x$(N)")
display(Y)
```

```
The Y matrix is now 5x716, while X is 5x7x716
```

```
5×716 Matrix{Float64}:
 1.80829  1.28093  1.81482  2.31254  …  1.53039  1.52823  1.46094  1.60944
 1.86342  1.51585  1.91991  2.34858     1.59881  2.4065   1.49669  1.45944
 1.78937  1.93017  1.95838  2.37349     1.60405  2.55886  1.55984  1.42712
 1.84653  1.91903  2.00707  2.3689      1.26794  2.64418  1.6536   1.49437
 1.85645  2.20097  2.08985  2.35053     1.55823  2.58664  1.61586  1.34142
```

Pooled Estimation with `PanelOls()`

using the `PanelOls()` function from the `FinEcmt_OLS` module. The output is a named tuple. Use `keys(f0)` to see the names of the entries. The function reports several different estimates of the VCV matrix. We here use White's estimate of the standard errors.

However, the function can (optionally) report a VCV matrix that allows for clustering (see further below in the notebook).

```
@doc2 PanelOls

#using CodeTracking
#println(@code_string PanelOls([1],[1]))
```

```
PanelOls(y,x,m=0,clust=[];FixNaNQ=false,HacMethod=:NW)
```

Pooled OLS estimation.

Input

- `y0::Matrix`: TxN matrix with the dependent variable, `y[t,i]` is for period `t`, individual `i`
- `x0::3D Array`: TxKxN matrix with K regressors
- `m::Int`: (optional), scalar, number of lags in covariance estimation
- `clust::Vector{Int}`: (optional), N vector with cluster number for each individual, `[ones(N)]`
- `FixNaNQ::Bool`: (optional), true: replace all cases (y[t,i],x[t,:,i]) with some NaN/missing with (0,0), using PanelReplaceNaN()
- `HacMethod::Symbol`: :NW for Newey-West (tent shaped weights), :HH for Hodrick-Hansen (flat weights)

Output

- `fnOutput::NamedTuple`: named tuple with the following elements
  - `theta` (K*L)x1 vector, LS estimates of regression coeefficients on kron(z,x)
  - `CovDK` (K*L)x(K*L) matrix, Driscoll-Kraay covariance matrix
  - `CovC` covariance matrix, cluster
  - `CovNW` covariance matrix, Newey-West (or White if m=0)
  - `CovAR` covariance matrix, Arellano (handles autocorrelation)
  - `CovCAR` covariance matrix, cluster, Arellano (handles autocorrelation)
  - `CovLS` covariance matrix, iid
  - `R2` scalar, (pseudo-) R2
  - `yhat` TxN matrix with fitted values
  - `Nb` T-vector, number of obs in each period

Notice

- for TxNxK -> TxKxN, do `x = permutedims(z,[1,3,2])`

Pooled Estimation with the `PanelOls()` Function

```
fO = PanelOls(Y,X)             #use `;FixNaNQ=true` to handle missing values

θ_pooled      = fO.theta
StdErr      = sqrt.(diag(fO.CovNW))     #or fO.CovLS to get traditional (iid) t-stats
tstat_pooled  = θ_pooled./StdErr

printblue("results from pooled OLS using PanelOls(), White's standard errors:\n")
printmat(θ_pooled,tstat_pooled,colNames=["coef","t-stat"],rowNames=["c";xNames])

printred("compare with the result from `OlsNW()`: should be the same")
```

```
results from pooled OLS using PanelOls(), White's standard errors:

                  coef    t-stat
c                1.285    28.513
exper/100        7.837     8.954
exper^2/100     -0.201    -5.264
tenure/100       1.206     2.346
tenure^2/100    -0.024    -0.828
```

```
south           -0.196   -13.247
union            0.110     6.928

compare with the result from `OlsNW()`: should be the same
```

First Difference Estimation

by first creating first difference (ΔY,ΔX) and then using PanelOls.

```
ΔY = Y[2:end,:]   - Y[1:end-1,:]         #first differences
ΔX = X[2:end,:,:] - X[1:end-1,:,:]
ΔX[:,1,:] .= 1                           #put back a non-zero constant (variable 1)

fO = PanelOls(ΔY,ΔX)

θ_Δ     = fO.theta
StdErr  = sqrt.(diag(fO.CovNW))
tstat_Δ = θ_Δ./StdErr

printblue("results from first difference model using PanelOls(), White's standard errors:\n")
printmat(θ_Δ,tstat_Δ,colNames=["coef","t-stat"],rowNames=["c";xNames])
```

```
results from first difference model using PanelOls(), White's standard errors:

                  coef    t-stat
c                0.010     0.633
exper/100        3.548     2.277
exper^2/100     -0.045    -0.933
tenure/100       1.293     2.527
tenure^2/100    -0.083    -2.329
south           -0.024    -0.395
union            0.044     3.115
```

Fixed Effects Estimation

We drop the intercept and then apply the FixedIndivEffects() function from the FinEcmt_OLS module. It removes individual fixed effects. Then we apply the PanelOls() function. This can also handle unbalanced panels.

```
(yˣ,xˣ) = FixedIndivEffects(Y,X[:,2:end,:])    #drop the intercept

fO = PanelOls(yˣ,xˣ)

θ_FE     = fO.theta
StdErr   = sqrt.(diag(fO.CovNW))
tstat_FE = θ_FE./StdErr

printblue("results from fixed effects model using PanelOls(), White's standard errors:\n")
printmat(θ_FE,tstat_FE,colNames=["coef","t-stat"],rowNames=xNames)
```

```
results from fixed effects model using PanelOls(), White's standard errors:

                  coef    t-stat
exper/100        4.108     6.616
exper^2/100     -0.041    -1.640
tenure/100       1.391     4.445
tenure^2/100    -0.090    -4.624
south           -0.016    -0.411
union            0.064     4.675
```

Clustered Standard Errors

We now redo the FE estimation but provide information on clustering for the standard errors. For simplicity, the clusters are defined as the value of the South dummy in $t = 1$. Clearly, this could be done for the pooled OLS and the first difference models as well.

```
clust = convert.(Int,X[1,6,:])      #define clusters based on South/North in t=1

fO = PanelOls(yˣ,xˣ,0,clust)         #0 autocorrelations, but clustering

θ       = fO.theta
StdErrW = sqrt.(diag(fO.CovNW))       #White's std
StdErrC = sqrt.(diag(fO.CovC))        #clustered std
tstatW  = θ./StdErrW
tstatC  = θ./StdErrC

printblue("FE estimation with clustered standard errors:\n")
printmat(θ,tstatW,tstatC;colNames=["coef","t-stat White","t-stat Clust"],
                         rowNames=xNames,width=15)
```

```
FE estimation with clustered standard errors:

                      coef   t-stat White   t-stat Clust
exper/100            4.108          6.616          5.586
exper^2/100         -0.041         -1.640         -2.027
tenure/100           1.391          4.445          3.829
tenure^2/100        -0.090         -4.624         -4.880
south               -0.016         -0.411         -0.467
union                0.064          4.675          3.631
```

Unbalanced Panels (extra)

The `PanelOls(;FixNaNQ=true)` handles an unbalanced panel (NaNs/missings in `(y,x)`) by zeroing out all of `(y[t,i],x[t,:,i])` if there is a NaN/missing value for observation `(t,i)`.

Importantly, `FixedIndivEffects()` always calculates means based on only those observations that have no NaNs/missings, that is, based on those observations that are effectlively used in `PanelOls()`. This carries over to `FixedIndivTimeEffects()` and the other functions

This is illustrated in the cell below by first setting some of the data to `NaN` and then apply the functions. (The results are similar, but clearly not identical, to those above.)

```
(Yc,Xc) = (copy(Y),copy(X[:,2:end,:]))   #create new arrays (we change them below by inserting some NaN
(Yc[1],Xc[end]) = (NaN,NaN)                #introduce some missings/NaNs

(yc˟,xc˟,) = FixedIndivEffects(Yc,Xc)    #demeans, using only those obs that have no missings/NaNs
fO = PanelOls(yc˟,xc˟;FixNaNQ=true)

θ      = fO.theta
StdErr = sqrt.(diag(fO.CovNW))
tstat  = θ./StdErr

printblue("Results after zeroing out observations with some missing values/NaNs:\n")
printmat(θ,tstat,colNames=["coef","t-stat"],rowNames=xNames)
```

```
Results after zeroing out observations with some missing values/NaNs:

                  coef    t-stat
exper/100        4.080     6.563
exper^2/100     -0.039    -1.579
tenure/100       1.402     4.478
tenure^2/100    -0.090    -4.667
```

```
south           -0.016    -0.413
union            0.064     4.673
```

Individual *and* Time Fixed Effects (extra)

Redo the FE panel regression, but first we reconstruct `(yˣ,xˣ)` to handle *both* individual and time fixed effects.

For an unbalanced panel this is done in steps. First, remove individual effects (as above). Second, regress everything from the first step on time dummies. These two steps are done by the `FixedIndivTimeEffects()` function. Third, do a regression of the results from the 2nd step.

Reversing step 1 and 2 can be done, and makes sense when $T > N$. Use `FixedTimeIndivEffects()` for this.

```
(yˣ,xˣ) = FixedIndivTimeEffects(Y,X[:,2:end,:])

fO = PanelOls(yˣ,xˣ)

θ      = fO.theta
StdErr = sqrt.(diag(fO.CovNW))
tstat  = θ./StdErr

printblue("Panel regression with individual and time fixed effects:\n")
printmat(θ,tstat,colNames=["coef","t-stat"],rowNames=xNames)
```

```
Panel regression with individual and time fixed effects:

                  coef    t-stat
exper/100        6.713     4.654
exper^2/100     -0.045    -1.762
tenure/100       1.347     4.279
tenure^2/100    -0.090    -4.641
south           -0.014    -0.358
union            0.065     4.801
```

An Event Study as a Panel Regression

This notebook creates an artificial data set with event—in order to demonstrate how an event study can be implemented as a panel regression with dummies.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS

using Dates, DelimitedFiles, Statistics, LinearAlgebra
```

Load Data

The data set contains daily data for "dates", the equity market return, riskfree rate and the the returns of the 25 Fama-French portfolios. All returns are in percent.

```
x   = readdlm("Data/MomentumSR.csv",',')
dN  = Date.(x[:,1],"yyyy-mm-dd")                  #Julia dates
x   = Float64.(x[:,2:end])
(Rm,Rf,R) = (x[:,1],x[:,2],x[:,3:end])

println("size of dN, Rm, Rf, R")
println(size(dN)," ",size(Rm)," ",size(Rf)," ",size(R))

Re  = R .- Rf              #excess returns
Rme = Rm .- Rf             #market excess return
(T,N) = size(Re);
```

```
size of dN, Rm, Rf, R
(11598,) (11598,) (11598,) (11598, 25)
```

Add artificial Events to the Return Data

…to get a new data set that can be used to illustrate an event study.

```
vv_Events = floor.(Int,range(100,T-100,length=25))    #event indices
σ = std(Re)                                           #magnitude of event effect

ReX = copy(Re)
for i in 1:N
  ReX[vv_Events[i]:vv_Events[i]+1,i] .+= [σ,σ/2]     #add some extra returns
end
```

Panel Regression with Individual Dummies

and (market return, cionstant).

The `EventDummy()` function takes dates or indices to create dummy variables for `p` periods before the event to `q` periods after.

For simplicity, all assets have the same slope coefficients, including the same (α,β) for the "CAPM part." This could easily be changed by, for intance, introdcing `[Rme .* dummy_1,...,Rme .* dummy_N]` as new regressors.

```
(p,q) = (2,3)                 #dummies for tˣ-p to tˣ+q

D = zeros(Int,T,p+1+q,N)      #create 3D array with dummies
for i in 1:N
  (D[:,:,i],tˣ_i) = EventDummy(vv_Events[i],1:T,p,q)
end

z = repeat([Rme ones(T)],outer=(1,1,N))      #Tx2xN
X = cat(z,D;dims=2)                          #regressors, T x (2+p+1+q) x N

fO     = PanelOls(ReX,X)                     #pooled regression
θ      = fO.theta
StdErr = sqrt.(diag(fO.CovNW))               #White's VCV
tstat  = θ./StdErr

DummyNames = string.("D(",vcat(-p:1:0,1:1:q),")")
rowNames = ["β";"α";DummyNames]
printmat(θ,tstat;rowNames,colNames=["coef","t-stat"])
```

```
           coef    t-stat
β         0.979   424.088
α         0.004     3.171
D(-2)     0.129     1.359
D(-1)     0.114     1.559
D(0)      1.333    20.009
D(1)      0.479     4.119
D(2)     -0.042    -0.445
D(3)      0.047     0.414
```

Traditional Event Study

where all assets have the same (α,β) in the CAPM regression. This could easily be changed.

```
fO = PanelOls(ReX,z)                  #pooled CAPM regression, same (α,β) for all
AR = ReX - fO.yhat                    #abnormal return after regressing on (Rme,c)
σ_AR = std(AR)                        #std of AR

AR_event = fill(NaN,N,p+1+q)          #p periods before, q after
for i in 1:N                          #pick out returns around event daya
  AR_event[i,:] = AR[vv_Events[i]-p:vv_Events[i]+q,i]
end

AR_avg   = mean(AR_event,dims=1)'
AR_tstat = AR_avg./(σ_AR/sqrt(N))

println("Average returns around event days")

printmat(AR_avg,AR_tstat;rowNames=DummyNames,colNames=["coef","t-stat"])

printblue("the average returns are very similar (but not identicial) to the panel regression")
```

```
Average returns around event days
           coef    t-stat
D(-2)     0.129     0.993
D(-1)     0.114     0.878
D(0)      1.332    10.287
D(1)      0.479     3.694
D(2)     -0.042    -0.327
D(3)      0.046     0.358

the average returns are very similar (but not identicial) to the panel regression
```

Instrumental Variables

This notebook uses a function for 2SLS and illustrates it by redoing an empirical example.

Load Packages and Extra Functions

The key functions used for OLS and IV/2SLS are from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, LinearAlgebra
```

Loading the Data

The next cells uses data from Mroz (1987) to replicate old results. See also Greene (2018) and Hill et al (2018). The chapter gives more details.

A remark on the code

The data set contains many different variables. To import them to a named tuple, we use the `Readcsv()` from the `FinEcmt_OLS` module. (This is convenient, but not important for the focus of this notebook. An alternative is to use the `DataFrames.jl` package.)

```
X = Readcsv("Data/mroz.csv",0;ToFloat=true);

println("Variables in X: ",keys(X))
T = length(X.wage)
c = ones(T);              #create a constant
```

```
Variables in X: (:taxableinc, :federaltax, :hsiblings, :hfathereduc, :hmothereduc, :siblings, :lfp, :h
```

OLS

estimation of the log wage on education, experience and experience^2. Only data points where wage > 0 are used.

```
vv = X.wage .> 0          #find data points where X.wage > 0
                          #OLS of log wage, for observations with wage>0
(b_OLS,_,_,Covb,) = OlsNW(log.(X.wage[vv]),[c X.exper X.exper.^2 X.educ][vv,:])
Stdb_ols = sqrt.(diag(Covb))

colNames = ["coef","t-stat"]
rowNames = ["c","exper","exper^2","educ",]
printblue("OLS estimates:\n")
printmat(b_OLS,b_OLS./Stdb_ols;colNames,rowNames,prec=4)
```

```
OLS estimates:

             coef    t-stat
c         -0.5220   -2.6010
exper      0.0416    2.7344
exper^2   -0.0008   -1.9402
educ       0.1075    8.1697
```

IV (2SLS)

using the function `TwoSLS()` function.

In this application, the mother's education is used as an instrument for the person's education.

```
@doc2 TwoSLS
```

```
TwoSLS(y,x,z,NWQ=true,m=0)
```

Input

- `y::VecOrMat`: Tx1 or T-vector of the dependent variable
- `x::VecOrMat`: Txk matrix (or vector) of regressors
- `z::VecOrMat`: TxL matrix (or vector) of instruments
- `NWQ:Bool`: if true, then Newey-West's covariance matrix is used, otherwise Gauss-Markov
- `m::Int`: scalar, bandwidth in Newey-West; 0 means White's method

Output

- `b::Vector`: k-vector, regression coefficients
- `fnOutput::NamedTuple`: with
  - res Tx1 or Txn matrix, residuals y - yhat
  - yhat Tx1 or Txn matrix, fitted values
  - Covb matrix, covariance matrix of vec(b) = [beq1;beq2;…]
  - R2 1xn, R2
  - R2_stage1 k-vector, R2 of each x[:,i] in first stage regression on z
  - δ_stage1 Lxk matrix, coeffs from 1st stage x = z'δ
  - Stdδ_stage1 Lxk matrix, std of δ

Requires

- Statistics, LinearAlgebra
- CovNW

```
#using CodeTracking
#println(@code_string TwoSLS([1],[1],[1]))    #print the source code
```

```
(b_iv,fO2) = TwoSLS(log.(X.wage[vv]),[c X.exper X.exper.^2 X.educ][vv,:],
                                     [c X.exper X.exper.^2 X.mothereduc][vv,:])

zNames = ["c","exper","exper^2","mothereduc"]

printblue("first-stage estimates: coeffs (each regression in its own column)")
printmat(fO2.δ_stage1;colNames=rowNames,rowNames=zNames)
```

```
tstats = fO2.δ_stage1./fO2.Stdδ_stage1
printblue("first-stage estimates: t-stats")
printmat(tstats[:,4];colNames=rowNames[2:2],rowNames=zNames)    #2:2 to make it a vector

printblue("first-stage estimates: R²")
printmat(fO2.R2_stage1';colNames=rowNames)
```

```
first-stage estimates: coeffs (each regression in its own column)
                    c     exper   exper^2      educ
c               1.000    -0.000     0.000     9.775
exper           0.000     1.000    -0.000     0.049
exper^2        -0.000     0.000     1.000    -0.001
mothereduc      0.000    -0.000    -0.000     0.268

first-stage estimates: t-stats
                exper
c              23.753
exper           1.152
exper^2        -0.959
mothereduc      8.481

first-stage estimates: R²
         c     exper   exper^2      educ
       NaN     1.000     1.000     0.153
```

```
t_iv = b_iv./sqrt.(diag(fO2.Covb))
printblue("IV estimates")
printmat(b_iv,t_iv;colNames=["coef","t-stat"],rowNames)

printblue("DWH test (Chisq(1)): ")
println̲Ps(fO2.DWHtest)
```

```
IV estimates
              coef    t-stat
c            0.198     0.407
exper        0.045     2.888
exper^2     -0.001    -2.145
educ         0.049     1.301

DWH test (Chisq(1)):
     2.851
```

GMM

This notebook shows a simple example of how GMM can be used to estimate model parameters. It starts with an exactly identified case and then moves on to different ways of estimating an overidentified case (pre-defined weighting matrix, recombining the moment conditions, optimal weighting matrix).

Load Packages and Extra Functions

The general GMM functions are from the (local) `FinEcmt_MLEGMM` module. In contrast, the functions for the moment conditions ("the model") are coded in the notebook below.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
using FinEcmt_MLEGMM: GMMAgbar, GMMExactlyIdentified, GMMgbarWgbar, meanV, nlsolvePs
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_MLEGMM.jl"))
using .FinEcmt_OLS
using .FinEcmt_MLEGMM: GMMAgbar, GMMExactlyIdentified, GMMgbarWgbar, meanV
=#
```

```
using DelimitedFiles, Statistics, LinearAlgebra
```

Loading Data

```
x = readdlm("Data/FFmFactorsPs.csv",',',skipstart=1)   #start on line 2, column 1
x = x[:,2]         #excess market returns, in %

T = size(x,1)
```

660

Exactly Identified GMM

This section describes the exactly identified GMM, that is, when we have as many moment conditions as parameters. In this case GMM is the same as the classical method of moments.

Traditional Estimation of Mean and Standard Deviation

The next cell applies the traditional way of estimating the mean and the standard deviation. The standard errors are text book formulas based on an assumption of iid N().

```
μ  = mean(x)
σ = std(x,corrected=false)       #corrected="false" uses the 1/T formula

par_a = [μ,σ]

printblue("Traditional estimates:\n")
xx = [par_a [sqrt((σ^2/T));σ/sqrt(2*T)]]
colNames = ["coef","std"]
parNames = ["μ","σ"]
printmat(xx;colNames,rowNames=parNames)
```

```
Traditional estimates:

       coef       std
μ     0.608     0.179
σ     4.595     0.126
```

GMM Point Estimates

To estimate the mean and standard deviation of x_t, use the following moment condition

$$g_t(\beta) = \begin{bmatrix} x_t - \mu \\ (x_t - \mu)^2 - \sigma^2 \end{bmatrix},$$

where $\beta = [\mu, \sigma]$.

The parameter values (μ, σ) that make these moment conditions hold are the same as from the traditional method. It is easy to solve for these parameters when the moment conditions are linear in the parameters

(as they are here). However, to facilitate adapting the code to non-linear models, we solve for the parameters by a numerical method.

```
"""
    Gmm2MomFn(par,x)

Calculate traditional 2 moment conditions for estimating [μ,σ]. Returns a Tx2 matrix

# Input
- `par::Vector`: [μ,σ]
- `x::Vector`:   T-vector with data

# Output
- `g::Matrix`:    Tx2, moment conditions

"""
function Gmm2MomFn(par,x)
    (μ,σ) = (par[1],par[2])
    u     = x .- μ
    g     = hcat(u, u.^2 .- σ^2)  #Tx2
    return g
end
```

```
Gmm2MomFn
```

A Remark on the Code

…in the next cells.

- The `meanV()` function calculates the sample mean of each column in a matrix and returns a vector.
- The `p->meanV(Gmm2MomFn(p,x))` defines an anonymous function (in terms of the vector of parameters `p`) that returns a vector of the sample averages of the moment conditions.
- `par1 = nlsolvePs(p->meanV(Gmm2MomFn(p,x)),par_a)` solves for the vector `p` that makes the average moment conditions equal to `[0,0]`. (This could be done manually in this case, but perhaps not in another application.)

```
par_1 = nlsolvePs(p->meanV(Gmm2MomFn(p,x)),par_a)  #numerically solve for the estimates

printblue("GMM estimates:")
printmat(par_1;rowNames=parNames)
```

```
g    = Gmm2MomFn(par_1,x)        #Tx2, moment conditions
gbar = meanV(g)                  #2-vector with average moment conditions

printblue("Checking if mean of moment conditions = 0")
printmat(gbar;rowNames=["g₁","g₂"])
```

```
GMM estimates:
μ     0.608
σ     4.595

Checking if mean of moment conditions = 0
g₁     0.000
g₂     0.000
```

GMM Distribution

The distribution of the basic GMM estimates is

$\hat{\beta} \xrightarrow{a} N(\beta_0, V_T)$,

where

$V_T = (D'\Sigma^{-1}D)^{-1}/T$,

$\Sigma = \text{Var}(\sqrt{T}\bar{g})$,

$D = \text{plim}\frac{\partial \bar{g}}{\partial \beta'}$.

(This holds for exactly identified models and models using the optimal weighting matrix.)

A Remark on the Code

- `CovNW(g,1,1)` estimates Σ by using the Newey-West method with one lag.
- `D` is easy to calculate analytically in this case.

```
D = [1 0;
     0 2*par_1[2]]           #Jacobian
Σ  = CovNW(g,1,1)            #variance of sqrt(T)*gbar, NW with 1 lag
V_T = inv(D'inv(Σ)*D)/T

printblue("GMM estimates:\n")
printmat(par_1,sqrt.(diag(V_T));colNames,rowNames=parNames)
```

```
printstyled("Compare with the traditional estimates",color=:red,bold=true)
```

```
GMM estimates:

       coef       std
μ     0.608     0.182
σ     4.595     0.184

Compare with the traditional estimates
```

A Function for Exactly Identified GMM

The functions printed in the next cell combines the previous code into a function for getting both point estimates and standard errors. The Jacobian is calculated numerically. The result should be the same as before. (For the rest of the notebook, the code will come in functions like this.)

```
@doc2 GMMExactlyIdentified
```

```
GMMExactlyIdentified(GmmMomFn::Function,x,par0,m)
```

Estimates GMM coeffs and variance-covariance matrix from an exactly identified model. The Jacobian is calculated numerically.

Input

- `GmmMomFn::Function`: for the moment conditions, called as `GmmMomFn(p,x)` where `p` are the coefficients and `x` is the data.
- `x::VecOrMat`: data
- `par0::Vector`: initial guess
- `m::Int`: number of lags in NW covariance matrix

```
#using CodeTracking
#println(@code_string GMMExactlyIdentified(cos,[1],[1],1))
```

```
(par_1b,Std_1b,) = GMMExactlyIdentified(Gmm2MomFn,x,par_a,1)

printblue("Results from GMMExactlyIdentified():\n")
printmat(par_1,sqrt.(diag(V_T));colNames,rowNames=parNames)

printstyled("Compare with GMM results from above",color=:red,bold=true)
```

```
Results from GMMExactlyIdentified():

       coef       std
μ     0.608     0.182
σ     4.595     0.184

Compare with GMM results from above
```

Overidentified GMM

This section discusses an overidentified case: more moment conditions than parameters.

Warning: some of the variables (`g`,`Σ`, etc) are overwritten with new values.

The Moment Conditions

If x_t is $N(\mu, \sigma)$, then the following moment conditions should all be zero (in expectation)

$$g_t(\beta) = \begin{bmatrix} x_t - \mu \\ (x_t - \mu)^2 - \sigma^2 \\ (x_t - \mu)^3/\sigma^3 \\ (x_t - \mu)^4/\sigma^4 - 3 \end{bmatrix},$$

where $\beta = [\mu, \sigma^2]$.

The first moment condition defines the mean μ, the second defines the variance σ^2, while the third and forth are the skewness and excess kurtosis respectively.

```
"""
    Gmm4MomFn(par,x)

Calculate 4 moment conditions for estimating [μ,σ]

# Input
- `par::Vector`: [μ,σ]
- `x::Vector`:   T-vector with data

# Output
- `g::Matrix`:   Tx4, moment conditions

"""
function Gmm4MomFn(par,x)
```

```
  (μ,σ) = par
  u     = x .- μ
  g     = hcat(u, u.^2 .- σ^2, (u/σ).^3, (u/σ).^4 .- 3)    #Tx4
  return g
end
```

`Gmm4MomFn`

Overidentified GMM: Minimizing gbar'W*gbar

The following code applies a numerical method to solve a minimization problem with the weighting matrix

$$W = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

The results should be the same (or at least very close to) the previous results, since this W matrix puts all weight on the first two moments. Changing W, for instance, by setting `W[3,3]=0.1 and W[4,4]=0.1` will give other estimates.

We define the loss function as $\bar{g}' W \bar{g}$.

The expressions for variance-covatriance matrix are in the lecture notes.

A Function for gbar'W*gbar Estimation

```
@doc2 GMMgbarWgbar
```

```
GMMgbarWgbar(GmmMomFn::Function,W,x,par0,m;SkipCovQ=false)
```

Estimates GMM coeffs and variance-covariance matrix from gbar'W*gbar. The Jacobian is calculated numerically.

Input

- `GmmMomFn::Function`: for the moment conditions, called as `GmmMomFn(p,x)` where `p` are the coefficients and `x` is the data.
- `W::Matrix`: length(gbar)xlength(gbar)
- `x::VecOrMat`: data
- `par0::Vector`: initial guess
- `m::Int`: number of lags in NW covariance matrix
- `SkipCovQ::Bool`: if true: the Jacobian and variance-covariance matrix are not calculated. This can be used to speed up calculations in iterative computations.

```
#println(@code_string GMMgbarWgbar(cos,[1],[1],[1],1))
```

```
W     = diagm(0⇒[1.0,1.0,0.0,0.0])   #weighting matrix, try changing it
#W     = diagm(0⇒[1.0,1.0,0.1,0.1])
(par2,StdErr2,_,Σ2,_,gTest2) = GMMgbarWgbar(Gmm4MomFn,W,x,par_a,1)

printblue("GMM estimates (gbar'W*gbar):\n")
printmat(par2,StdErr2;colNames,rowNames=parNames)

printblue("Test of overidentifying restrictions:\n")
printlnPs(gTest2)
```

```
GMM estimates (gbar'W*gbar):

       coef       std
μ     0.608     0.182
σ     4.595     0.184

Test of overidentifying restrictions:

     6.701
```

Overidentified GMM: Minimizing gbar'W*gbar, Iterating over W

The following code iterates over the weighting matrix by using $W = \Sigma^{-1}$, where $\Sigma = \text{Cov}(\sqrt{T}\bar{g})$ is from the previous iteration.

A Remark on the Code

- `(maximum(abs,par - par_old) > 1e-3) || (i < 2)` loops while the |change| in the parameters > 1e-3 (and at least once).
- `par_old = copy(par)` makes an independent copy of the `par`. This is not needed here (since `par` is overwritten further below), but often a good routine.

```
println("\niterated GMM, using optimal weighting matrix, starting with Σ from previous estimation")

(par,par_old,Σ,i) = (copy(par2),fill(Inf,length(par2)),copy(Σ2),1)

println("\n\niterating over W starting with the W choice above")
while (maximum(abs,par - par_old) > 1e-3) || (i < 2)   #require at least one iteration
    #global par, par_old, i, W, Σ    #only needed in script
    global StdErr2, D
    println("-------iteration  $i, old and new parameters--------")
    par_old = copy(par)              #update par_old
    W       = inv(Σ)
    (par,StdErr2,_,Σ,D) = GMMgbarWgbar(Gmm4MomFn,W,x,par_old,1)
    i       = i + 1
    printlnPs(par_old',"\n",par')
 end

printblue("\nGMM estimates (gbar'W*gbar, iteration over W):")
xx = [par StdErr2]
printmat(xx;colNames,rowNames=parNames,width=12)
```

```
iterated GMM, using optimal weighting matrix, starting with Σ from previous estimation


iterating over W starting with the W choice above
-------iteration  1, old and new parameters--------
     0.608     4.595
     0.694     4.469
-------iteration  2, old and new parameters--------
     0.694     4.469
     0.714     4.455
-------iteration  3, old and new parameters--------
     0.714     4.455
     0.716     4.453
```

```
-------iteration  4, old and new parameters--------
     0.716     4.453
     0.716     4.453

GMM estimates (gbar'W*gbar, iteration over W):
         coef         std
μ       0.716       0.174
σ       4.453       0.180
```

```
printblue("W matrix used in the last iteration, scaled:\n")

momNames = ["g₁","g₂","g₃","g₄"]
printmat(W/W[1,1],colNames=momNames,rowNames=momNames)
```

```
W matrix used in the last iteration, scaled:

          g₁        g₂        g₃        g₄
g₁     1.000     0.048    -0.710    -0.131
g₂     0.048     0.016    -0.046    -0.022
g₃    -0.710    -0.046     0.904     0.178
g₄    -0.131    -0.022     0.178     0.054
```

```
V_T = inv(D'inv(Σ)*D)/T                      #with optimal weighting matrix

printblue("\nGMM estimates (gbar'W*gbar, iteration over W):")
xx = [par StdErr2 sqrt.(diag(V_T))]
printmat(xx;colNames=[colNames;"std ver. 2"],rowNames=parNames,width=12)

printred("Notice that the standard errors (calculated in two ways) are (almost) the same after the itera
```

```
GMM estimates (gbar'W*gbar, iteration over W):
         coef         std  std ver. 2
μ       0.716       0.174       0.174
σ       4.453       0.180       0.180

Notice that the standard errors (calculated in two ways) are (almost) the same after the iterations
```

Overidentified GMM: A*g = 0

The following code from estimates the parameters by combining the 4 original moment conditions in $\bar{g}$ into 2 effective moment conditions, $A\bar{g}$, where A is a 2×4 matrix

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

This particular A matrix implies that we use the classical estimators of the mean and standard deviation. Changing A, for instance, by setting $A[1,3] = 0.001$ will give different estimates.

The next cell prints a function for doing A*gbar GMM.

```
@doc2 GMMAgbar
```

```
GMMAgbar(GmmMomFn::Function,A,x,par0,m)
```

Estimates GMM coeffs and variance-covariance matrix from A*gbar. The Jacobian is calculated numerically.

Input

- `GmmMomFn::Function`: for the moment conditions, called as `GmmMomFn(p,x)` where `p` are the coefficients and `x` is the data.
- `A::Matrix`: length(p) x length(gbar)
- `x::VecOrMat`: data
- `par0::Vector`: initial guess
- `m::Int`: number of lags in NW covariance matrix

```
#using CodeTracking
#println(@code_string GMMAgbar(cos,[1],[1],[1],1))
```

```
A = [1 0 0 0;                  #A in A*gbar=0 (here: all weight on first two moments)
     0 1 0 0]                    #try setting A[1,3] = 0.1 and A[2,4] = 0.5
#A = [1 0 0.1 0;
#     0 1 0 0.5]

(par_3b,Std_3b,_,_,gTest3) = GMMAgbar(Gmm4MomFn,A,x,par_a,1)

printblue("GMM estimates (A*gbar) from GMMAgbar():\n")
printmat(par_3b,Std_3b;colNames,rowNames=parNames)
```

```
printblue("Test of overidentifying restrictions:\n")
printlnPs(gTest3)
```

```
GMM estimates (A*gbar) from GMMAgbar():

       coef       std
μ     0.608     0.182
σ     4.595     0.184

Test of overidentifying restrictions:

     6.701
```

Time Series Analysis

This notebook discusses MA and ARMA models.

Load Packages and Extra Functions

The key functions in this notebook are from the (local) `FinEcmt_TimeSeries` module, but we also use StatsBase.jl package for estimating autocorrelations.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS, FinEcmt_TimeSeries
using FinEcmt_MLEGMM: MLE
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_TimeSeries.jl"))
using .FinEcmt_OLS, .FinEcmt_TimeSeries
using .FinEcmt_MLEGMM: MLE
=#
```

```
using DelimitedFiles, LinearAlgebra, Distributions, StatsBase
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Load Data

```
xx     = readdlm("Data/ReSP500Daily.csv",',',skipstart=1)
y_data = Float64.(xx[:,2])
xx     = nothing

T    = size(y_data,1)
println("Sample size: $T")

y_data0 = replace(y_data,NaN⇒0);     #there are approx 1% missing values. Try replacing with 0
```

```
Sample size: 11603
```

Descriptive Statistics

Autocorrelations 1

The sth autocorrelation ρ_s is the correlation of y_t and y_{t-s}. It is a useful tool for describing the properties of data.

With iidN data, the t-stat of an autocorrelation is $\sqrt{T}\rho_s$.

We can (jointly) test the first L autocorrelations by the Box-Pierce test, which (under the null hypothesis of no autocorrelation) has a χ^2_L distribution.

```
L = 5
lags = 1:L
ρ_data0 = autocor(y_data0,lags)      #using the StatsBase package and missings replaced by 0
ρ_data = AutoCorr(y_data,lags)       #from local package, discarding (y(t),y(t-s)) if any is missing

printblue("Autocorrelations (with replacing or discarding missings):\n")
printmat(ρ_data0,ρ_data,sqrt(T)*ρ_data;colNames=["autocor (0)","autocor","t-stat"],rowNames=lags)

BP = T*sum(ρ_data.^2)            #the Box-Pierce test statistic

printblue("Box-Pierce test:\n")
printmat(BP,quantile(Chisq(L),0.9);rowNames=["test statistic","10% critical value"])
```

```
Autocorrelations (with replacing or discarding missings):

 autocor (0)   autocor    t-stat
1    -0.054    -0.055    -5.920
```

```
2    -0.006    -0.006    -0.619
3    -0.013    -0.013    -1.404
4    -0.026    -0.026    -2.829
5    -0.007    -0.007    -0.782

Box-Pierce test:

test statistic        46.020     9.236
```

Autocorrelations 2

…via OLS instead, to get robust standard errors. Handles missing values.

```
(ρ_ols,Stdρ_ols,_,BPstat_ols) = AutoCorrOls(y_data,lags)
tstat_ols = ρ_ols./Stdρ_ols

printblue("Autocorrelations from OLS:\n")
printmat(ρ_ols,tstat_ols;colNames=["autocorrr","t-stat"],rowNames=lags,cell00="lag")

printblue("Box-Pierce test:\n")
printmat(BPstat_ols,quantile(Chisq(L),0.9);rowNames=["test statistic","10% critical value"])
```

```
Autocorrelations from OLS:

lag autocorrr    t-stat
1      -0.059    -2.454
2      -0.013    -0.494
3      -0.015    -0.797
4      -0.032    -1.914
5      -0.008    -0.336

Box-Pierce test:

test statistic        13.404     9.236
```

MA(q)

This section discusses the properties and estimation of moving average (MA) processes.

The next cell shows how a shock to an MA process propagates over time, the *impulse response function* (IRF). For *stationary models*, the effect of a shock eventually disappears. Notice that the IRF and the autocorrelation of an MA(q) Process goes to zero at lag $q + 1$.

The `ARMAFilter()` (from local module) creates an ARMA series from a series of residuals. To calculate the IRF, we apply that to the case when just the *initial* residual is non-zero.

```
#@doc2 ARMAFilter

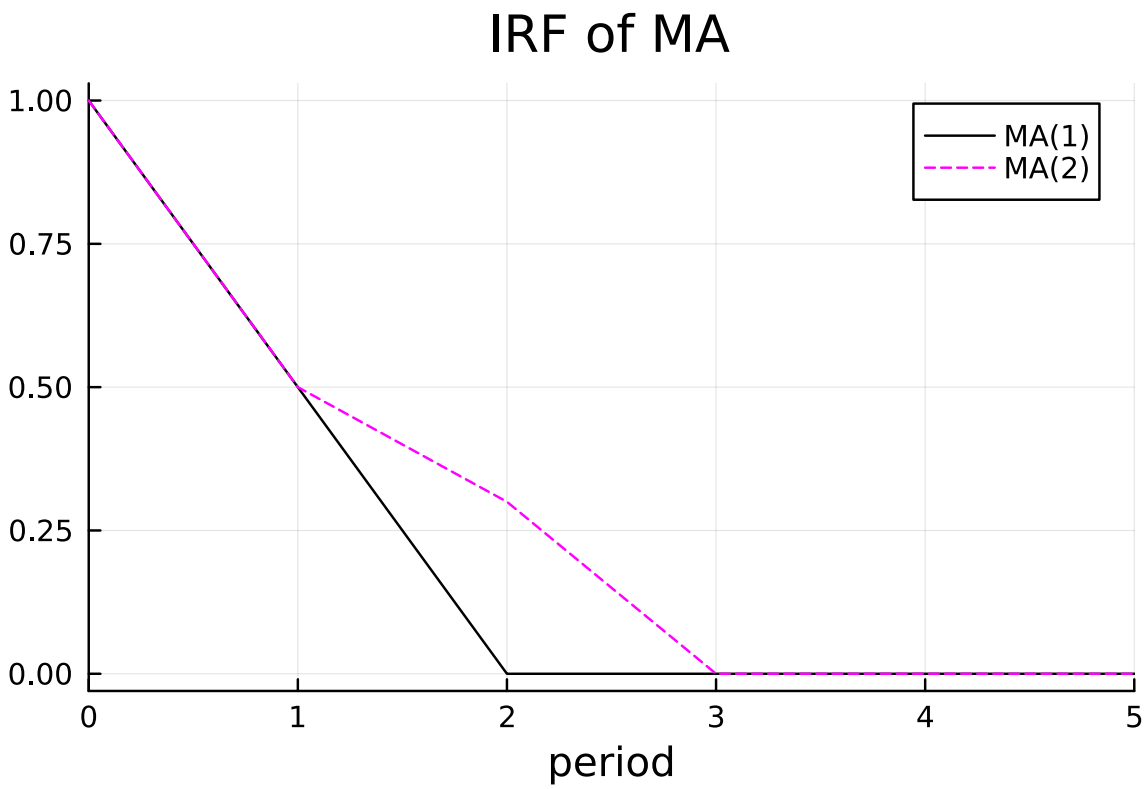
#using CodeTracking
#println(@code_string ARMAFilter([1],[1]))    #print the source code
```

```
θ = 0.5              #MA(1)
(θ₁,θ₂) = (0.5,0.3)  #MA(2)

t_max = 15
ϵ     = zeros(t_max)
ϵ[1]  = 1                    #a single shock, otherwise zeros

y1 = ARMAFilter(ϵ,[],θ)
y2 = ARMAFilter(ϵ,[],[θ₁,θ₂])

p1 = plot( 0:t_max-1,[y1 y2],
           label = ["MA(1)" "MA(2)"],
           linecolor = [:black :magenta],
           linestyle = [:solid :dash],
           xlims = (0,5),
           xlabel = "period",
           title = "IRF of MA" )
display(p1)
```

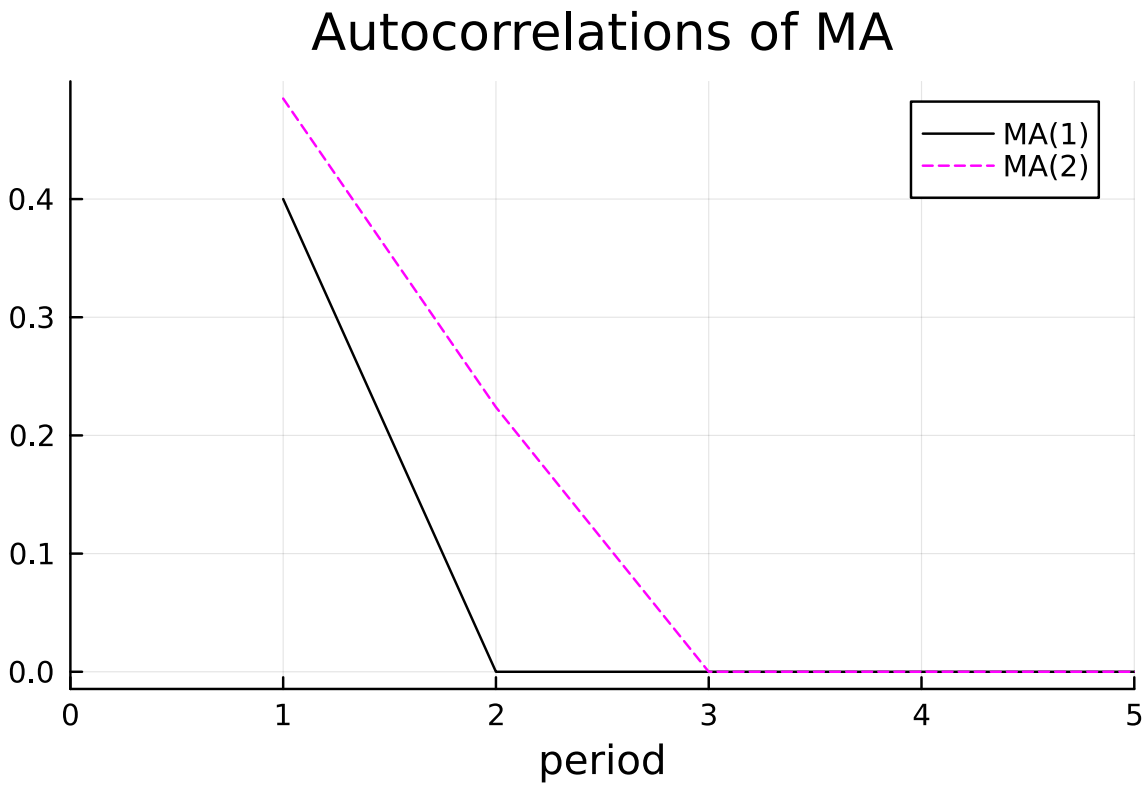


Autocorrelations of an MA(q)

It is straightforward to calculate the autocorrelations implied by an MA(q) model. The next cell illustrates some results from the `MAqToAutocorr()` function (in local module).

```
ρMA1 =  MAqToAutocorr([θ],t_max)        #using the θ values from above
ρMA2 =  MAqToAutocorr([θ₁,θ₂],t_max)

p1 = plot( 1:t_max,[ρMA1 ρMA2],
           label = ["MA(1)" "MA(2)"],
           linecolor = [:black :magenta],
           linestyle = [:solid :dash],
           xlims = (0,5),
           xlabel = "period",
           title = "Autocorrelations of MA" )
display(p1)
```



Estimation of an MA(q) (extra)

To estimate an MA model, we apply MLE and use the Optim.jl package for the numerical optimization.

The `MAqLL(par,y_data)` function (from local module) calculates the likelihood function and returns it as its first output. It is then used in the `MLE()` function to find the point estimates and calculate (different versions of) the VCV matrix.

Ideally, the MA model estimation should handle missing data. This can be done by using a Kalman filter (beyond the scope of this notebook) or GMM. Below, we use a simpler (approximate) approach by using the data where missings are replaced by 0.

```
par0 = [0.0,0.0,0,0,1]   #θ₁,θ₂,θ₃,θ₄,σ          #estimate an MA(4)

LLtFun_MA(par,y,x) = MAqLL(par,y)[1]             #log likelihood fn
(par,_,_,stdS,) = MLE(LLtFun_MA,par0,y_data0,nothing)

(θhat,σhat) = (par[1:end-1],par[end])
```

```
printblue("MLE of MA(4):\n")
printmat([θhat;σhat],stdS;colNames=["parameters","std"],rowNames=["θ₁","θ₂","θ₃","θ₄","σ"],width=15
```

```
MLE of MA(4):

       parameters            std
θ₁         -0.055          0.021
θ₂         -0.007          0.024
θ₃         -0.013          0.020
θ₄         -0.027          0.019
σ           1.104          0.024
```

AR(p)

This section discusses the properties and estimation of autoregressive (AR) processes.

Rewriting an AR(p) as a VAR(1)

An easy way to do AR(p) calculations is to first transform the AR(p) to a VAR(1) (also called companion form) and then apply the rather straightforward formulas for VAR(1) models. This is discussed in detail somewhat later in the chapter, but it is a useful numerical tool, so we introduce it here.

For instance, an AR(2)

$y_t = a_1 y_{t-1} + a_2 y_{t-2} + \epsilon_t.$

can be written as a 2-variable VAR(1)

$x_t = A_1 x_{t-1} + u_t,$

where

$x_t = \begin{bmatrix} y_t \\ y_{t-1} \end{bmatrix}$ and $A_1 = \begin{bmatrix} a_1 & a_2 \\ 1 & 0 \end{bmatrix}.$

This is done in the `CompanionFormAR()` function (from local module).

```
(a₁,a₂) = (0.85,-0.5)                  #AR(2)
(A,Ω) = CompanionFormAR([a₁,a₂])       #A and the Var(shocks)

printblue("The VAR(1) matrix, A:")
printmat(A)
```

```
The VAR(1) matrix, A:
     0.850    -0.500
     1.000     0.000
```

Impulse Response Function of an AR(p) Process

The IRF can be calculated by an explicit loop for the specific model. We do that below in order to show the basic idea. As a comparison, we apply the approach of first transforming to a VAR(1) and then using the simple formulas for that model

$ ∂x_{t+s}/∂u_{j,t} = A^s e_j, $ where is a vector of zeros, except that element j is one. The function `VAR1IRF()` (from local module) does this.

(The `ARMAFilter()` function could also be used but this is not done here.)

The models used here is an AR(2)

$y_t = a_1 y_{t-1} + a_2 y_{t-2} + \epsilon_t.$

```
t_max = 15                #calculations by a simple loop

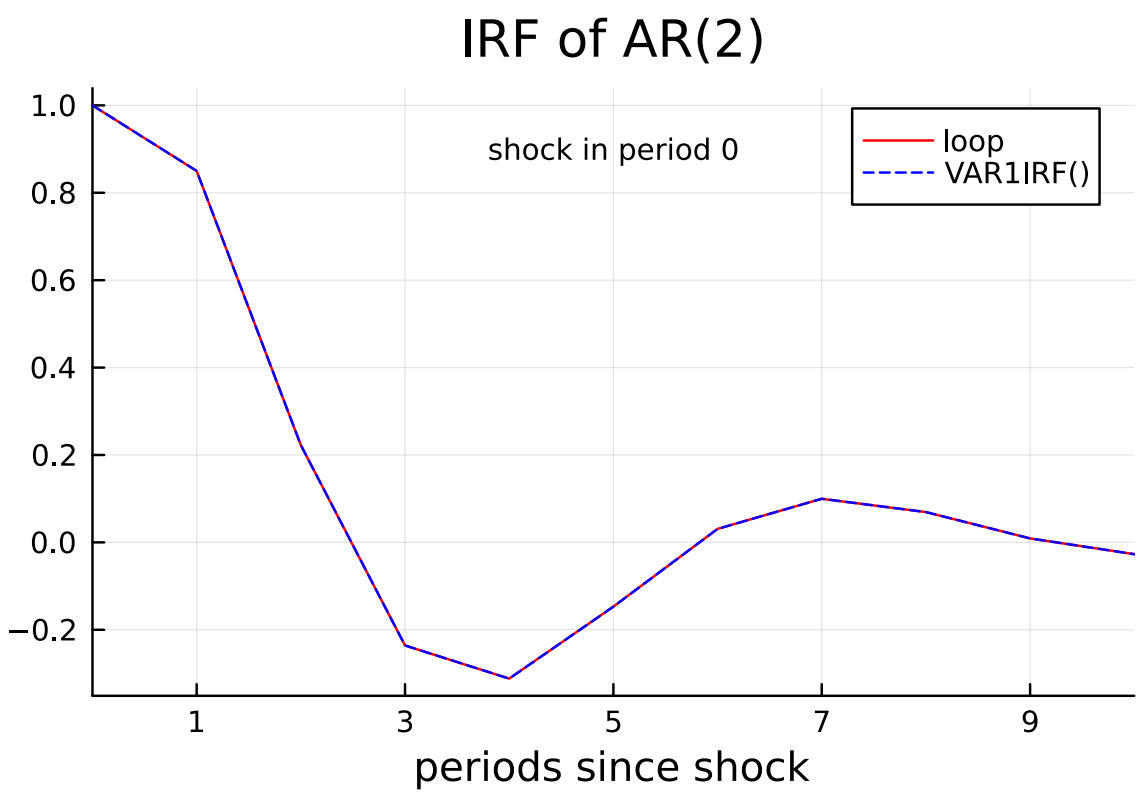
ϵ    = zeros(t_max)
ϵ[1] = 1                  #a single shock

y = copy(ϵ)
y[1] = ϵ[1]               #setting a₁*y[0] + a₂*y[-1] = 0
y[2] = a₁*y[1] + ϵ[2]     #setting a₂*y[0] = 0
for t in 3:t_max
    y[t] = a₁*y[t-1] + a₂*y[t-2] + ϵ[t]
end
```

```
irf_ar2 = VAR1IRF(A,t_max-1;m=1);     #same, but using VAR(1) function
                                      #m=1 to pick out results for only 1 variable
#irf_ar2_ = ARMAFilter(ϵ,[a₁,a₂],[])
#printmat(y,irf_ar2,irf_ar2_)
```

```
p1 = plot( 0:t_max-1,[y irf_ar2],
           linecolor = [:red :blue],
           linestyle = [:solid :dash],
           label = ["loop" "VAR1IRF()"],
           xticks = 1:2:t_max,
            xlims = (0,10),
           xlabel = "periods since shock",
```

```
           title = "IRF of AR(2)",
           annotation = (5,0.9,text("shock in period 0",8)) )
display(p1)
```



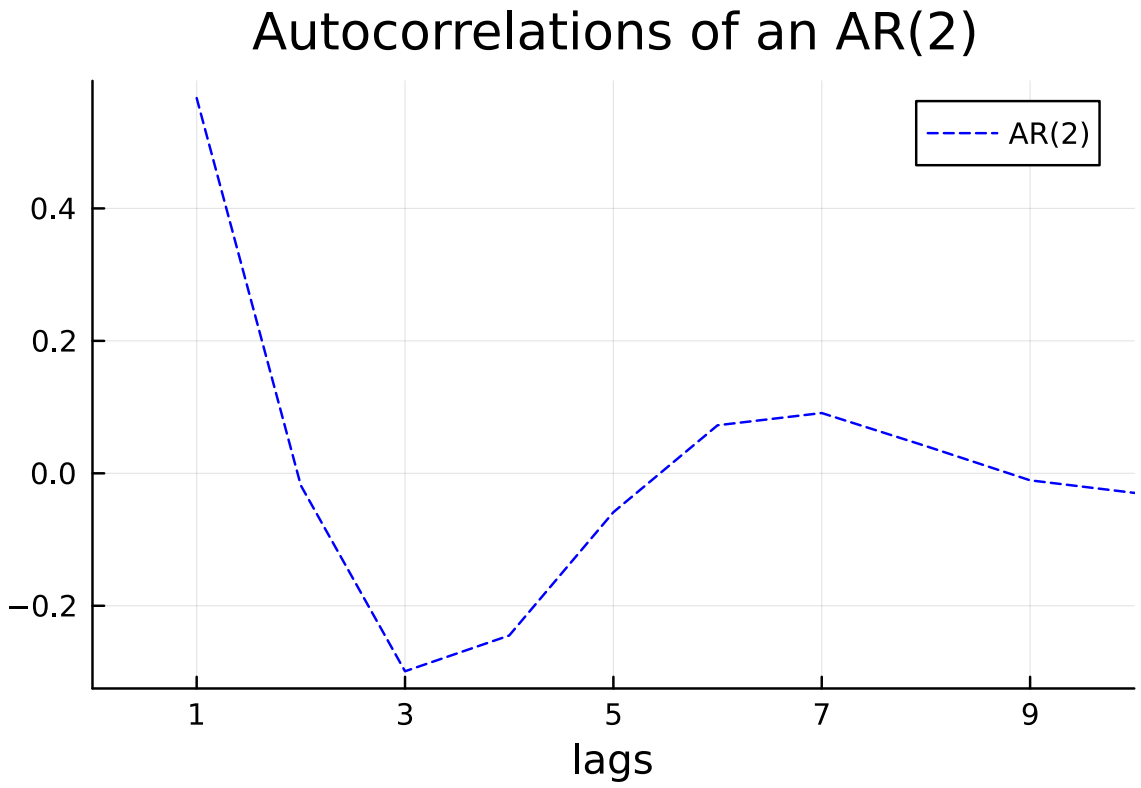
Autocorrelations Implied by an AR(p) Process

by using `VAR1AutoCov()` (from local module)

```
(_,_,ρ_ar2) = VAR1AutoCov(A,t_max;m=1)     #implied autocorrelations

p1 = plot( 1:t_max,ρ_ar2,
           label = "AR(2)",
           linecolor = :blue,
           linestyle = :dash,
           xticks = 1:2:t_max,
           xlims = (0,10),
           xlabel = "lags",
```

```
           title = "Autocorrelations of an AR(2)" )
display(p1)
```



Estimation of an AR(p) Process

AR models can be estimated by OLS.

The `ARpEst(y,p)` function (from local module) provides a simple way of doing this. It handles missing values.

```
#@doc2 ARpEst

#using CodeTracking
#println(@code_string ARpEst([1],2))    #print the source code
```

```
aAR1_data = ARpEst(y_data,1)
aAR2_data = ARpEst(y_data,2)
println∫Ps("Estimated AR(1) coef: ",aAR1_data)
println∫Ps("Estimated AR(2) coefs: ",aAR2_data)
```

```
Estimated AR(1) coef:     -0.055
Estimated AR(2) coefs:     -0.053    -0.009
```

More Descriptive Statistics: Sample Partial Autocorrelations

The partial autocorrelation coefficient is the regression coefficients on x_{t-p} in an AR(p) regression, that is, on the last (non-constant) regressor. It is a useful tool for describing the properties of your data.

```
pac_data0 = pacf(y_data0,lags)         #from the StatsBase package, does not handle missings
pac_data = PartialAutoCorr(y_data,lags)    #from local module, handles missings

printblue("autocorrelations and  partial autocorrelations:\n")
printmat(pac_data0,pac_data;colNames=["pac (0)","pac"],rowNames=lags)
```

```
autocorrelations and  partial autocorrelations:

    pac (0)       pac
1    -0.054    -0.055
2    -0.009    -0.009
3    -0.014    -0.015
4    -0.028    -0.035
5    -0.010    -0.012
```

The Roots of an AR(p) Model (extra)

The *roots* of an AR(p) is a formal way of determining whether it is stationary. (An informal way is to check whether the impulse response function goes to zero as the horizon is extended.)

An easy way to calculate the roots is to first transform the AR(p) to a VAR(1) and then calculate the eigenvalues of the VAR(1) matrix (A). If their absolute values are lower than one, then the model is stationary.

```
λ = eigen(A).values                 #eigenvalues of the companion form

println("absolute values of the eigenvalues: should be < 1 for stationarity")
printmat(abs.(λ))
```

```
absolute values of the eigenvalues: should be < 1 for stationarity
     0.707
     0.707
```

Forecasting with an AR(p) Process

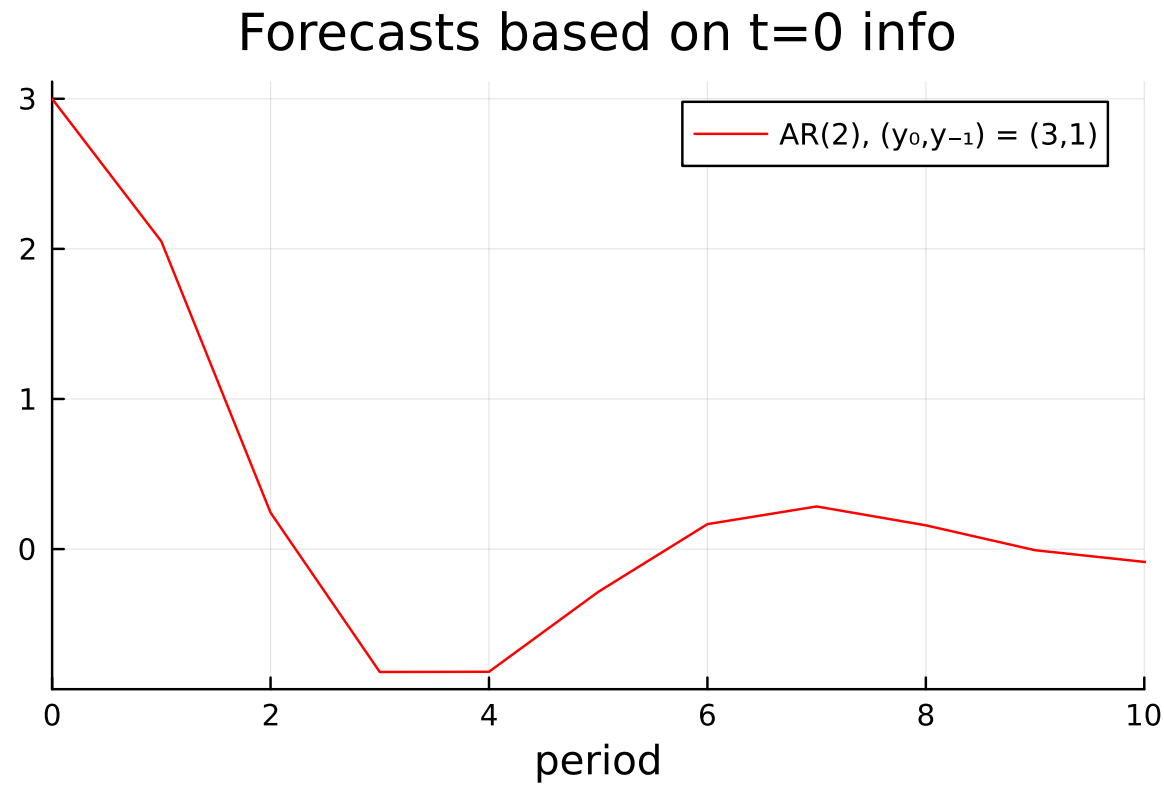
Forecasts can be calculated in several ways. The perhaps easiest approach is to use the VAR(1) form and notice that $\mathrm{E}_t x_{t+s} = A^s x_t$.

The `VAR1Forecast()` (from local module) implements this.

```
x_initial = [3,1]    #x[0],x[-1]

Ex, = VAR1Forecast(x_initial,A,t_max;m=1)       #applying VAR(1) function

p1 = plot( 0:t_max,Ex,
           label = "AR(2), (y₀,y₋₁) = (3,1)",
           linecolor = [:red :blue],
           linestyle = [:solid :dash],
            xlims = (0,10),
           xlabel = "period",
           title = "Forecasts based on t=0 info" )
display(p1)
```



Approximating an MA(q) by an AR(p)

by using the Yule-Walker equations.

The input is a vector of autocorrelations from an MA(q) model, and the output is the coefficients of an AR(p) model to replicate those.

In the figure we compare the autocorrelations implied by the AR(p) model to those of the original MA(q) model.

```
a2 = YuleWalker([1;ρMA2[1:2]])      #use the first 2 or 3 autocorrelations from an MA(2)
a3 = YuleWalker([1;ρMA2[1:3]])

printblue("MA(2)")
printmat([θ₁,θ₂],rowNames=["θ₁","θ₂"])

printblue("AR(2) coefs needed to approximate an MA(2)")
printmat(a2;rowNames=["a₁","a₂"])

printblue("AR(3) coefs needed to approximate an MA(2) even better")
printmat(a3;rowNames=["a₁","a₂","a₃"])
```

```
MA(2)
θ₁     0.500
θ₂     0.300

AR(2) coefs needed to approximate an MA(2)
a₁     0.492
a₂    -0.015

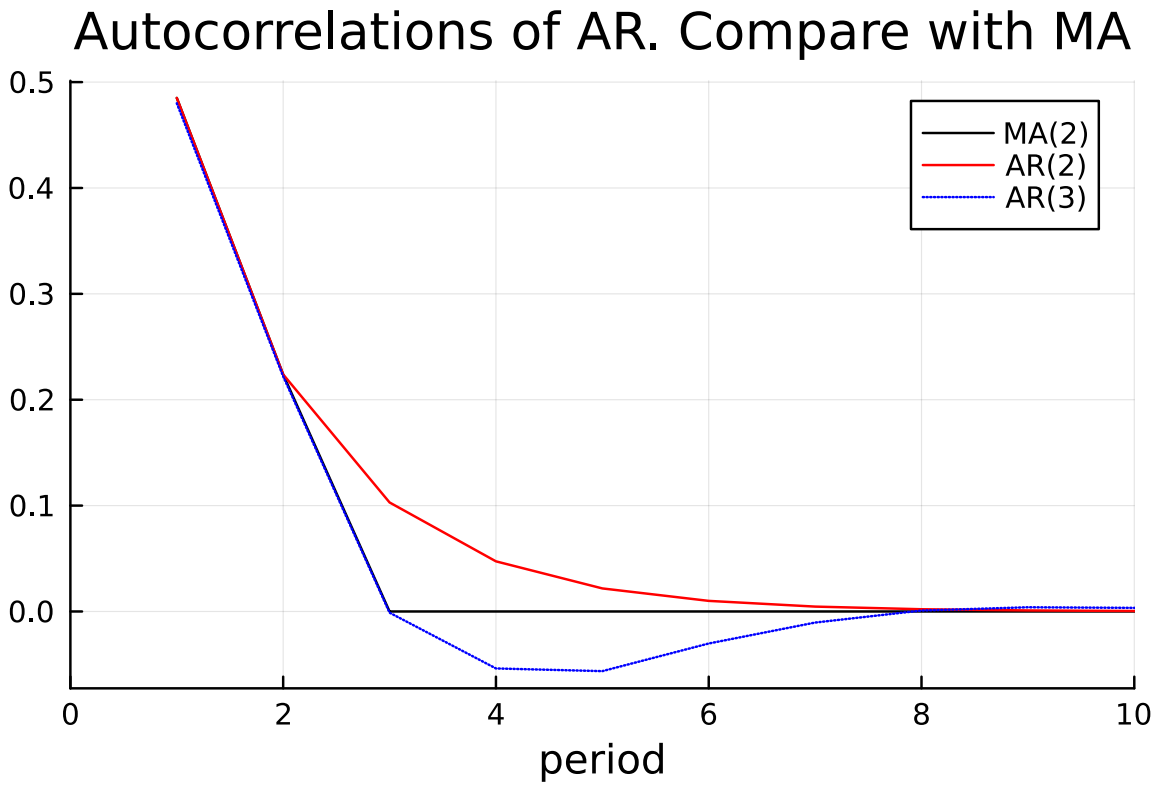
AR(3) coefs needed to approximate an MA(2) even better
a₁     0.490
a₂     0.051
a₃    -0.135
```

```
A_ar2, = CompanionFormAR(a2)                    #AR(2) to companion form
(_,_,ρ_ar2) = VAR1AutoCov(A_ar2,t_max;m=1)      #implied autocorrelations

A_ar3, = CompanionFormAR(a3)                    #AR(3) to companion form
(_,_,ρ_ar3) = VAR1AutoCov(A_ar3,t_max;m=1)      #implied autocorrelations

p1 = plot( 1:t_max,[ρMA2 ρ_ar2 ρ_ar3],
```

```
            label = ["MA(2)" "AR(2)" "AR(3)"],
            linecolor = [:black :red :blue],
            linestyle = [:solid :solid :dot],
            xlims = (0,10),
            xlabel = "period",
            title = "Autocorrelations of AR. Compare with MA" )
display(p1)
```



VAR(p)

This section some of the calculations for vector autoregressive (VAR) processes. Formally, this is very similar to the computations for the AR(p) above, since we used the approach of converting the AR() model to a VAR(1). However, there is one key difference: we are now interested in the results for more than one (the first) variable.

To illustrate this, consider a VAR(2)

$y_t = A_1 y_{t-1} + A_2 y_{t-2} + \epsilon_t,$

where y_t and y_{t-1} are vectors with two elements and

$$A_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.1 & -0.3 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} 0.1 & 0.1 \\ 0.1 & 0.2 \end{bmatrix}$$

or we change A_2 to a matrix of zeros to compare with the results in the lecture notes.

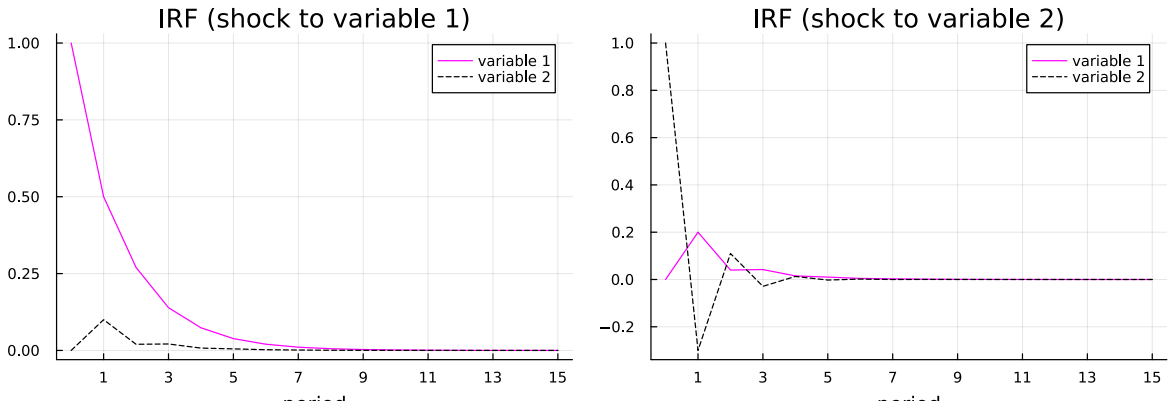
```
A1 = [0.5 0.2;
      0.1 -0.3]
#A2 = [0.1 0.1;                      #try also this
#      0.1 0.2]
A2 = zeros(2,2)

Ap = cat(A1,A2,dims=3)
A, = CompanionFormVAR(Ap)             #convert VAR(2) to larger VAR(1)
irf_var2 = VAR1IRF(A,t_max;m=2)

p1 = plot( 0:t_max,irf_var2[:,:,1],
           label = ["variable 1" "variable 2"],
           linecolor = [:magenta :black],
           linestyle = [:solid :dash],
           xticks = 1:2:t_max,
           xlabel = "period",
           title = "IRF (shock to variable 1)" )

p2 = plot( 0:t_max,irf_var2[:,:,2],
           label = ["variable 1" "variable 2"],
           linecolor = [:magenta :black],
           linestyle = [:solid :dash],
           xticks = 1:2:t_max,
           xlabel = "period",
           title = "IRF (shock to variable 2)" )

p = plot(p1,p2,layout=(1,2),size=(480*2,320))
display(p)
```



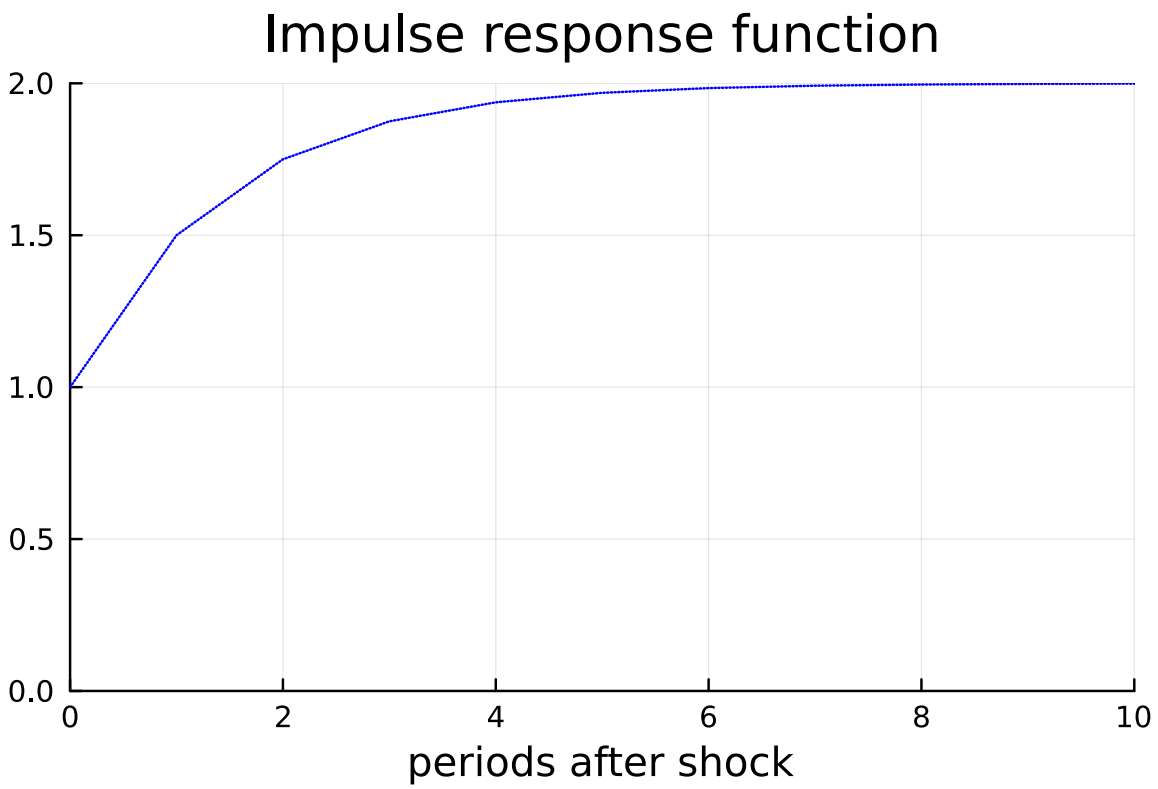
Non-Stationary Processes

The cells below shows the impulse response function and the roots of a non-stationary AR(3).

```
a = [1.5,-0.5]                      #AR(2) coefficients
A, = CompanionFormAR(a)

irf_ar3 = VAR1IRF(A,t_max-1;m=1)

p1 = plot( 0:t_max-1,irf_ar3,
           label = "",
           linecolor = :blue,
           linestyle = :dot,
           #xticks = 1:2:t_max,
           ylims = (0,2),
           xlims = (0,10),
           xlabel = "periods after shock",
           title = "Impulse response function" )
display(p1)
```




```
λ = eigen(A).values    #eigenvalues of the companion form

println("absolute values of the eigenvalues: should be < 1 for stationarity")
printmat(abs.(λ))
```

```
absolute values of the eigenvalues: should be < 1 for stationarity
     0.500
     1.000
```

Lasso and Elastic Net Regressions

This notebook presents a simple implementation of Lasso and elastic net regressions. It uses the OSQP.jl package for the numerical optimisation, rather than a tailor-made algorithm.

The purpose of using the Lasso and EN regressions is here to help with the model selection. These methods are useful for indicating which variables that are most important. With this variable selection done, we might choose to apply OLS on the chosen set of regressors.

Load Packages and Extra Functions

The key function for the LASSO/ridge/Elastic Net regressions is from the (local) `FinEcmt_Lasso` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS, FinEcmt_Lasso
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_Lasso.jl"))
using .FinEcmt_OLS, .FinEcmt_Lasso
=#
```

```
using Dates, DelimitedFiles, LinearAlgebra
```

```
using Plots, LaTeXStrings
default(size = (480,320),fmt = :png)
```

Loading Data

on monthly realized standard deviation for S&P500 (calculated from daily data) and regressors used to predict it.

```
(xx,header) = readdlm("Data/ModelSelectData.csv",',',header=true)

dN    = Date.(xx[:,1],"yyyy-mm-dd")                  #convert to Date
(y,x) = (Float64.(xx[:,2]),Float64.(xx[:,3:end]))    #dependent variable, regressors

(yName,xNames) = (header[2],vec(header[3:end]))
```

```
T = length(y)

println("Dependent variable: $yName \n")
println("Regressors: ")
printmat(xNames)
```

```
Dependent variable: ln std

Regressors:
ln std(-1)
ln std(-2)
ln VIX(-1)
ln VIX(-2)
     R(-1)
     R(-2)
```

An Elastic Net Regression

minimizes a combination of the Lasso and Ridge regression loss functions

$(Y - Xb)'(Y - Xb)/T + \gamma \sum |b_i - \beta_{i0}| + \lambda \sum (b_i - \beta_{i0})^2,$

where β_{i0} is the target for coefficient i (defaults to 0). Set $\lambda = 0, \gamma > 0$ to get a *Lasso* regression or instead $\lambda > 0, \gamma = 0$ to get a *ridge* regression.

Notice that we divide the sum of squared residuals by T. This helps the interpretation of the γ and λ values.

The problem is reformulated (see the code below) as a linear-quadratic problems with restrictions (see below). The code uses the `OSQP.jl` solver.

The function `LassoEN()` (included above) implements this.

```
#@doc LassoEN

@doc2 LassoEN
```

```
LassoEN(Y,X,γM=0,λ=0,β₀=0)
```

Do Lasso (set γ>0,λ=0), ridge (set γ=0,λ>0) or elastic net regression (set γ>0,λ>0). The function loops over the values in a vector γM, but requires λ to be a number (scalar).

Input

- `Y::Vector`:: T-vector, zero mean dependent variable
- `X::Matrix`: TxK matrix, zero mean regressors
- `γM::Vector`: nγ-vector with different values of γ (could also be a number)
- `λ::Number`: value of λ (a number)
- `β₀::Vector`: K-vector of target levels for the coeffs (could also be a common number)

Remark (details on the coding)

Choice variables `z = [b;t]` with lengths K and K respectively

The objective

`0.5*z'P*z + q'z`

effectively involves

`0.5*z'P*z =  b'(X'X/T+λI)b` and

`q'z = (-2X'Y/T-2λβ₀)'b + γ1't`

The restrictions `lb <= Az <= ub` imply

`-∞ <= β-t    <= β₀`

`β₀ <= β-t    <= ∞`

Requires

using OSQP, SparseArrays, LinearAlgebra

```
#using CodeTracking
#println(@code_string LassoEN([1],[1],0.1))    #print the source code
```

Lasso Regression

The next cell makes a Lasso regression for a single value of γ. The dependent variable is the (return of the) first mutual fund in `RFunds` (see data loading) and the regressors are (returns on) a number of benchmark portfolios (again, see data loading).

The data is standardised to have zero mean and unit variance before the estimation, by using the `StandardiseYX()` function (included above).

```
(Y,X) = StandardiseYX(y,x)

γ = 10^(-1.5)
(b,b_ls) = LassoEN(Y,X,γ)

printblue("OLS and Lasso coeffs (with γ=$(round(γ,digits=3))):\n")
printmat(b_ls,b;colNames=["OLS","Lasso"],rowNames=xNames,width=15)
```

```
OLS and Lasso coeffs (with γ=0.032):

                      OLS          Lasso
ln std(-1)          0.158          0.146
ln std(-2)          0.039          0.006
ln VIX(-1)          0.609          0.594
ln VIX(-2)         -0.042          0.000
R(-1)              -0.051         -0.047
R(-2)               0.003         -0.000
```

Redo the Lasso Regression with Different (gamma) Values

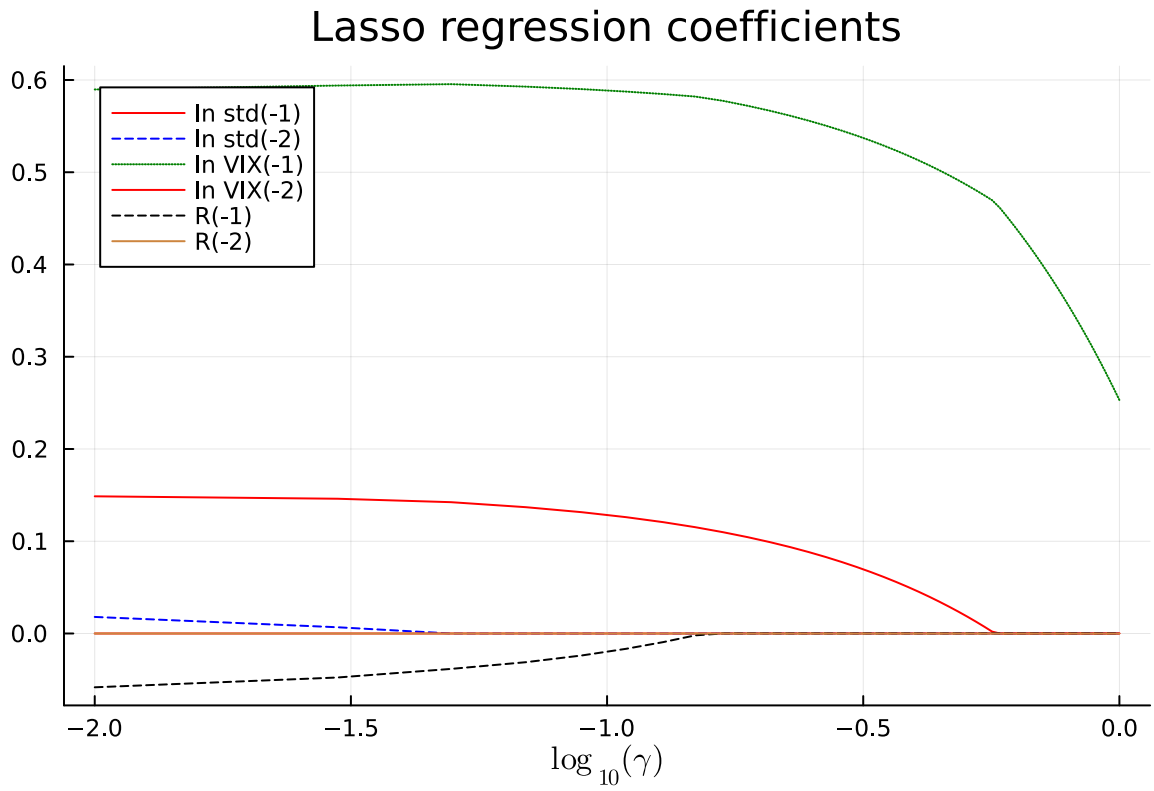
The function `LassoEN()` can loop over γ values (and exploit the fact that this only requires a partial updating of the problem).

You can also change the target level β_0 to be something else than zero.

```
nγ = 51
γM = range(0.01,1,length=nγ)
β₀ = 0.0     #change to eg. 0.1 to set another target, or specify a vector

bLasso, = LassoEN(Y,X,γM,0,β₀);
#printmat([γM bLasso'])
```

```
p1 = plot( log10.(γM),bLasso',
           title = "Lasso regression coefficients",
           color = [:red :blue :green :red :black :peru],
           linestyle = [:solid :dash :dot :solid :dash :solid],
           xlabel = L"\log_{10}(\gamma)",
           label = permutedims(xNames),
           legend = :topleft,
           size = (600,400) )
display(p1)
```



A Ridge Regression

minimizes

$(Y - Xb)'(Y - Xb)/T + \sum \lambda_i (b_i - \beta_{i0})^2$,

where β_{i0} is the target for coefficient i (defaults to 0) and λ_i the penalty on deviations from the target. We compare the results from using linear algrebra and optimization based ones from the `LassoEN()` function. A function for this is included above.

```
β₀ = 0              #change to eg. 0.1 to set another target

b_LS     = X\Y
b_ridge  = RidgeRegression(Y,X,0.1,β₀)
b_ridge2, = LassoEN(Y,X,0,0.1,β₀)

printblue("OLS and ridge regression:\n")
printmat(b_ls,b_ridge,b_ridge2;colNames=["OLS","analytical ridge","numerical ridge"],rowNames=xNames
```

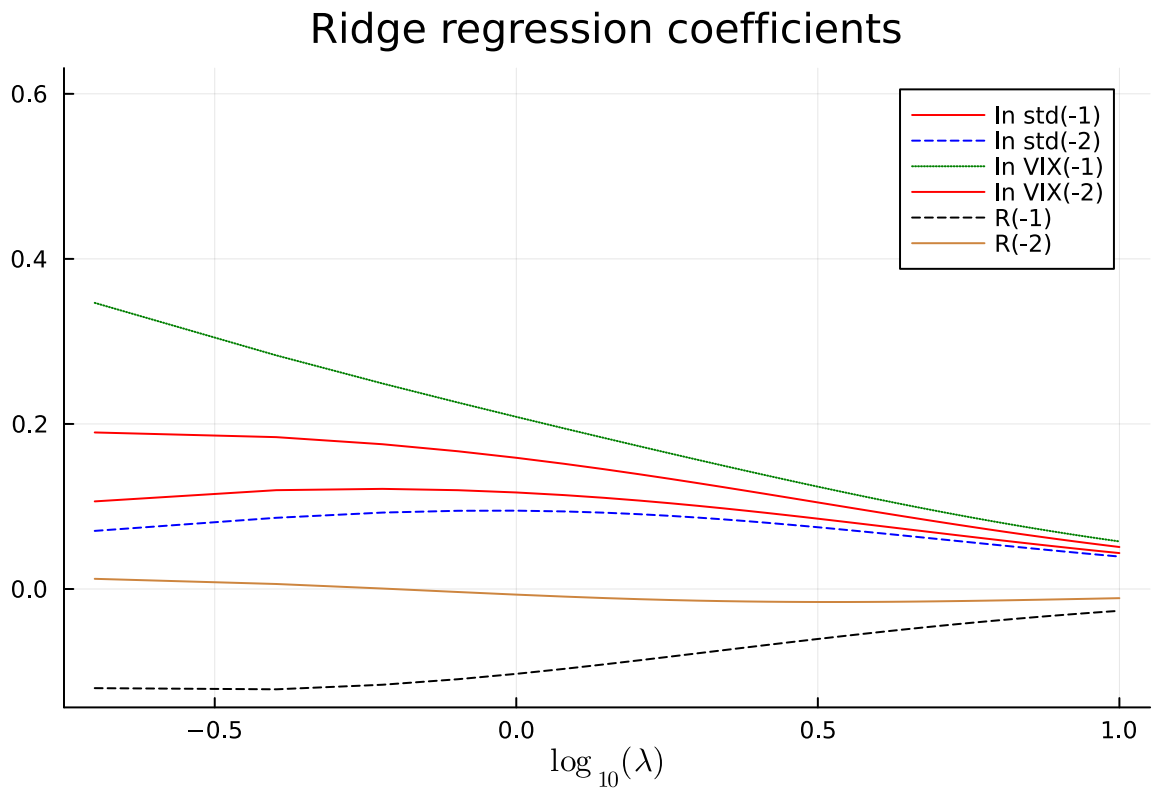
OLS and ridge regression:

	OLS	analytical ridge	numerical ridge
ln std(-1)	0.158	0.186	0.186
ln std(-2)	0.039	0.055	0.055
ln VIX(-1)	0.609	0.415	0.415
ln VIX(-2)	-0.042	0.080	0.080
R(-1)	-0.051	-0.108	-0.108
R(-2)	0.003	0.014	0.014

```
nλ = 51
λM = range(0,10,length=nλ)

bridge   = fill(NaN,size(X,2),nλ)
for i in 1:nλ                          #loop over λ values
    bridge[:,i] = RidgeRegression(Y,X,λM[i])
end
```

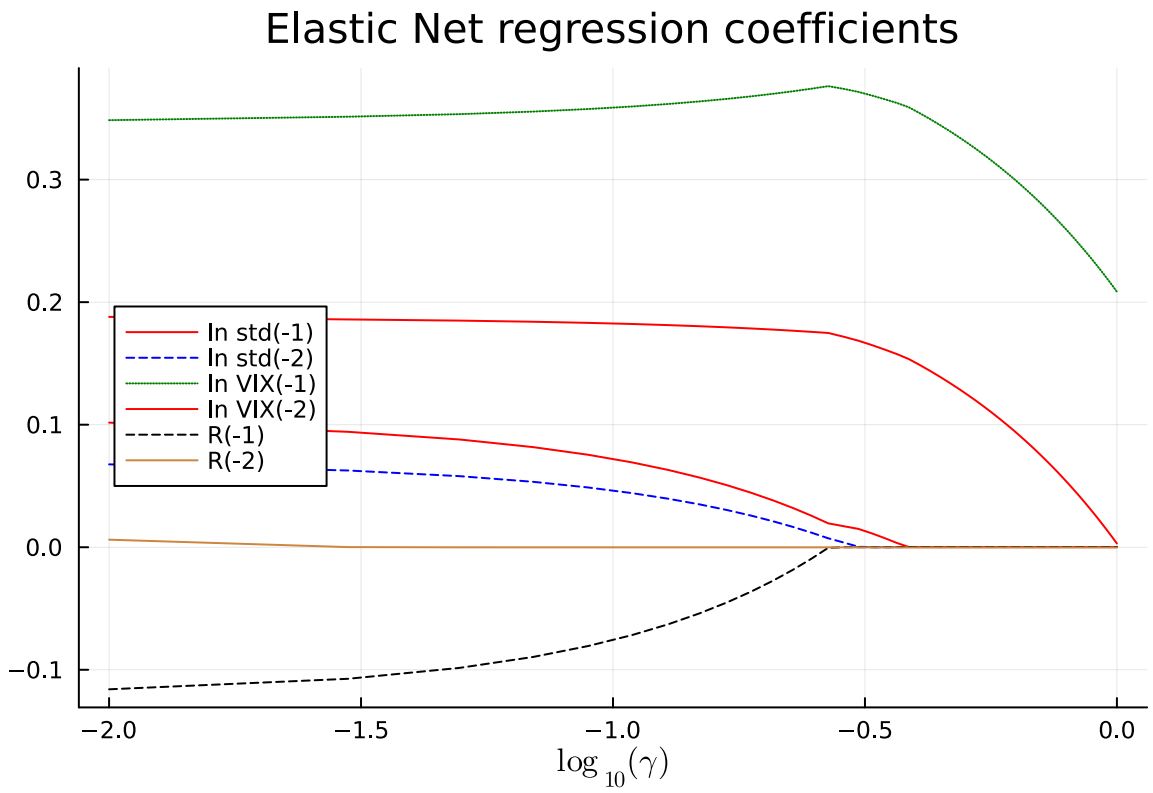
```
p1 = plot( log10.(λM),bridge',
           title = "Ridge regression coefficients",
           color = [:red :blue :green :red :black :peru],
           linestyle = [:solid :dash :dot :solid :dash :solid],
           xlabel = L"\log_{10}(\lambda)",
           label = permutedims(xNames),
           size = (600,400) )
display(p1)
```



An Elastic Net Regression

```
λ = 0.2
bEN, = LassoEN(Y,X,γM,λ);
```

```
p1 = plot( log10.(γM),bEN',
           title = "Elastic Net regression coefficients",
           color = [:red :blue :green :red :black :peru],
          linestyle = [:solid :dash :dot :solid :dash :solid],
           xlabel = L"\log_{10}(\gamma)",
           label = permutedims(xNames),
           legend = :left,
           size = (600,400) )
display(p1)
```



OLS on the Most Important Variables

The Lasso approach suggests that some regressors are much more important than others. We now apply OLS to this set.

```
vv = [1,3]
(Y,X) = (y,[x[:,vv] ones(T)])

(b,_,_,V,) = OlsGM(Y,X)
RegressionTable(b,V,vcat(xNames[vv],"c"))
```

```
                coef    stderr    t-stat   p-value
ln std(-1)     0.156     0.061     2.563     0.010
ln VIX(-1)     0.888     0.086    10.264     0.000
c             -0.387     0.146    -2.651     0.008
```

Efficient Markets

This notebook describes and tests the predictability of asset returns (autocorrelations, autoregressions, out-of-sample R^2, Mariano-Diebold test) and implements a simple trading strategy.

Load Packages and Extra Functions

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS, FinEcmt_TimeSeries
using FinEcmt_MLEGMM: MLE
```

```
 using LinearAlgebra, Dates, DelimitedFiles, Statistics, StatsBase

using Plots
default(size = (480,320),fmt = :png)
```

Load Data

Daily excess return on S&P 500.

```
xx = readdlm("Data/ReSP500Daily.csv",',',skipstart=1)
Re = Float64.(xx[:,2])
dN = Date.(xx[:,1],"yyyy-mm-dd")
xx = nothing

(Re,dN) = excise(Re,dN) #drop dates with missing values
T   = size(Re,1)
println("Sample size: $T")
```

```
Sample size: 11479
```

Autocorrelations

Autocorrelation s is

$\rho_s = \text{Corr}(R_t, R_{t-s})$

In large samples, $\sqrt{T}\hat{\rho}_s \sim N(0,1)$ if the true value is $\rho_s = 0$ for all s.

A Remark on the Code

The StatsBase.jl package contains methods for estimating autocorrelations (see `autocor()` below).

```
plags = 1:5        #different lags to calculate autocorrelations for
ρ     = AutoCorr(Re,plags)

printblue("autocorrelations and their t-stats:\n")
printmat(ρ,sqrt(T)*ρ;colNames=["ρ","t-stat"],rowNames=plags,cell00="lag")
```

```
autocorrelations and their t-stats:

lag         ρ    t-stat
1      -0.054    -5.768
2      -0.006    -0.624
3      -0.007    -0.786
4      -0.032    -3.436
5      -0.003    -0.278
```

Autoregressions

An AR(1) is

$R_t = c + a_1 R_{t-1} + \varepsilon_t.$

We also consider an asymmetric AR(1)

$R_t = \alpha + \beta\delta_{t-1}R_{t-1} + \gamma(1-\delta_{t-1})R_{t-1} + \varepsilon_t,$

where $\delta_{t-1} = 1$ if $R_{t-1} < 0$ and zero otherwise.

Both models can be estimated by OLS.

A Remark on the Code

- The function `OlsGM()` does OLS and reports, among other things, the variance-covariance matrix of the estimates (called `Covb` below).
- To get standard errors, we first extract the variances from the variance-covariance matrix (`diag(Covb)`) and then calculate the square roots (`sqrt.()`). Notice the dot. in the latter calculation: to calculate the square root of each element in the vector of variances.
- The t-stats are then just the point estimates (minus the null hypothesis, which is here 0) divided by the standard errors.

```
x  = [ones(T-1) Re[1:end-1]]            #R(t) is regressed on (1,R(t-1))
(b,_,_,Covb,) = OlsGM(Re[2:end],x)
Stdb  = sqrt.(diag(Covb))
tstat = b./Stdb

printblue("Results from an AR(1):\n")
printmat([b tstat];colNames=["coef","t-stat"],rowNames=["constant","slope"])
```

```
Results from an AR(1):

              coef    t-stat
constant     0.029     2.777
slope       -0.054    -5.775
```

```
δ = Re[1:end-1] .< 0        #dummy: 1 if R(t-1) < 0
x = hcat(ones(T-1), δ.*Re[1:end-1], (1.0.-δ).*Re[1:end-1])  #R(t) is regressed on (1,Q(t-1)R(t-1),[1-Q

(b,_,_,Covb,) = OlsGM(Re[2:end],x)
Stdb  = sqrt.(diag(Covb))
tstat = b./Stdb

printblue("Results from an AR(1) with dummies:\n")
printmat([b tstat];colNames=["coef","t-stat"],rowNames=["constant","slope (down)","slope (up)"])
```

```
Results from an AR(1) with dummies:

                  coef    t-stat
constant         0.004     0.266
slope (down)    -0.086    -5.666
slope (up)      -0.018    -1.131
```

Recursive Estimation and Out-of-Sample R2

The next cell does recursive estimation (longer and longer sample) and predicts one period ahead (outside of the sample). The performance of this prediction model is measured by an "out-of-sample" R^2_{OOS} defined as

$$R^2_{OOS} = 1 - \frac{\text{MSE(forecasting model)}}{\text{MSE(benchmark forecast)}}$$

A Remark on the Code

- The code loops over `t=100:Tb` where `Tb` is the effective length of the sample. Inside the loop we use data on `y[1:t-1]` and `x[1:t-1,:]`. This means that the first estimation uses the first 99 observations, the second estimation the first 100 observations and so forth. After the loop, we discard the first 100 observations from the output since they are just `NaN`.
- For estimation we use the basic OLS function `OlsM()`. Clearly, `OlsGMFn()` gives the same results (but is a tiny bit slower).

```
y   = Re[2:end]     #traditional symbol for the LHS variable
Tb  = length(y)     #length of the effective sample
dNb =  dN[2:end]

x   = hcat(ones(Tb), δ.*Re[1:end-1], (1.0.-δ).*Re[1:end-1])     #predictors

(ϵ,e) = (fill(NaN,Tb),fill(NaN,Tb))
for t in 100:Tb
    local b
    b,    = OlsGM(y[1:t-1],x[1:t-1,:])    #estimate on sample 1:t-1
    ϵ[t] = y[t] - x[t,:]'b         #forecast error for t, model
    e[t] = y[t] - mean(y[1:t-1])   #forecast error for t, historical average
end
(ϵ,e,dNb) = (ϵ[100:end],e[100:end],dNb[100:end])    #drop periods without forecasts

MSE_Model = mean(abs2,ϵ)   #MSE for out-of-sample forecasts, same as mean(ϵ.^2) but quicker
MSE_Bench = mean(abs2,e)
R2oos     = 1 - MSE_Model/MSE_Bench

printblue("Performance of out-of-sample forecasting:\n")
printmat([MSE_Model;MSE_Bench;R2oos];rowNames=["MSE of AR(1)","MSE of hist avg","R2_oos"])
```

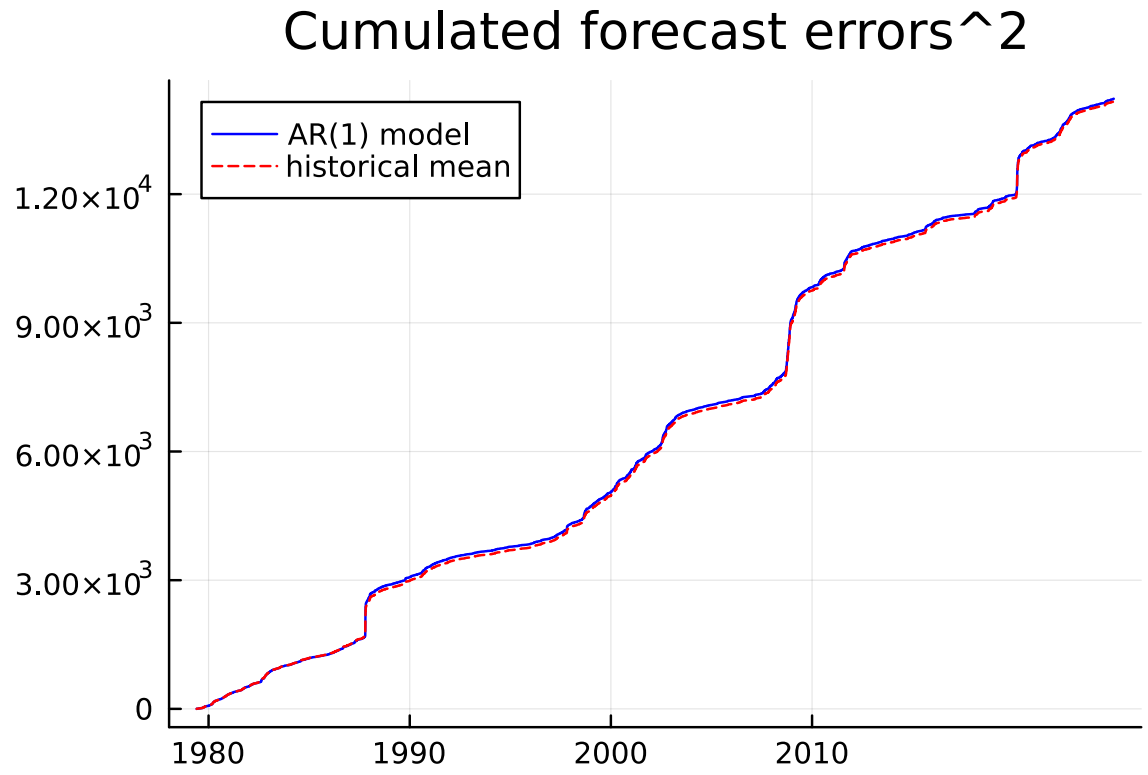
```
Performance of out-of-sample forecasting:

MSE of AR(1)        1.250
```

```
MSE of hist avg     1.245
R2_oos             -0.004
```

```
xTicksLoc = [Date(1980);Date(1990);Date(2000);Date(2010)]
xTicksLab = Dates.format.(xTicksLoc,"Y")     #formatting of date axis

p2 = plot( dNb,[cumsum(e.^2) cumsum(e.^2)],
      linecolor = [:blue :red],
      linestyle = [:solid :dash],
      label = ["AR(1) model" "historical mean"],
      legend = :topleft,
      xticks = (xTicksLoc,xTicksLab),
      title = "Cumulated forecast errors^2" )
display(p2)
```



Mariano-Diebold and Clark-West Tests

The Mariano-Diebold and Clark-West tests compare the prediction errors of two models (e: benchmark; ϵ: your model). Notice that the Mariano-Diebold test is not well suited for nested models (when your

model is an augmented version of the baseline model). Use the Clark-West in that case.

If $g_1 = e^2$ and $g_2 = \epsilon^2$ are not autocorrelated, then the standard deviation of the sample average is $\text{Std}(g_i)/\sqrt{T}$. For simplicity, this assumption is used below. (Otherwise, use a Newey-West estimator or similar.)

```
"""
    MDCW(e,ϵ)

Moment conditions doing MD or CW tests of two series of forrecast errors (e: benchmark, ϵ: from the large

"""
MDCW(e,ϵ) = hcat(e.^2 - ϵ.^2, 2*e.*(e - ϵ))   #Mariano-Diebold and Clark&West
```

```
MDCW
```

```
g  = MDCW(e,ϵ)          #e,ϵ are from the recursive estimation (above)
Tg = size(g,1)

μ      = mean(g,dims=1)
Stdμ   = std(g,dims=1)/sqrt(Tg)
tstats = μ./Stdμ

printblue("t-stats of tests of difference in performance (errors^2)\n")
printmat(tstats';rowNames=["Mariano-Diebold","Clark-West"])
```

```
t-stats of tests of difference in performance (errors^2)

Mariano-Diebold    -0.481
Clark-West          0.495
```

A Trading Strategy

This section implements a momentum strategy (buy past winners, short sell past losers), which is rebalanced daily. For simplicity, we disregard trading costs.

Implementing the Strategy

1. Rank the 25 assets according to the lagged return R_{t-1}.
2. (In the evening of) period $t-1$ buy 1/5 of each of the 5 best assets based on the ranking in point 1. Similarly, buy -1/5 (short-sell) each of the 5 worst assets. Collect these portfolio weights in a vector w_t.
3. In period t , the return on the portfolio is $R_{p,t} = w_t' R_t$.
4. Repeat for all periods

A Remark on the Code

- With `x = [9,7,8]`, the `rankPs(x)` function (see below) gives the output `[3,1,2]`. This says, for instance, that `7` is the lowest number (rank 1).
- The portfolio weights are not stored for use outside the loop. Only the strategy (portfolio) return is.

Loading Data

The data set contains daily data of the equity market return, riskfree rate and the returns of the 25 Fama-French portfolios. All returns are in percent.

```
x   = readdlm("Data/MomentumSR.csv",',')
dN  = Date.(x[:,1],"yyyy-mm-dd")
x   = Float64.(x[:,2:end])
(Rm,Rf,R) = (x[:,1],x[:,2],x[:,3:end])

println("size of dN, Rm, Rf, R")
println(size(dN),"\n",size(Rm),"\n",size(Rf),"\n",size(R))

Re25 = R .- Rf              #excess returns
Rme  = Rm .- Rf

(T,n) = size(Re25);          #number of periods and assets
```

```
size of dN, Rm, Rf, R
(11598,)
(11598,)
(11598,)
(11598, 25)
```

```
"""
    rankPs(x)

Calculates the ordinal rank of eack element in a vector `x`. As an aternative,
use `ordinalrank` from the `StatsBase.jl` package.

"""
rankPs(x) = invperm(sortperm(x))
```

rankPs

```
m = 5   #number of assets in long/short leg of portfolio
L = 22  #window of signal starts here

Rₚ = fill(NaN,T)
for t in L+1:T           #loop over periods, save portfolio returns
    #local r,w,z       #local/global is needed in script
    z     = vec(mean(Re25[t-L:t-1,:],dims=1))
    r     = rankPs(z)
    w     = -(r.<=m)/m + ((n-m+1).<=r)/m      #short, long leg
    Rₚ[t] = w'Re25[t,:]            #same as sum(w.*R[t,:])
end

Rₚ = Rₚ[L+1:end];
```

We calculate the mean (excess) return, its standard deviation and the Sharpe ratio. Annualising is done by assuming 250 trading days per year (means are multiplied by 250 and standard deviation by $\sqrt{250}$). Compare with the excess return on passively holding an equity market index.

```
μ = mean(Rₚ)                 #strategy
σ = std(Rₚ)

Rme = Rm - Rf                #market
μₘ  = mean(Rme[L+1:end])
σₘ  = std(Rme[L+1:end])

printblue("Annualised results:\n")
result  = [μ*250;σ*sqrt(250);μ/σ*sqrt(250)]
resultₘ = [μₘ*250;σₘ*sqrt(250);μₘ/σₘ*sqrt(250)]
printmat(result,resultₘ,colNames=["Strategy","market"],rowNames=["mean","std","SR"])
```

```
Annualised results:

      Strategy    market
mean     9.510     8.836
std     11.160    17.334
SR       0.852     0.510
```

To cumulate the returns to a return index, use $(1 + R_1)$, $(1 + R_1)(1 + R_2)$, etc. However, this does not work for excess returns, so convert them to net returns by adding the riskfree rate.

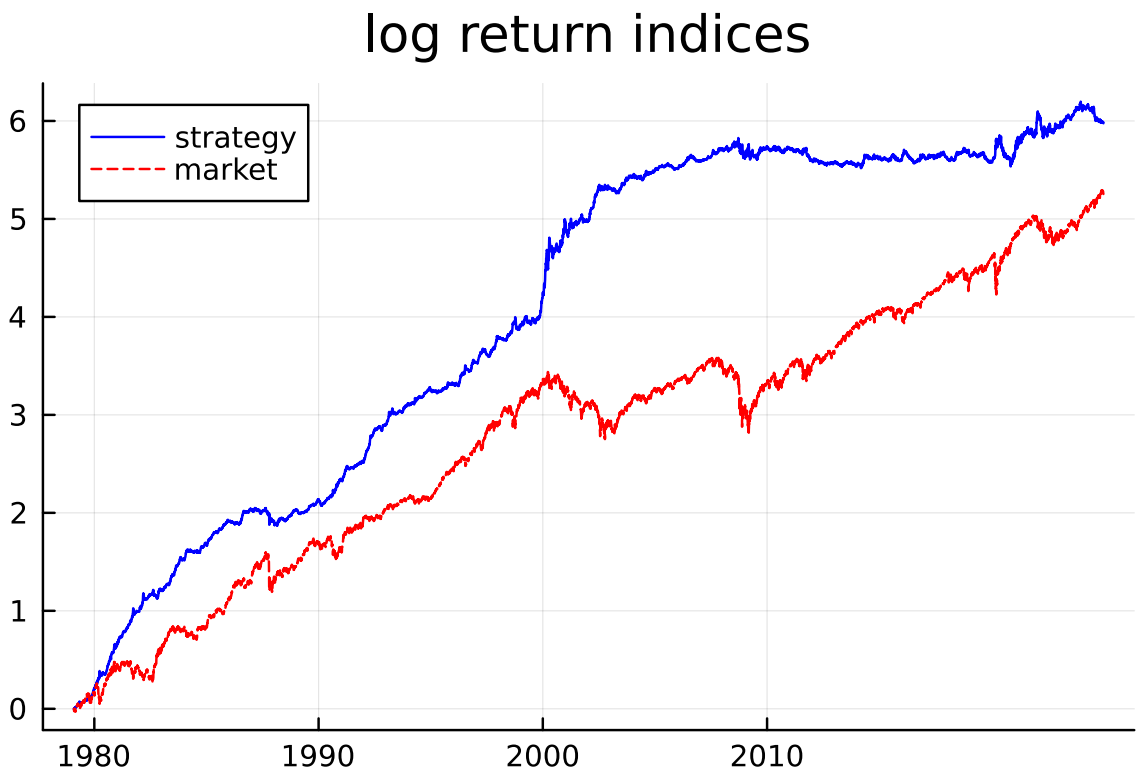
It is often more useful to show the logarithm of the return index. The slope can then be interpreted as a return.

```
Rbₚ = Rₚ + Rf[L+1:end]                #add Rf to make it a net return

Vₚ = cumprod(1 .+ Rbₚ/100)            #cumulate to return index ("value")
Vₘ = cumprod(1 .+ Rm[L+1:end]/100);  #notice /100 since percentage returns
```

```
xTicksLoc = [Date(1980);Date(1990);Date(2000);Date(2010)]
xTicksLab = Dates.format.(xTicksLoc,"Y")

plot( dN[L+1:end],log.([Vₚ Vₘ]),
      linecolor = [:blue :red],
      linestyle = [:solid :dash],
      label = ["strategy" "market"],
      legend = :topleft,
      xticks = (xTicksLoc,xTicksLab),
      title = "log return indices" )
```


log return indices
strategy
market
6
5
4
3
2
1
0
1980
1990
2000
2010

Estimating and Testing Distributions

This notebook illustrates basic descriptive statistics (means, std, skewness, kurtosis), histograms, the Jarque-Bera test (of normality), Q-Q plots and tests of the empirical distribution function (Kolmogorov-Smirnov). It uses the Distributions.jl and StatsBase.jl packages.

You may also consider the HypothesisTests.jl package.

Load Packages and Extra Functions

The key functions used in this notebook (`JarqueBeraTest()` and `KolSmirTest()`) are from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics, Distributions, StatsBase
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Load Data from a csv File

```
xx = readdlm("Data/ReSP500Daily.csv",',',skipstart=1)
x  = Float64.(xx[:,2])
xx = nothing
x = excise(x)               #delete NaNs

T   = size(x,1)
println("Sample size: $T")
```

```
Sample size: 11479
```

Histograms

The next few cells shows some basic descriptive statistics for the variable x (US equity market excess return).

```
μ = mean(x)
σ = std(x)
(xmin,xmax) = extrema(x)

printblue("Basic descriptive stats:\n")
xx = [μ,σ,xmin,xmax]
printmat(xx;rowNames=["μ","σ","min","max"])
```

```
Basic descriptive stats:

μ       0.027
σ       1.112
min   -20.485
max    10.787
```

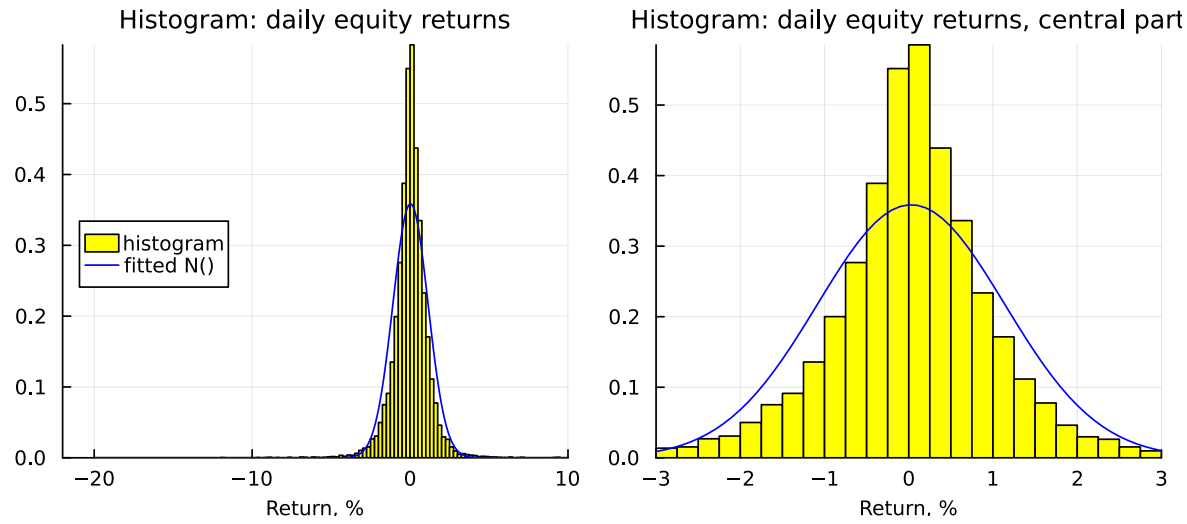
```
xGrid = -20:0.1:10
pdfX  = pdf.(Normal(μ,σ),xGrid)        #"Distributions.jl" wants σ, not σ^2

p1 = histogram( x,bins = -20:0.25:12,
                normalized = true,     #normalized to have area=1
                label = "histogram",
                color = :yellow,
                legend = :left,
                xlim = (-22,10),
```

```
                title = "Histogram: daily equity returns",
                titlefontsize = 10,
                xlabel = "Return, %",
                guidefontsize = 8 )
plot!(xGrid,pdfX,linewidth=1,color=:blue,label="fitted N()")

p2 = histogram( x,bins = -5:0.25:5,
                normalized = true,
                color = :yellow,
                legend = false,
                xlim = (-3,3),
                title = "Histogram: daily equity returns, central part",
                titlefontsize = 10,
                xlabel = "Return, %",
                guidefontsize = 8 )
plot!(xGrid,pdfX,linewidth=1,color=:blue,legend=false)

pAll = plot(p1,p2,layout=(1,2),size=(700,300))                #set up subplots
display(pAll)
```

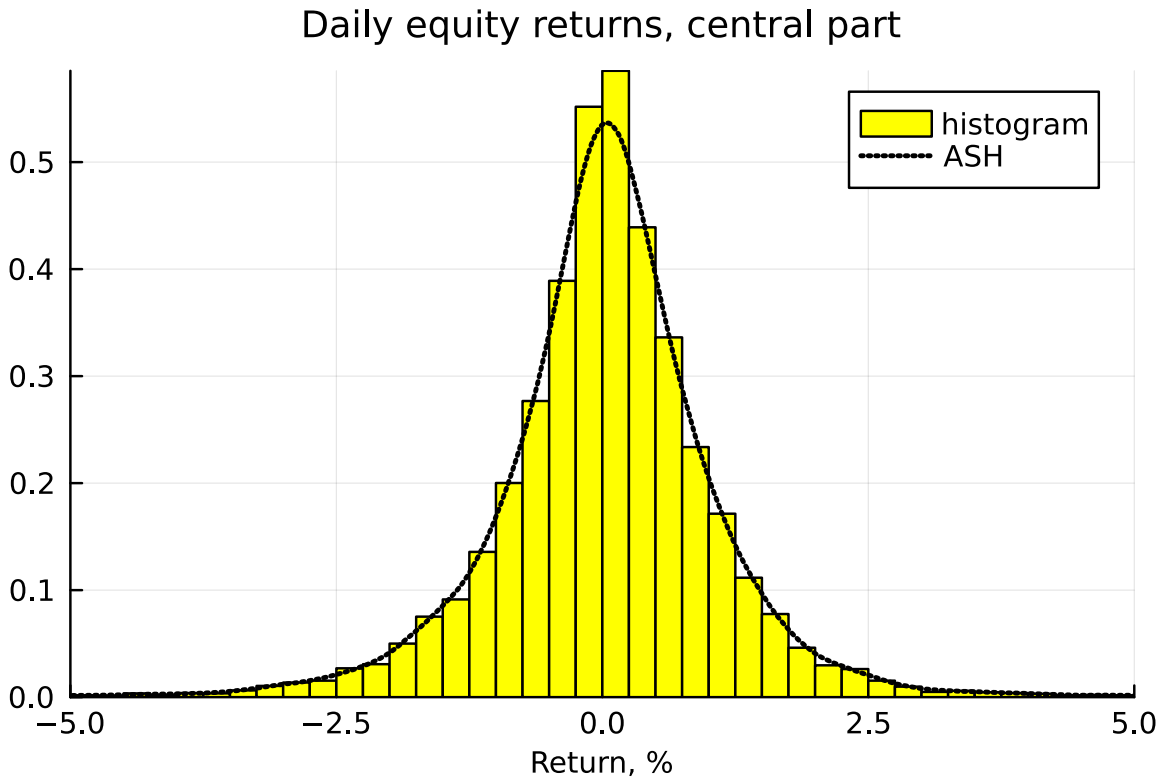


Averaged Shifted Histogram

An averaged shifted histogram (ASH) first calculates histogram values over many bins and then averages across nearby bins. The output is a smoother histogram and something that comes very close to a kernel density estimate (see below).

```
(ashPdf1,BinMid_ash1,_) = HistAsh(x,-5:0.5:5,21)   #21 shifts

p2 = histogram( x,bins = -5:0.25:5,
                normalized = true,
                label = "histogram",
                color = :yellow,
                xlim = (-5,5),
                title = "Daily equity returns, central part",
                titlefontsize = 10,
                xlabel = "Return, %",
                guidefontsize = 8 )
plot!(BinMid_ash1,ashPdf1,linestyle=:dot,linewidth=2,color=:black,label="ASH")
display(p2)
```



Q-Q Plot

The Q-Q plot shows the empirical quantiles against the theoretical quantiles (possibly from an estimated distribution). If the theoretical distribution is a good fit to the data, then the results should cluster closely

around a 45 degree line.

A Remark on the Code

The `quantile()` function for a distribution (from the `Distributions.jl` package) works slightly different from the `quantile()` function for data (from the `StatsBase.jl` package). In short,

- `quantile.(Normal(μ,σ),pval1)` calculates a vector of quantiles of a `Normal(μ,σ)` distribution.
- `quantile(x,pval1)` calculates a vector of empirical quantiles from the vector `x` (notice: no dot).

As an alternative, do `dist = fit(Normal,x); quantile.(dist,pval1)`

```
pval1 = 0.02:0.01:0.98                          #quantiles 0.01 to 0.99, to QQ plot
pval2 = collect(0.0025:0.0025:0.01)                    #extreme quantiles
pval3 = collect(0.99:0.0025:0.9975)



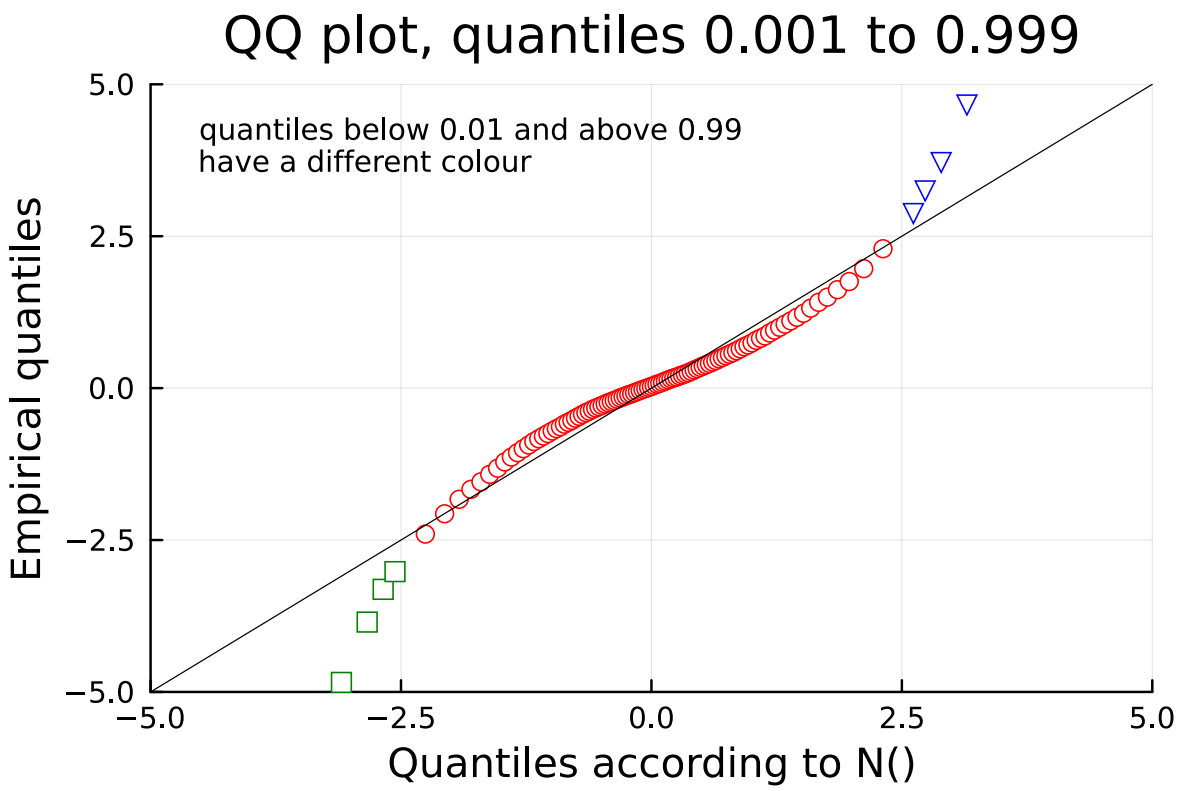
qEmp1 = quantile(x,pval1)                       #empirical quantiles, pval1; no dot
qEmp2 = quantile(x,pval2)                       #same, but for pval2
qEmp3 = quantile(x,pval3)
qN1   = quantile.(Normal(μ,σ),pval1)            #quantiles of N()
qN2   = quantile.(Normal(μ,σ),pval2)
qN3   = quantile.(Normal(μ,σ),pval3)
```

```
4-element Vector{Float64}:
 2.615025420308094
 2.7329645458145944
 2.892525117712593
 3.149695237130577
```

```
txt = text("quantiles below 0.01 and above 0.99\nhave a different colour",8,:left)

p1 = scatter( qN1,qEmp1,legend=false,
              markercolor = :white,
              markerstrokecolor = :red,
              xlim = (-5,5),
              ylim = (-5,5),
              title = "QQ plot, quantiles 0.001 to 0.999",
              xlabel = "Quantiles according to N()",
              ylabel = "Empirical quantiles",
              annotation = (-4.5,4,txt) )
```

```
scatter!(qN2,qEmp2,markercolor=:white,markerstrokecolor=:green,markershape=:rect,legend=false)
scatter!(qN3,qEmp3,markercolor=:white,markerstrokecolor=:blue,markershape =:dtriangle,legend=false)
plot!([-5,5],[-5,5],color=:black,linewidth=0.5)
display(p1)
```



Skewness, Kurtosis and the Jarque-Bera Test

The `JarqueBeraTest()` function reports skewness, kurtosis and the B-J statistic.

```
@doc2 JarqueBeraTest
```

```
JarqueBeraTest(x)
```

Calculate the JB test for each column in a matrix. Reports `(skewness,kurtosis,JB)`.

```
#using CodeTracking
#println(@code_string JarqueBeraTest([1]))
```

```
(skewness,kurtosis,JB,pvals) = JarqueBeraTest(x)

#critval_BJ   = quantile(Chisq(2),0.9)                  #critical values
#critval_skew = quantile(Normal(0,sqrt(6/T)),0.9)
#critval_kurt = quantile(Normal(0,sqrt(24/T)),0.9)

printblue("More descriptive stats:\n")
xx = hcat([skewness;kurtosis-3;JB],collect(pvals))
printmat(xx;colNames=["Estimate","p-value"],
            rowNames=["skewness","excess kurtosis","Jarque-Bera"],width=15)
```

```
More descriptive stats:

                     Estimate        p-value
skewness                -0.804          0.000
excess kurtosis         18.771          0.000
Jarque-Bera         169756.417          0.000
```

The Empirical Distribution Function and the Kolmogorov-Smirnov Test

The empirical distribution function (edf) shows the frequence of data points below a given threshold. The Kolmogorov-Smirnov (K-S) test is designed to investigate whether the edf differs from a theoretical (possibly estimated) cdf. It is implemented in the function `KolSmirTest()`.

An edf does essentially the following: rank all elements in `x` and divide the ranks by `T` to get relative ranks (0 to 1).

The K-S test finds the largest difference between the edf and the theoretical cdf, D. The 5% critical value of $\sqrt{T}D$ is 1.36.

```
@doc2 KolSmirTest
```

```
KolSmirTest(x1,TheoryCdf::Function)
```

Calculate the Kolmogorov-Smirnov test

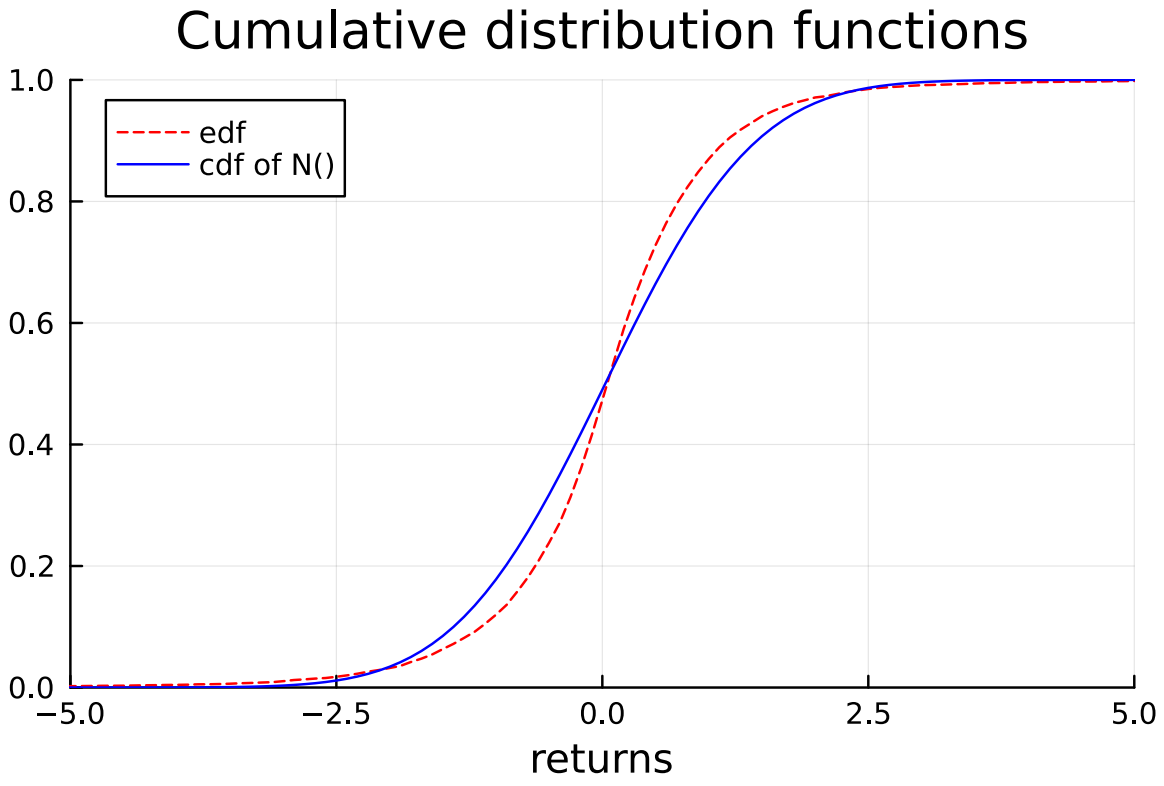
Output

- `KSstat::Float64`: KS test statistic
- `xD::Number`: x value with the largest diff beteen empirical and theoretical cdf

```
#println(@code_string KolSmirTest([1],cos))
```

```
edfx = ecdf(x)                    #construct the empirical distribution function
                                  #from the StatsBase.jl package
edfEmp = edfx(xGrid)             #evaluate the empirical distribution function at xGrid
cdfN   = cdf.(Normal(μ,σ),xGrid);       #cdf of Normal(μ,σ), for comparison
```

```
p1 = plot( xGrid,[edfEmp cdfN],
           label = ["edf" "cdf of N()"],
           linecolor = [:red :blue],
           linestyle = [:dash :solid],
           title = "Cumulative distribution functions",
           xlabel = "returns",
           xlim = (-5,5),
           ylim = (0,1) )
display(p1)
```



```
(KSstat,xD) = KolSmirTest(x,z->cdf(Normal(μ,σ),z))

printblue("Kolmogorov-Smirnov test:\n")
printmat([KSstat,1.36,xD];rowNames=["sqrt(T)*D","5% critval","at which x"])
```

```
Kolmogorov-Smirnov test:

sqrt(T)*D      8.603
5% critval     1.360
at which x    -0.489
```

Kernel Density Estimate

The `KernelDensity()` function estimates the probability density function (pdf) by using a gaussian kernel. The bandwidth parameter (`h`) can be supplied by the caller, otherwise it defaults to the rule of thumb value $h = 1.06\text{std}(x_t)/T^{0.2}$. In practice, the choice of kernel is less important that the choice of bandwidth.

The estimate of the pdf at value $\tilde{x}$ is $\hat{f}(\tilde{x}) = \sum_{t=1}^{T} w(u_t)/(Th)$,

where $u_t = (x_t - \tilde{x})/h$ and $w()$ is one of the kernel functions listed below.

The function outputs both the estimate the estimate of pdf(x), denoted `fx`, and its standard deviation, denoted `Stdfx`. The values are estimated at a grid of `x` values, supplied by the user.

As an alternative to this code, consider the KernelDensity.jl package. Also, compare with the ASH used before.

```
using FinEcmt_KernelRegression
```

```
@doc2 KernelDensity
```

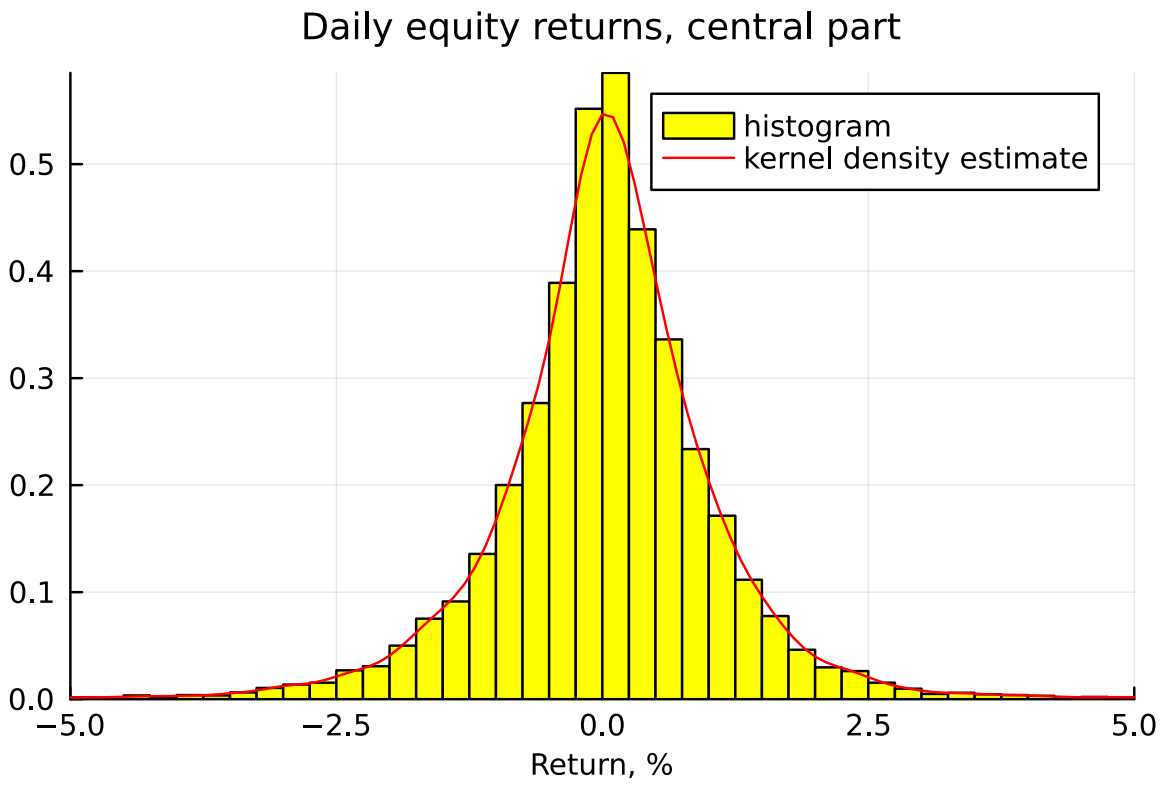
```
KernelDensity(x,xGrid,h=[],KernelFun=GaussianKernel)
```

Compute a kernel density estimate at each value of the grid `xGrid`, using the data in vector `x`. The bandwidth `h` can be specified (otherwise a default value is used). The kernel function defaults to a standard normal density function, but other choices are available.

```
#using CodeTracking
#println(@code_string KernelDensity([1],[1]))
```

```
xGrid = -5:0.1:5                              #use many different values in the grid
(fx,Stdfx) = KernelDensity(x,xGrid)

p2 = histogram( x,bins = -5:0.25:5,
                normalized = true,
                label = "histogram",
                color = :yellow,
                xlim = (-5,5),
                title = "Daily equity returns, central part",
                titlefontsize = 10,
                xlabel = "Return, %",
                guidefontsize = 8 )
plot!(xGrid,fx,linewidth=1,color=:red,label="kernel density estimate")
display(p2)
```



Maximum Likelihood Estimation

This notebook illustrates maximum likelihood estimation and how to calculate different standard errors (from the information matrix, the gradients and the "sandwich" approach).

The application is very simple: estimating the mean and variance of a random variable. Some of the subsequent chapters (notebooks) work with more complicated models, for instance, GARCH models.

Load Packages and Extra Functions

The notebook first implements MLE step-by-step. At the end it also presents the `MLE()` function from the (local) `FinEcmt_MLEGMM` module that wraps those calculations.

For the numerical optimization we use the Optim.jl package and for calculating derivatives the FiniteDiff.jl package. (ForwardDiff.jl is an alternative.)

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
using FinEcmt_MLEGMM: MLE     #load only the MLE() function
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_MLEGMM.jl"))
using .FinEcmt_OLS
using .FinEcmt_MLEGMM: MLE     #load only the MLE() function
=#
```

```
using LinearAlgebra, DelimitedFiles, Statistics, Optim

#loading and renaming some functions from FiniteDiff (to get shorter names)
using FiniteDiff: finite_difference_hessian as hessian, finite_difference_jacobian as jacobian
```

Loading Data

```
xx = readdlm("Data/ReSP500Daily.csv",',',skipstart=1)
x  = Float64.(xx[:,2])
xx = nothing
x = excise(x)               #delete NaNs

T   = size(x,1)
println("Sample size: $T")
```

```
Sample size: 11479
```

Traditional Estimates

of the mean μ and the variance σ^2.

The standard errors of the mean and standard deviation are from traditional textbook formulas.

```
T = length(x)

(μ_trad,σ²_trad) = (mean(x),var(x,corrected=false))   #corrected=false gives 1/T formula, not 1/(T-1)

std_trad = sqrt.([σ²_trad,2*σ²_trad^2]/T)   #standard errors, textbook formulas

printblue("Traditional estimates and their std:\n")
xx = [[μ_trad,σ²_trad] std_trad]
printmat(xx;colNames=["estimate","std"],rowNames=["μ","σ²"])
```

```
Traditional estimates and their std:

    estimate       std
μ      0.027     0.010
σ²     1.237     0.016
```

Point Estimates from ML

The next few cells define a log likelihood function and estimate the coefficients by maximizing it.

The (log) Likelihood Function for Estimating the Parameters of a N(,)

A Remark on the Code

- (μ,σ²) = par splits up the vector par into two numbers (for the mean and variance)

```
"""
    NormalLL(par,x)

Calculate log likelihood for a `N(μ,σ²)` distribution.

`par = [μ,σ²]` is a vector with the parameters , `x` is a vector with data
"""
function NormalLL(par,x)
    (μ,σ²) = (par[1],abs(par[2]))                #make sure σ²>=0
    σ      = sqrt(σ²)
    z      = (x .- μ)./σ
    LLt    = logpdfNorm.(z) .- log(σ)            #shorter, from local module
   #LLt   = -(1/2)*log(2*pi) - (1/2)*log(σ²) .- (1/2)*abs2.(x.-μ)/σ²  #vector, all x[t]
    return LLt
end;
```

Try the Likelihood Function

```
par0 = [0.0,1.0]           #initial parameter guess of [μ,σ²]

LLt = NormalLL(par0,x)     #just trying the log likelihood fn

println("log likelihood value at par0: ",sum(LLt))
```

```
log likelihood value at par0: -17653.242
```

Optimize the Likelihood Function

```
Sol = optimize(par->-sum(NormalLL(par,x)),par0,BFGS())  #minimize -sum(LLt)
parHat = Optim.minimizer(Sol)                    #the optimal solution

println("log-likelihood at point estimate (compare with the value above): ",-Optim.minimum(Sol))
```

```
printblue("\nParameter estimates:\n")
xx = [[μ_trad,σ²_trad] parHat]
printmat(xx;colNames=["traditional","MLE"],rowNames=["μ","σ²"],width=13)
```

```
log-likelihood at point estimate (compare with the value above): -17509.267

Parameter estimates:

    traditional          MLE
μ         0.027        0.027
σ²        1.237        1.237
```

Standard Errors I: Information Matrix

If the likelihood function is correctly specified, then MLE is typically asymptotically normally distributed as

$\sqrt{T}(\hat{\theta} - \theta) \rightarrow^d N(0, V)$, where $V = I(\theta)^{-1}$ with

$I(\theta) = -\mathrm{E}\frac{\partial^2 \ln L_t}{\partial\theta\partial\theta'}$

where $I(\theta)$ is the information matrix (*not* an identity matrix) and $\ln L_t$ is the contribution of period t to the log likelihood function.

A Remark on the Code

The code below calculates numerical derivatives. It does so by noticing that $\mathrm{E}\frac{\partial^2 \ln L_t}{\partial\theta\partial\theta'} = \frac{\partial^2 \mathrm{E} \ln L_t}{\partial\theta\partial\theta'}$, so we can differentiate the mean (across data points) log likelihood value. (Clearly, we could equally well calculate T different 2×2 matrices and then average.)

- `hessian(par->mean(NormalLL(par,x)),parHat)` calculates the 2nd derivatives of the average log-likelihood function.

```
Ia = -hessian(par->mean(NormalLL(par,x)),parHat) #(-1) * 2nd derivatives of mean(LLt)

Ia       = (Ia+Ia')/2         #to guarantee symmetry, fixes possible rounding errors
vcv    = inv(Ia)/T       #variance-covariance matrix of estimates (not sqrt(T)*estimates)
std_hess = sqrt.(diag(vcv))

printblue("standard errors:\n")
```

```
xx = [std_trad std_hess]
printmat(xx;colNames=["traditional","MLE (InfoMat)"],rowNames=["μ","σ²"],width=18)
```

```
standard errors:

         traditional     MLE (InfoMat)
μ              0.010             0.010
σ²             0.016             0.016
```

Standard Errors II: Gradients and Sandwich

An alternative way of calculating the information matrix is to use the outer product of the gradients:

$J(\theta) = \mathrm{E}\left[\frac{\partial \ln L_t}{\partial \theta}\frac{\partial \ln L_t}{\partial \theta'}\right]$

This would coincide with $I(\theta)$ as defined above, in a very large sample and if the model is correctly specified.

A Remark on the Code

The code below fills row t of a $T \times 2$ matrix (called `δL`) with $\frac{\partial \ln L_t}{\partial \theta}$. For each t, the outer product is a 2×2 matrix, and then we average across t. This is done by calculating `J = δL'δL/T`. A loop would also work well.

Std from the Gradients

```
δL = jacobian(par->NormalLL(par,x),parHat)   #Tx2
J  = δL'δL/T               #mean outer product, 2xT * Tx2, could loop instead

vcv      = inv(J)/T
std_grad = sqrt.(diag(vcv))                          #std from gradients

printblue("standard errors:\n")
xx = [std_trad std_hess std_grad]
printmat(xx;colNames=["traditional","MLE (InfoMat)","MLE (gradients)"],rowNames=["μ","σ²"],width=18)
```

```
standard errors:

          traditional     MLE (InfoMat)   MLE (gradients)
μ               0.010             0.010             0.011
σ²              0.016             0.016             0.005
```

Std from the Sandwich Estimator

We could also use the "sandwich" estimator

$V = I(\theta)^{-1} J(\theta) I(\theta)^{-1}$.

When the model (likelihood function) is misspecified, then the three variance-covariance matrices may differ, and the sandwich approach is often the most robust.

```
vcv       = inv(Ia) * J * inv(Ia)/T
std_sandw = sqrt.(diag(vcv))                          #std from sandwich

printblue("standard errors:\n")
xx = [std_trad std_hess std_grad std_sandw]
printmat(xx,colNames=["traditional","MLE (InfoMat)","MLE (gradients)","MLE (sandwich)"],rowNames=["μ
```

```
standard errors:

          traditional     MLE (InfoMat)   MLE (gradients)    MLE (sandwich)
μ               0.010             0.010             0.011             0.010
σ²              0.016             0.016             0.005             0.053
```

Try this: replace the data series `x` with simulated data from a $N()$ distribution. Then, do the different standard errors get closer to each other?

A Function for MLE (extra)

The next cell uses a function which combines the computations of several of the cells above. This function will be used in later notebooks.

It requires a function for the log-likehood function as written above, that is, taking `(par,x)` as inputs and generating a T-vector `LLt` as output.

```
@doc2 MLE
```

```
MLE(LLtFun,par0,y,x,lower,upper)
```

Calculate ML point estimates of K parameters and three different types of standard errors: from the Information matrix, from the gradients and the sandwich approach.

Input

- `LLtFun::Function`: name of log-likelihood function
- `par0::Vector`: K-vector, starting guess of the parameters
- `y::VecOrMat`: vector or matrix with the dependent variable
- `x::VecOrMat`: vector or matrix with data, use `nothing` if not needed
- `lower::Vector`: lower bounds on the parameters, nothing or fill(-Inf,K) if no bounds
- `upper::Vector`: upper bounds on the parameters, nothing or fill(Inf,K) if no bounds

Requires

- `using FiniteDiff: finite_difference_hessian as hessian, finite_difference_jacobian as jacobian`

Notice

The `LLtFun` should take `(par,y,x)` as inputs and generate a T-vector `LLt` as output.

```
#using CodeTracking
#println(@code_string MLE(NormalLL,[1],[1],[1]))    #print the source code
```

```
LLtFun(par,y,x) = NormalLL(par,y)
(parHat,std_hess,std_grad,std_sandw,LL_t) = MLE(LLtFun,par0,x,nothing)

printblue("point estimates and standard errors:\n")
xx = [parHat std_hess std_grad std_sandw]
printmat(xx;colNames=["estimate","std (InfoMat)","std (gradients)","std (sandwich)"],rowNames=["μ","
```

```
point estimates and standard errors:

            estimate     std (InfoMat)   std (gradients)    std (sandwich)
μ              0.027             0.010             0.011             0.010
σ²             1.237             0.016             0.005             0.053
```

Volatility Models: GARCH

This notebook estimates different GARCH models.

As an alternative, consider the ARCHModels.jl package.

Load Packages and Extra Functions

This notebook uses likelihood functions from the `FinEcmt_TimeSeries` module and then calls on the `MLE()` function from the `FinEcmt_MLEGMM` module. You may want to look at the MLE notebook before the current one.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS, FinEcmt_TimeSeries
using FinEcmt_MLEGMM: MLE
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_TimeSeries.jl"))
include(joinpath(pwd(),"src","FinEcmt_MLEGMM.jl"))
using .FinEcmt_OLS, .FinEcmt_TimeSeries
using .FinEcmt_MLEGMM: MLE
=#
```

```
using Dates, DelimitedFiles, Statistics, LinearAlgebra
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Loading Data

The data is for *net* S&P 500 returns, not excess returns. The reason is that VaR etc are typically expressed in terms of returns, rather than in terms of excess returns.

```
xx = readdlm("Data/RSP500Daily.csv",',',skipstart=1)   #net returns
dN = Date.(xx[:,1])
R  = Float64.(xx[:,2])
xx = nothing

vv = FindNN(R)                          #not missings
(dN,R) = (dN[vv],R[vv])

y = R[2:end]                            #dependent variable, y(t)
x = [ones(size(y)) R[1:end-1]]          #regressors, [1, y(t-1)]

T   = size(y,1)
println("Sample size: $T")
```

```
Sample size: 11589
```

AR(1) + GARCH(1,1) Model

The next cells estimate an AR(1) where the residuals follow a different GARCH models. The results are used to calculate a time-varying (daily) value at risk and time-varying correlations.

The Likelihood Function

Consider a regression equation, where the residual follows a GARCH(1,1) process

$y_t = x_t'b + u_t$ with $u_t = v_t\sigma_t$ and

$\sigma_t^2 = \omega + \alpha u_{t-1}^2 + \beta\sigma_{t-1}^2$.

Notice that we require (ω, α, β) to all be positive and $\alpha + \beta < 1$.

If $v_t \sim N(0,1)$, then the likelihood function is $\sum_{t=1}^{T} \ln L_t$ where

$\ln L_t = \ln \phi(u_t/\sigma_t) - \ln \sigma_t$.

The likelihood function of a GARCH(1,1) model is in `garch11LL`.

A Remark on the Code

- For simplicity, the σ_t^2 is calculated in a loop. As an alternative, consider the `filt()` function from the DSP.jl package.
- the code is in `src/Garch.jl`. The next cell prints the loglikelihood function.

```
@doc2 garch11LL
```

```
garch11LL(par,y,x)
```

Calculate (`LL_t,σ²,yhat,u`) for regression `y = x'b + u` where u follows a GARCH(1,1) process with paramaters (`ω,α,β`).

Input

- `par::Vector`: parameters, `[b;ω;α;β]`
- `y::VecOrMat`: Tx1, dependent variable
- `x::VecOrMat`: Txk, regressors

```
#using CodeTracking
#println(@code_string garch11LL([1],[1] [1]))    #print the source code
```

Try the GARCH(1,1) Likelihood Function

…just to see that it delivers something.

```
par0   = [mean(y),0,var(y)*0.05,0.05,0.90]       #initial guess of b0,b1,ω,α,beta
(LLt,) = garch11LL(par0,y,x)                     #trying the log lik

printlnPs("Value of log-likelihood fn at starting guess of the parameters: ",sum(LLt))
```

```
Value of log-likelihood fn at starting guess of the parameters: -15856.467
```

Maximize the GARCH(1,1) Likelihood Function

by using the `MLE()` function from the `FinEcmt_MLEGMM` module. It wants at least four inputs: (`LLfunction,startingGuess,y,x`), but can also accept lower and upper bounds on the estimated parameters.

```
lower = [-Inf,-0.99, 0,0,0]      #upper and lower bounds on the parameters: b0,b1,ω,α,beta
upper = [ Inf, 0.99,10,1,1]

garch11LL_(par,y,x) = garch11LL(par,y,x)[1]    #to get a function with just one output as MLE wants
(parHat,std_hess,std_grad,std_sandw,) = MLE(garch11LL_,par0,y,x,lower,upper)

printblue("point estimates and standard errors:\n")
coefNames = ["b₀","b₁","ω","α","β"]
xx = [parHat std_hess std_grad std_sandw]
printmat(xx;colNames=["estimate","std (InfoMat)","std (gradients)","std (sandwich)"],rowNames=coefNa
```

```
point estimates and standard errors:

          estimate   std (InfoMat) std (gradients)  std (sandwich)
b₀           0.064           0.007           0.008           0.008
b₁           0.001           0.010           0.010           0.010
ω            0.019           0.002           0.001           0.004
α            0.094           0.006           0.002           0.014
β            0.890           0.007           0.003           0.015
```

Value at Risk from GARCH(1,1)

calculated by assuming conditional (time-varying) normality,

$\text{VaR} = -(\mu_t - 1.645\sigma_t)$,

where μ_t are the predictions from the estimated mean equation ($x_t'b$) and σ_t from the GARCH(1,1) model.

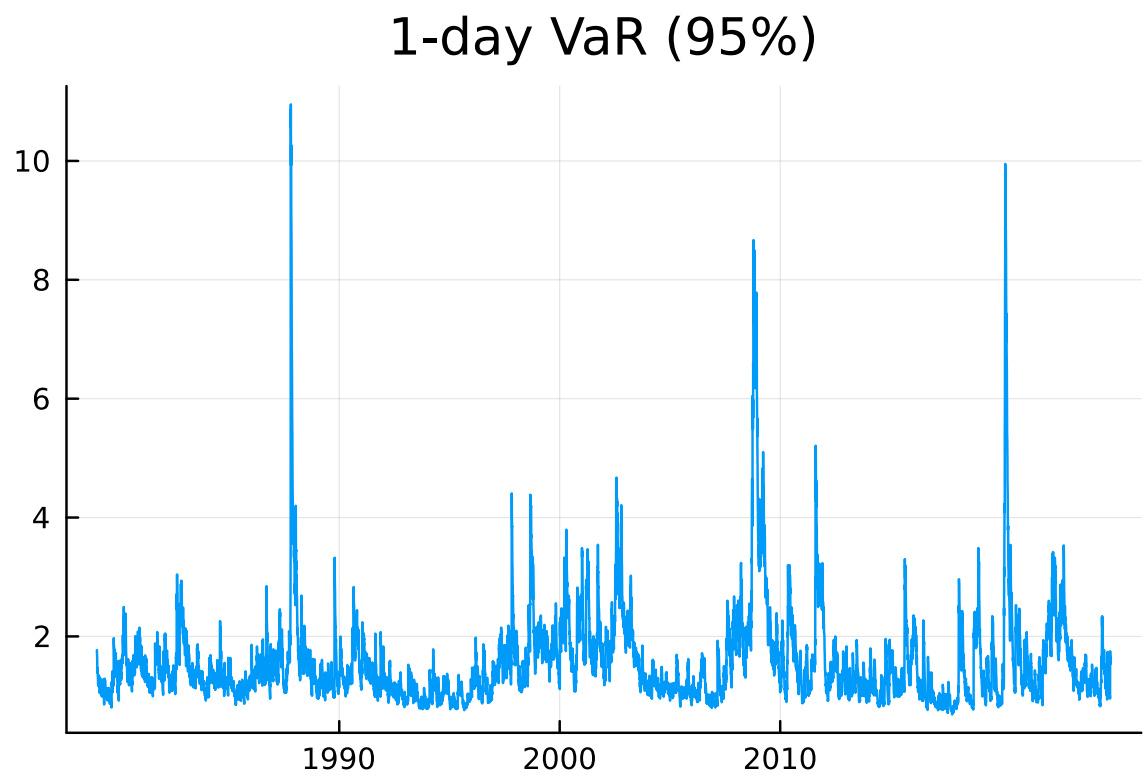
The fitted values of μ_t and σ_t are calculated by calling `garch11LL(parHat,y,x)`, that is, using the point estimates and the data in the likelihood function. The outputs are `(LL_t,σ²,yhat,u,v)`, so we need the 2nd and 3rd.

```
(σ²,μ) = garch11LL(parHat,y,x)[2:3]      #get the fitted values of the σ²- and μ-vectors
VaR95  = -(μ - 1.645*sqrt.(σ²))

xTicksLoc = [Date(1990),Date(2000),Date(2010)]
xTicksLab = Dates.format.(xTicksLoc,"Y")

p1 = plot( dN[2:end],VaR95,
           xticks = (xTicksLoc,xTicksLab),
           legend = false,
```

```
           title = "1-day VaR (95%)" )
display(p1)
```




```
CovRatio = mean((-y) .>= VaR95)             #coverage ratio for VaR
println("Coverage ratio for VaR(95%): ",CovRatio)
```

```
Coverage ratio for VaR(95%):      0.052
```

eGARCH(1,1)

This section estimates an eGARCH model.

The structure of the likelihood function `egarch11LL()` is similar to `garch11LL()` used before.

```
# @doc2 egarch11LL
```

```
#using CodeTracking
#println(@code_string egarch11LL([1],[1],[1]))    #print the source code
```

Estimate an eGARCH(1,1) model

```
par0 = [mean(y),0,var(y)*0.05,0.05,0.90,0]   #initial parameter guess: b0,b1,ω,α,β,γ

lower = [-Inf,-0.99,-Inf,-Inf,0,-Inf]       #upper and lower bounds on the parameters
upper = [ Inf, 0.99, Inf, Inf,1, Inf]

egarch11LL_(par,y,x) = egarch11LL(par,y,x)[1]          #to get a function with just one output
(parHat,std_hess,std_grad,std_sandw,) = MLE(egarch11LL_,par0,y,x,lower,upper)

printblue("eGARCH(1,1), parameter estimates\n")
coefNames = ["b₀","b₁","ω","α","β","γ"]
printmat(parHat,std_sandw;colNames=["coef","std (sandw)"],rowNames=coefNames)
```

```
eGARCH(1,1), parameter estimates

        coefstd (sandw)
b₀     0.033     0.007
b₁     0.001     0.010
ω     -0.121     0.012
α      0.154     0.016
β      0.975     0.004
γ     -0.106     0.012
```

Volatility Models: DCC

This notebook estimates a multivariate GARCH model. We first apply an EWMA approach, then MLE.

As an alternative, consider the ARCHModels.jl package.

Load Packages and Extra Functions

This notebook uses likelihood functions from the (local) `FinEcmt_TimeSeries` module and then calls on the `MLE()` function from the (local) `FinEcmt_MLEGMM` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS, FinEcmt_TimeSeries
using FinEcmt_MLEGMM: MLE
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_TimeSeries.jl"))
include(joinpath(pwd(),"src","FinEcmt_MLEGMM.jl"))
using .FinEcmt_OLS, .FinEcmt_TimeSeries
using .FinEcmt_MLEGMM: MLE
=#
```

```
using Dates, DelimitedFiles, Statistics, LinearAlgebra
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Load Data

for two return series (DAX and FTSE).

```
xx  = readdlm("Data/DAXFTSE.csv",',',skipstart=1)
dN  =
dN = Date.(xx[:,1])
R  = Float64.(xx[:,2:end])
xx  = nothing

R1 = R[2:end,1]                          #DAX
T  = length(R1)
x1 = [ones(T) R[1:end-1,1]]        #regressors, [1, R1(t-1)]

R2 = R[2:end,2]                          #FTSE
x2 = [ones(T) R[1:end-1,2]];       #regressors, [1, R2(t-1)]
```

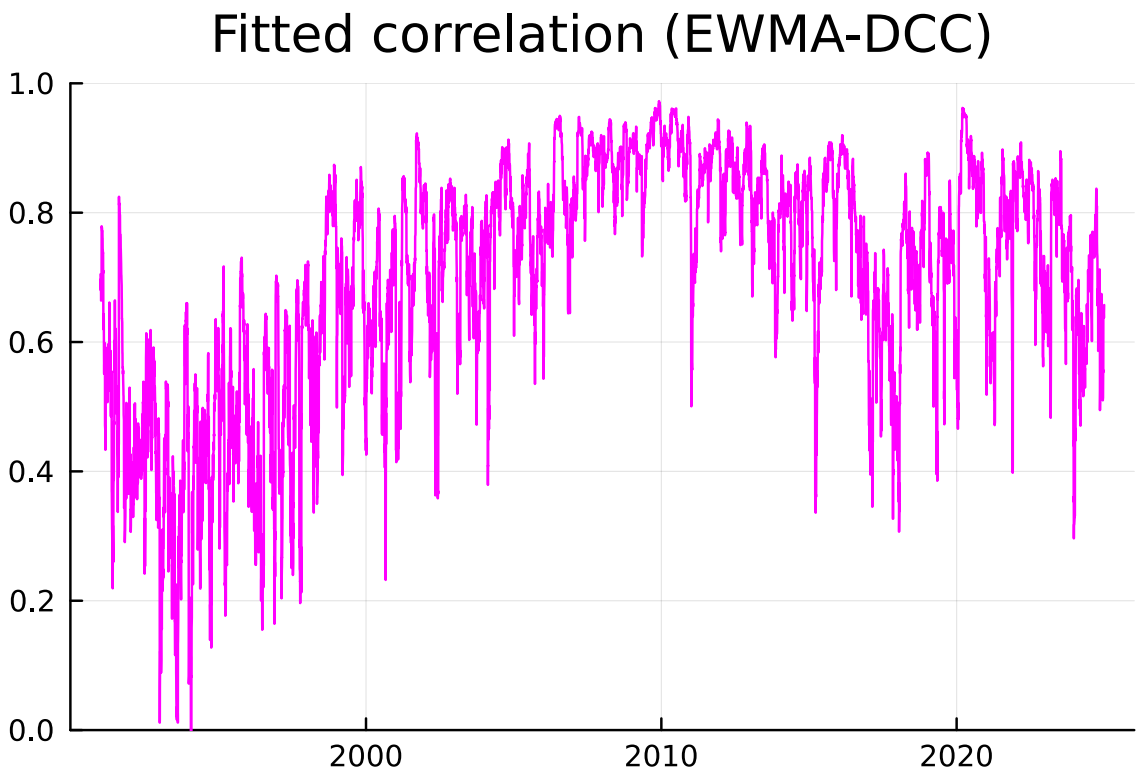
EWMA-DCC

We first apply the DCC approach by using an EWMA approach.

```
λ = 0.95
(_,R,_,_) = Dcc_EWMA([R1 R2],λ)               #R is a correlation matrix

xTicksLoc = [Date(2000),Date(2010),Date(2020)]
xTicksLab = Dates.format.(xTicksLoc,"Y")

p1 = plot( dN[2:end],R[:,1,2],
           linecolor = :magenta,
           xticks = (xTicksLoc,xTicksLab),
           legend = false,
           ylims  = (0.0,1),
           title = "Fitted correlation (EWMA-DCC)" )
display(p1)
```



DCC: Estimate Univariate eGARCH(1,1) Models

for each of the two return series. Also, calculate the standardized residuals as $v_t = u_t/\sigma_t$.

The estimation is done by the `MLE()` function from the `FinEcmt_MLEGMM` module. It wants at least four inputs: (`LLfunction,startingGuess,y,x`), but can also accept lower and upper bounds on the estimated parameters.

```
y = [R1 R2]                    #Tx2 (Txn), so we can easily loop over y[:,i]
x = cat(x1,x2,dims=3)          #TxKx2
(T,K,n) = size(x)

par0 = [mean(R1),0,var(R1)*0.05,0.05,0.90,0]  #initial parameter guess: b0,b1,ω,α,β,γ

lower = [-Inf,-0.99,-Inf,-Inf,0,-Inf]          #upper and lower bounds on the parameters
upper = [ Inf, 0.99, Inf, Inf,1, Inf];
```

```
egarch11LL_(par,y,x) = egarch11LL(par,y,x)[1];    #to get a function with just one output, practical

(v,σ²) = (fill(NaN,T,n),fill(NaN,T,n)) #to fill with results, stand. residuals, variances
parHat   = fill(NaN,K+4,n)        #estimated parameters
for i in 1:n             #loop over estimations of y[:,i]
    local y_i, x_i, parHat_i
    (y_i,x_i) = (y[:,i],x[:,:,i])
    parHat_i, = MLE(egarch11LL_,par0,y_i,x_i,lower,upper)    #estimate
    (_,σ²[:,i],_,_,v[:,i]) = egarch11LL(parHat_i,y_i,x_i)    #fitted values
    parHat[:,i] = parHat_i
end

printblue("eGARCH(1,1), parameter estimates:\n")
coefNames = ["b₀","b₁","ω","α","β","γ"]
printmat(parHat;colNames=["asset 1","asset 2"],rowNames=coefNames)
```

```
eGARCH(1,1), parameter estimates:

     asset 1   asset 2
b₀     0.034     0.011
b₁    -0.011     0.001
ω     -0.089    -0.092
α      0.125     0.115
β      0.979     0.984
γ     -0.089    -0.100
```

DCC: Estimation

This section estimates a DCC model by ML from the two series of standardized residuals (from the eGARCH estimation above).

The cells below use the `MLE()` function to estimate the DCC. However, the structure of the DCC model is a bit different from the main use case for `MLE()`. In particular, notice

1. To impose the necessary restrictions (α, β being positive and summing to less than 1), we estimate `(a,b)` but they imply the following `(α,β)` via the `DccParTrans()` function: $\alpha = e^a/(1+e^a+e^b)$ and $\beta = e^b/(1+e^a+e^b)$.
2. We use the `DccLL_()` function for the loglikelihood. It's the same as the `DccLL()` function, except that it only outputs the loglikelihood contributions.
3. We call as `MLE(DccLL_,par0,data,nothing)`, where `par0` are starting guesses for `(a,b)` and `data` is a vector of arrays, where `v = data[1], σ² = data[2], Qbar = data[3]`. The last argument is redundant (there are no "regressors" in this case).

```
#@doc2 DccLL
```

```
par0 = [0.2,1.5]                #Initial guess of parameters

#(α,β) = DccParTrans(par0)       #we estimate (a,b), but they imply (α,β)
#println("Initial guess of parameters")
#printmat([α,β];colNames=["coef"],rowNames=["α","β"])

Qbar = cov(v)                   #unconditional covariance matrix of v
data = [v,σ²,Qbar]              #vector of arrays, unpacked inside DccLL

DccLL_(par,y,x) = DccLL(par,y,x)[1]         #LL function, just one output
parHat, = MLE(DccLL_,par0,data,nothing)
(α,β)  = DccParTrans(parHat)                              #(a,b) → (α,beta)

println("Estimated parameters (before and after transformation):")
printmat(parHat,[α,β];colNames=["(a,b)","(α,β)"],rowNames=["(a) α","(b) β"])
```

```
Estimated parameters (before and after transformation):
          (a,b)     (α,β)
(a) α     1.649     0.050
(b) β     4.577     0.940
```

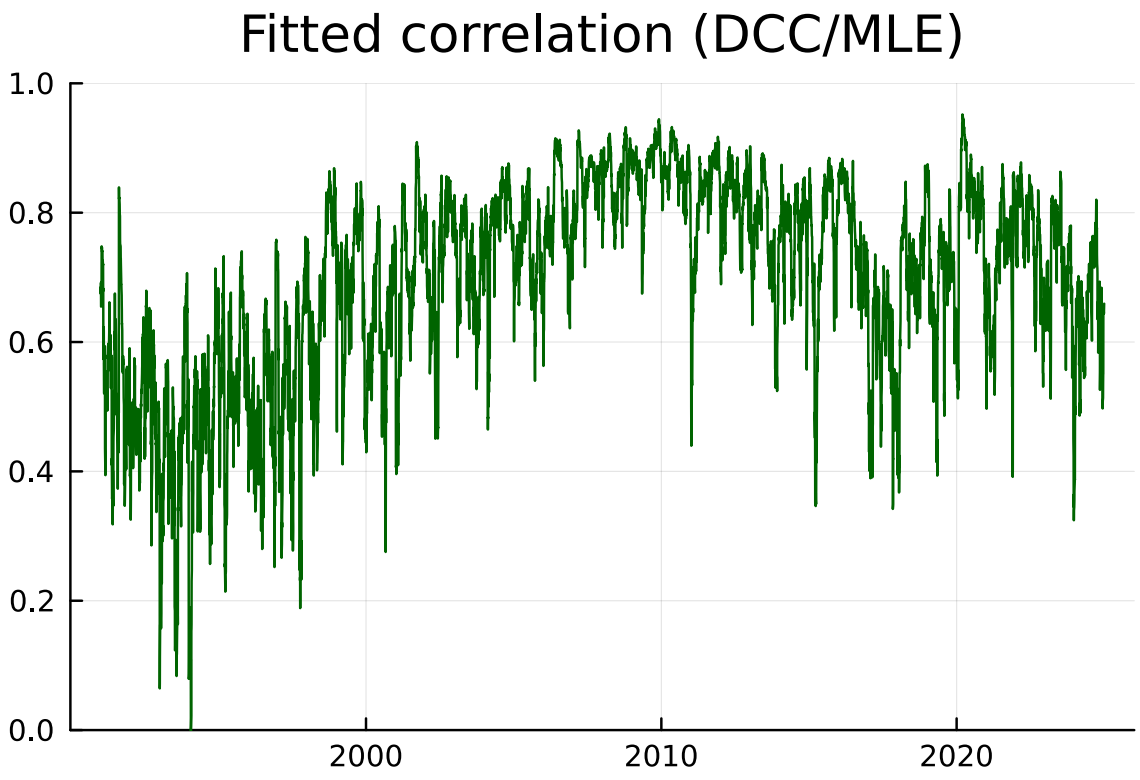
DCC: Calculate the Fitted Variance-Covariance and Correlation Matrices

The next cell calculates the fitted variance-covariance matrices (nxnxT) by using the point estimates and data in the likelihood function, exported as the 2nd output. It then converts the results to the implied correlation, which we plot.

```
(_,S,R) = DccLL(parHat,data)            #Txnxn, T fitted covariance matrices

xTicksLoc = [Date(2000),Date(2010),Date(2020)]
xTicksLab = Dates.format.(xTicksLoc,"Y")

p1 = plot( dN[2:end],R[:,1,2],
           linecolor = :darkgreen,
           xticks = (xTicksLoc,xTicksLab),
           legend = false,
           ylims  = (0.0,1),
           title = "Fitted correlation (DCC/MLE)" )
display(p1)
```


Fitted correlation (DCC/MLE)
1.0
0.8
0.6
0.4
0.2
0.0
2000
2010
2020

Risk Measures

This notebook illustrates the *Value at Risk* (VaR) and *Expected Shortfall*. To improve the performance, a model for time-varying risk is estimated and incorporated into the calculations. The model is backtested on a data set.

We use the the Distributions.jl package to calculate probability values and quantiles.

Load Packages and Extra Functions

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using Printf, Dates, Distributions, DelimitedFiles

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Value at Risk (VaR) for a N(, ²) Return

$\text{VaR}_{95\%} = -(5^{th}$ percentile of the return distribution)

With a $N(\mu, \sigma^2)$ distribution this gives approximately

$\text{VaR}_{95\%} \approx -(\mu - 1.64\sigma)$

A Remark on the Code

- The Distributions.jl package defines a normal distribution as `Normal(μ,σ)`. Notice that it uses the standard deviation, not the variance. For instance, to calculate the 5th quantile, use `quantile(Normal(μ,σ),0.05)` and to calculate the pdf value at each element of a vector `x`, use `pdf.(Normal(μ,σ),x)`.

```
"""
    VaRNormal(μ,σ,α)

Calculate VaR at a confidence level α (eg. 0.95) from a N() distribution with mean μ and standard deviati
"""
function VaRNormal(μ,σ,α)
    c   = quantile(Normal(0,1),1-α)
    VaR = -(μ + c*σ)             #same as -quantile(Normal(μ,σ),1-α)
    return VaR
end
```

```
VaRNormal
```

```
(μ,σ) = (8,16)

VaR_b = -(μ - 1.64*σ)
VaR   =  VaRNormal(μ,σ,0.95)

printblue("with μ=$μ and σ=$σ, we have approximately:\n")
printmat([μ - 1.64485*σ,VaR_b,VaR];rowNames=["5th quantile","≈ VaR 95%","VaR 95%"])
```

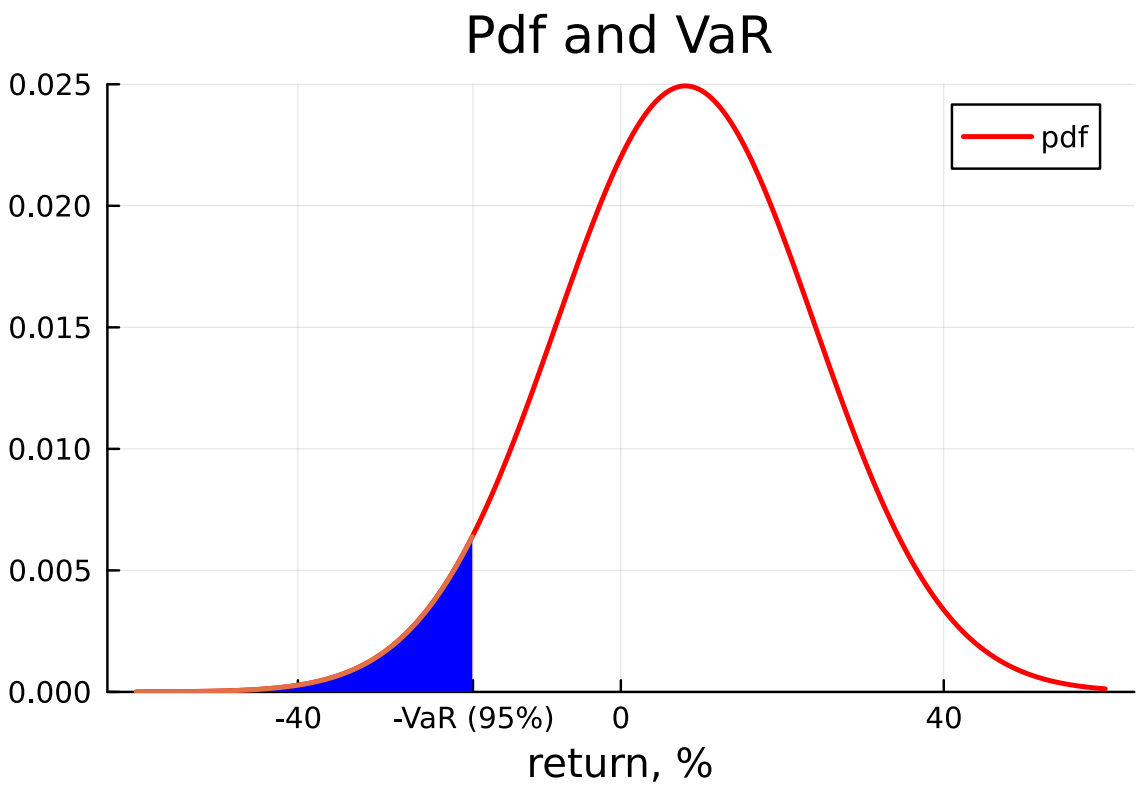
```
with μ=8 and σ=16, we have approximately:

5th quantile   -18.318
≈ VaR 95%       18.240
VaR 95%         18.318
```

```
R    = range(-60,60,length=301)
pdfR = pdf.(Normal(μ,σ),R)
Rb   = R[R .<= -VaR]             #or filter(<=(-VaR₉₅),R)

p1 = plot( R,pdfR,
           linecolor = :red,
           linewidth = 2,
```

```
            label = "pdf",
            ylims = (0,0.025),
            xticks = ([-40,-VaR,0,40],["-40","-VaR (95%)","0","40"]),
            title = "Pdf and VaR",
            xlabel = "return, %" )
plot!(Rb,pdf.(Normal(μ,σ),Rb),fillcolor=:red,linewidth=2,fill=(0,:blue),label="")
#vline!([-VaR],linecolor=:blue,label="-VaR (95%)")
display(p1)
```



Loading Daily S&P 500 Data

```
xx = readdlm("Data/RSP500Daily.csv",',',skipstart=1)   #net returns
dN = Date.(xx[:,1])
R  = Float64.(xx[:,2])
xx = nothing
```

```
vv = FindNN(R)                          #not missings
(dN,R) = (dN[vv],R[vv])

T = size(R,1)
println("Sample size: $T")

P = cumprod(1 .+ R/100);               #index level
```

```
Sample size: 11590
```

Backtesting a Static VaR from N() on Data

To backtest a VaR model, we study the relative frequency of loss > VaR, that is, if $R < -VaR$.

Backtesting for the Full Sample

The code below does this for different confidence levels (0.95,0.96,…) of the VaR. For instance, at the 0.95 confidence levels we 1. calculate the VaR as the (negative of the) 0.05 quantile of a normal distribution (with the mean and std estimated from the sample, which is just one possible choice) 2. count the relative frequency of the R < -VaR (it should be 0.05).

Then repeat for other confidence levels, eg. 0.96.

```
μ_emp = mean(R)                        #mean and std of data (empirical)
σ_emp = std(R)

α     = 0.90:0.005:0.995               #different confidence levels
L     = length(α)

(VaR_L,BreakFreq_L) = (fill(NaN,L),fill(NaN,L))   #for different confidence levels
for i in 1:L                    #loop over confidence levels
    VaR_L[i]       = VaRNormal(μ_emp,σ_emp,α[i])
    BreakFreq_L[i] = mean(R .< -VaR_L[i])          #freq of breaking the VaR
end
```

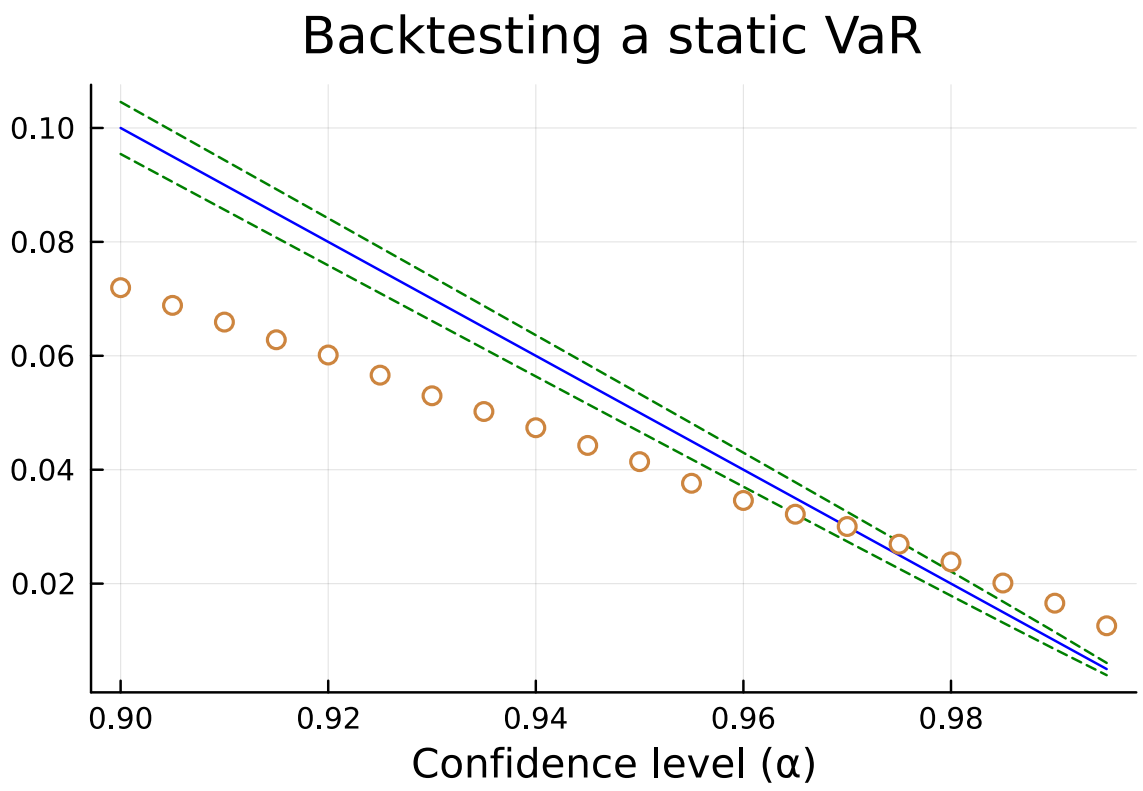
```
std_freq = sqrt.( α.*(1.0.-α)/T )      #std of dummy for breaking VaR

confB = 1.0 .- α    .+ hcat(-1.64*std_freq,1.64*std_freq);

#printblue("Backtesting a static VaR:\n")
```

```
#printmat([α VaR_L BreakFreq_L];colNames=["conf level","N()-based VaR","break freq"],width=18)
#printred("The break frequency should be 1-α")
```

```
plot( α,hcat(1.0.-α,confB),
      linecolor = [:blue :green :green],
      linestyle = [:solid :dash :dash],
      title = "Backtesting a static VaR",
      xlabel= "Confidence level (α)",
      legend = false
)
scatter!(α,BreakFreq_L,markercolor=:white,markerstrokecolor=:peru,markerstrokewidth=2)
```



Backtesting on a Moving Data Window

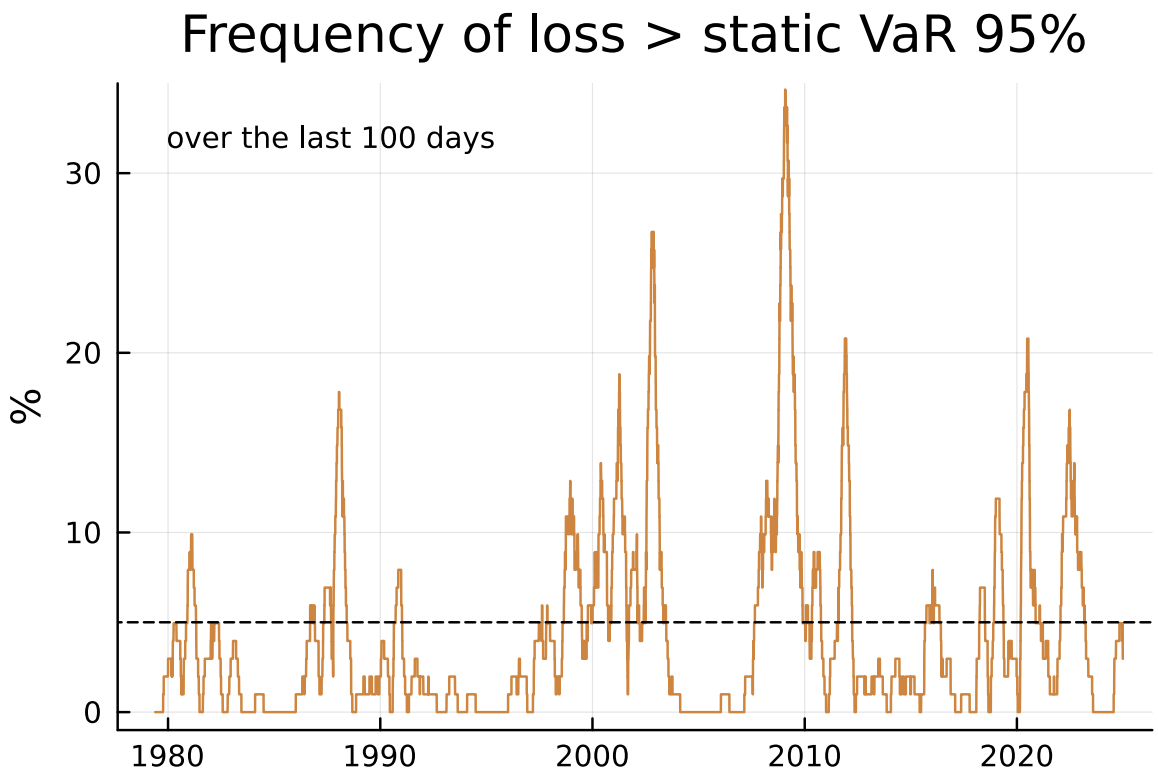
The code below also studies the relative frequency of loss > VaR, but over a *moving data window* of 100 days. This allows us to investigate if there are long periods of failures of the VaR model. To be parsimonious, we do this for a single confidence level, 0.95.

```
VaR = VaRNormal(μ_emp,σ_emp,0.95)        #static VAR at 95% confidence level

BreakFreq = fill(NaN,T)                 #vector, freq(loss>VaR) on moving data window
for t in 101:T                           #loop over end-points of the data window
    BreakFreq[t] = mean(R[t-100:t] .< -VaR)
end
```

```
xTicksLoc = Date(1980):Year(10):Date(2020)
xTicksLab = Dates.format.(xTicksLoc,"Y")

p1 = plot( dN,BreakFreq*100,
           linecolor = :peru,
           ylim = (-1,35),
           legend = false,
           xticks = (xTicksLoc,xTicksLab),
           title = "Frequency of loss > static VaR 95%",
           ylabel = "%",
           annotation = (Date(1980),32,text("over the last 100 days",8,:left)) )
hline!([5],linecolor=:black,line=(:dash,1))
display(p1)
```



A Simple Dynamic VaR with Time-Varying Volatility

To improve the performance of the VaR model, we combine a simple time series model for volatility with an N()-based VaR calculation. For simplicity, we here use an EWMA rather than a GARCH models. (The results are fairly similar.)

For simplicity, we first construct an simple estimate of σ_t^2 and μ_t as a backward looking exponential moving average (cf. RiskMetrics)

$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1-\lambda)(R_{t-1} - \mu_{t-1})^2$,

where

$\mu_t = \lambda \mu_{t-1} + (1-\lambda) R_{t-1}$.

where we use $\lambda \approx 0.94$.

We then redo the $\text{VaR}_{95\%}$ calculation using

$\text{VaR}_t = -(\mu_t - 1.64\sigma_t)$

and study if it has better properties than the static VaR.

The text rather uses a GARCH(1,1) model for σ_t^2. The results are very similar.

```
"""
    EWMA(x,λ,μ_start,σ²_start)

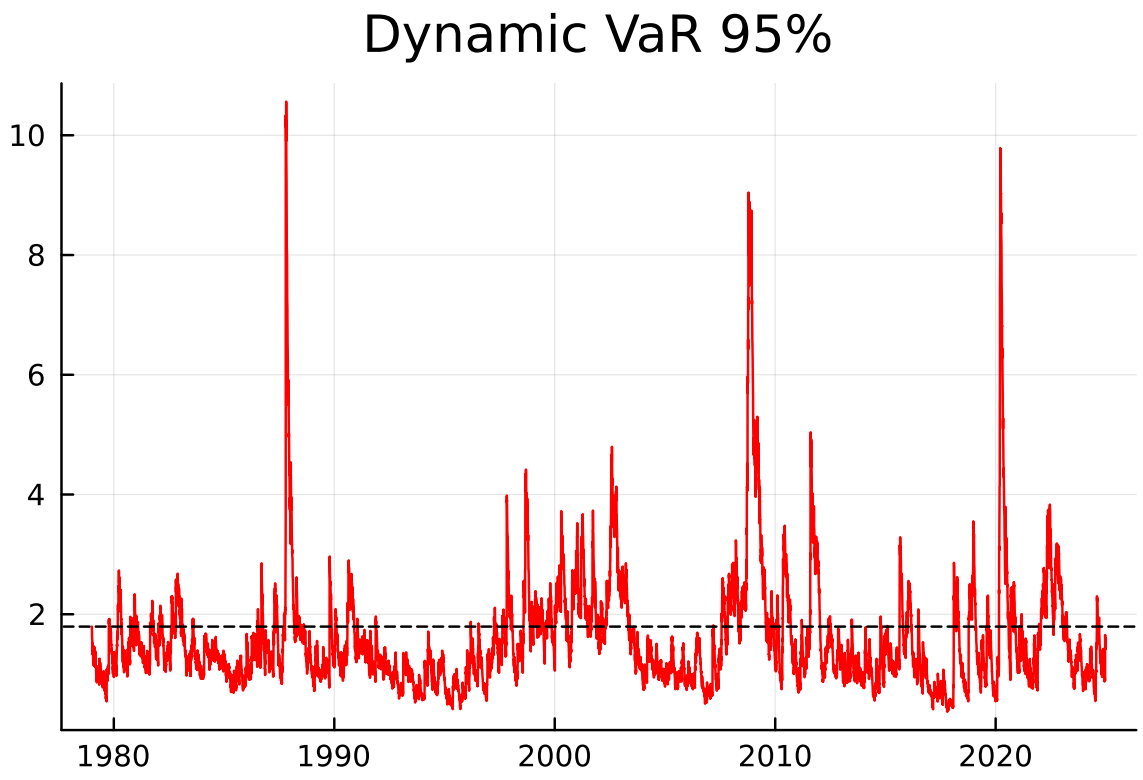
Calculate an EWMA mean and variance
"""
function EWMA(x,λ,μ_start,σ²_start)
    T = length(x)
    (μ,σ²) = (fill(μ_start,T),fill(σ²_start,T))
    for t in 2:T
        μ[t]  = λ*μ[t-1]  + (1-λ)*x[t-1]
        σ²[t] = λ*σ²[t-1] + (1-λ)*(x[t-1]-μ[t-1])^2
    end
    return μ,σ²
end
```

EWMA

```
λ = 0.94
(μ,σ²) = EWMA(R,λ,μ_emp,σ_emp^2)              #EWMA estimates of (μ,σ²)

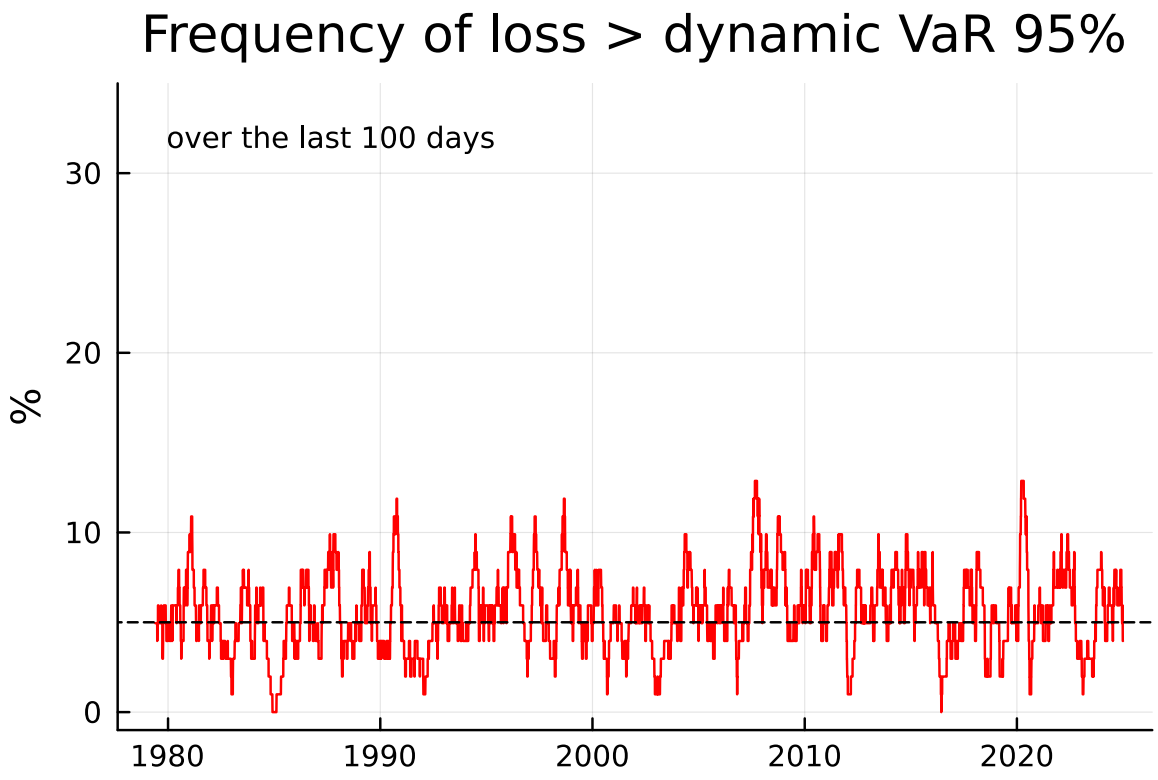
VaRd = VaRNormal.(μ,sqrt.(σ²),0.95);        #VaR for each date
```

```
p1 = plot( dN,VaRd,
           linecolor = :red,
           #ylim = (-1,35),
           legend = false,
           xticks = (xTicksLoc,xTicksLab),
           title = "Dynamic VaR 95%",
           ylabel= "",
           annotation = (Date(1980),32,text("over the last 100 days",8,:left)) )
hline!([VaR],linecolor=:black,line=(:dash,1))
display(p1)
```




```
BreakFreq = fill(NaN,T)         #backtesting, freq(loss>VaR) on moving data window
for t in 101:T
    BreakFreq[t] = mean(R[t-100:t] .< -VaRd[t-100:t])
end
```

```
p1 = plot( dN,BreakFreq*100,
           linecolor = :red,
           ylim = (-1,35),           #same ylim as for static model
           legend = false,
           xticks = (xTicksLoc,xTicksLab),
           title = "Frequency of loss > dynamic VaR 95%",
           ylabel= "%",
           annotation = (Date(1980),32,text("over the last 100 days",8,:left)) )
hline!([5],linecolor=:black,line=(:dash,1))
display(p1)
```



Expected Shortfall

The Expected Shortfall (ES) is the average loss, conditional on exceeding the VaR level. In terms of the returns (not losses) this is

$\text{ES}_\alpha = -\text{E}(R|R \leq -\text{VaR}_\alpha)$

For a normally distributed return $R \sim N(\mu, \sigma^2)$ we have

$\text{ES}_{95\%} = -\left(\mu - \frac{\phi(-1.64)}{0.05}\sigma\right)$,

where $\phi()$ is the pdf of a $N(0,1)$ variable. For other confidence levels, change -1.64 and 0.05.

Alternatively, we can calculate the ES as `-mean(truncated(Normal(μ,σ);upper=q))` where `q` is the quantile of the return distribution.

```
"""
    ESNormal(μ,σ,α)

Calculate ES at a confidence level α (eg. 0.95) from a N() distribution with
```

```
mean μ and standard deviation σ.

Notice that this can also be calculated as
```
q = quantile(Normal(μ,σ),1-α)
ES = -mean(truncated(Normal(μ,σ);upper=q))
```

"""
function ESNormal(μ,σ,α)
    c  = quantile(Normal(0,1),1-α)
    ϕ  = pdf(Normal(0,1),c)
    ES = -(μ - ϕ*σ/(1-α))
    return ES
end
```

ESNormal

```
(μb,σb) = (8,16)
ES =  ESNormal(μb,σb,0.95)

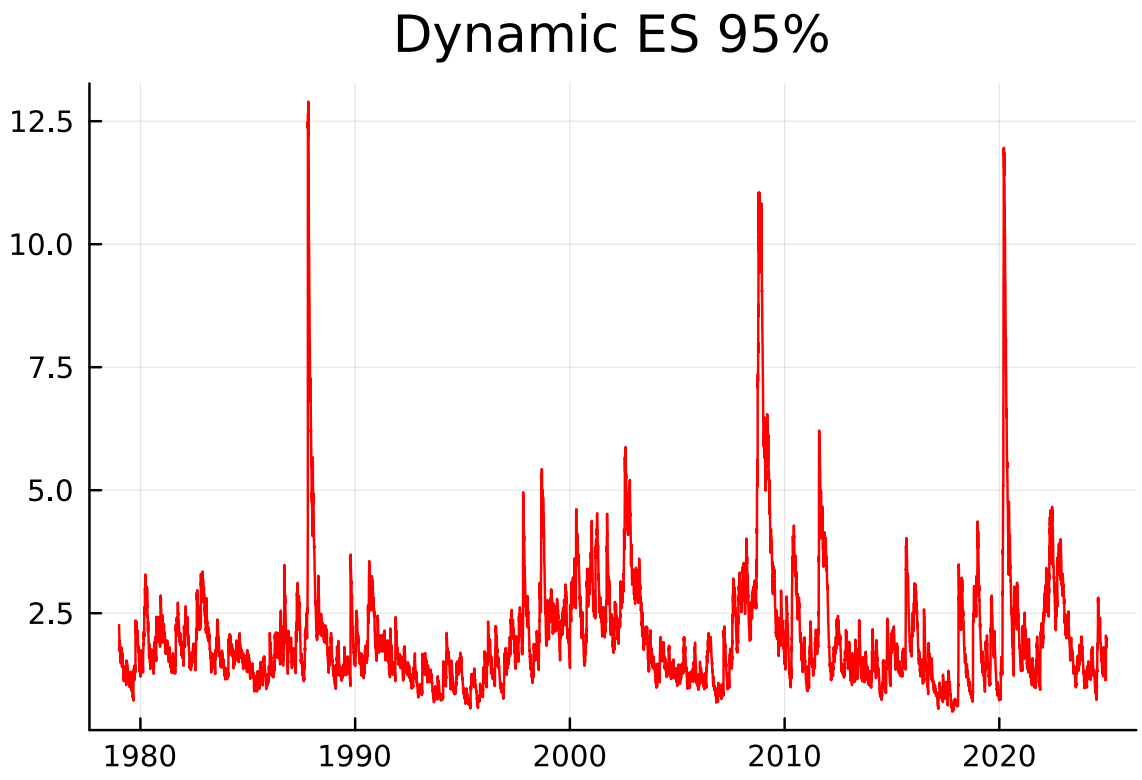
printblue("ES 95% with μ=$μb and σ=$σb is:\n")
println∩Ps(ES)

ESd = ESNormal.(μ,sqrt.(σ²),0.95);     #for each date
```

ES 95% with μ=8 and σ=16 is:

    25.003

```
p1 = plot( dN,ESd,
           linecolor = :red,
           legend = false,
           xticks = (xTicksLoc,xTicksLab),
           title = "Dynamic ES 95%",
           ylabel= "" )
display(p1)
```



Backtesting ES

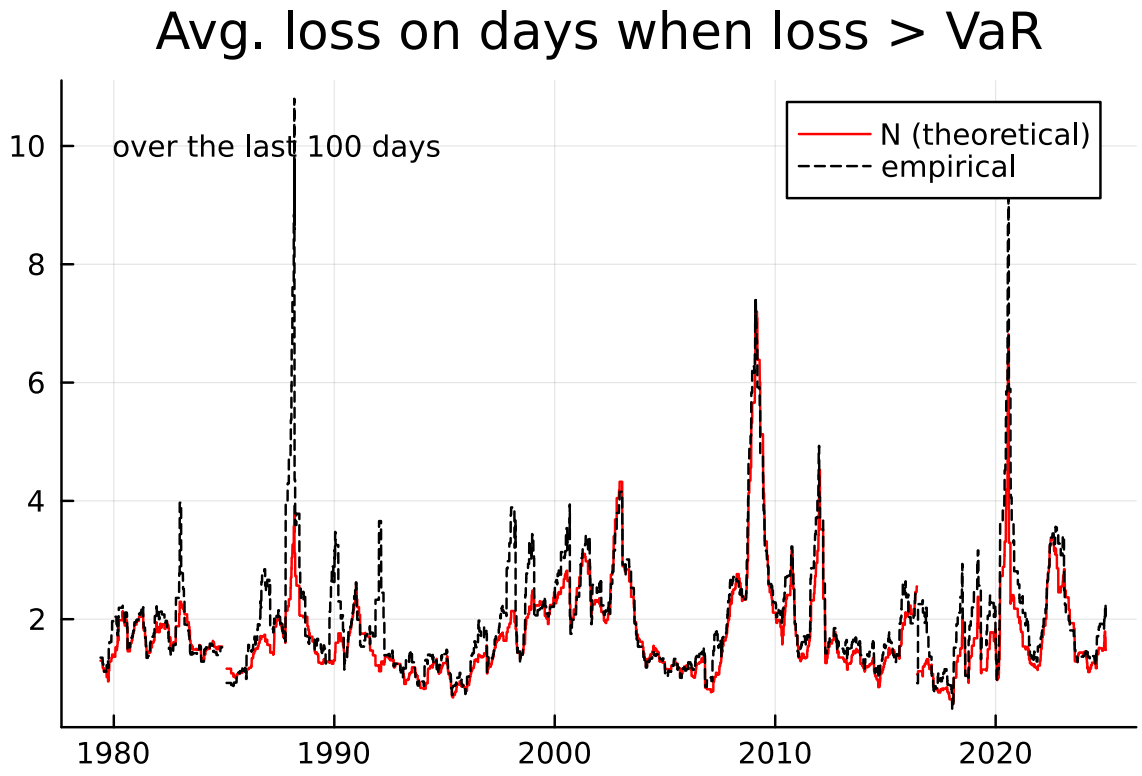
by comparing the empirical average return (conditional on loss > VaR) with the predictions from the model with $\mathrm{N}(\mu_t, \sigma_t^2)$.

Notice that the interactive dummy δ means that we are effectively calculating results only for those observations when $R < -VaR$.

```
ES_backtest = fill(NaN,T,2)          #backtesting ES
for t in 101:T
  δ               =  R[t-100:t] .< -VaRd[t-100:t]          #dummy: 1 if loss > VaR
 ES_backtest[t,1] =  sum(δ.*ESd[t-100:t])/sum(δ)        #mean N()-theoretical loss|loss>VaR
 ES_backtest[t,2] = -sum(δ.*R[t-100:t])/sum(δ)          #mean empirical loss|loss>VaR
end
```

```
p1 = plot( dN,ES_backtest,
           linecolor = [:red :black],
           linestyle = [:solid :dash],
           label = ["N (theoretical)" "empirical"],
```

```
            xticks = (xTicksLoc,xTicksLab),
            title = "Avg. loss on days when loss > VaR",
            ylabel= "",
            annotation = (Date(1980),10,text("over the last 100 days",8,:left)) )
display(p1)
```



Target Semi-Variance

The target semi-variance is the average value of `δ.*(x.-h).^2` where `δ[t]=1` if `x[t] <= h` and zero otherwise. The "Sortino" ratio is the ratio of the average `x-h` divided by the square root of the target semi-variance. It is an alternative to the Sharpe ratio.

```
"""
    TargetSemiVar(x,h)

Calculate target semi-variance
"""
```

```
function TargetSemiVar(x,h)
    δ = x .<= h
    TSV = mean( δ.*(x.-h).^2 )    #a function for target semi-variance
    return TSV
end
```

```
TargetSemiVar
```

```
TSV = TargetSemiVar(R,0)

println∩Ps("Target semi-variance: ",TSV)
```

```
Target semi-variance:      0.632
```

Maximum Drawdown

is the largest loss that could have been made in a sample, buy buying at the top and selling at the bottom.

```
"""
    HistMax(x)

Calculate historial max of a data series, up to and including `x[t]`
"""
function HistMax(x)
  T = length(x)
  xMax = fill(x[1],T)
  for t in 2:T
    xMax[t] = max(xMax[t-1],x[t])
  end
  return xMax
end


"""
    Drawdown(x)

Calculate the (relative) drawdown in a sample, compared to an earlier max level
```

```
"""
function Drawdown(x)
    x_HistMax = HistMax(x)
    D         = (x_HistMax - x)./x_HistMax         #relative drawdown
    return D, x_HistMax
end;
```

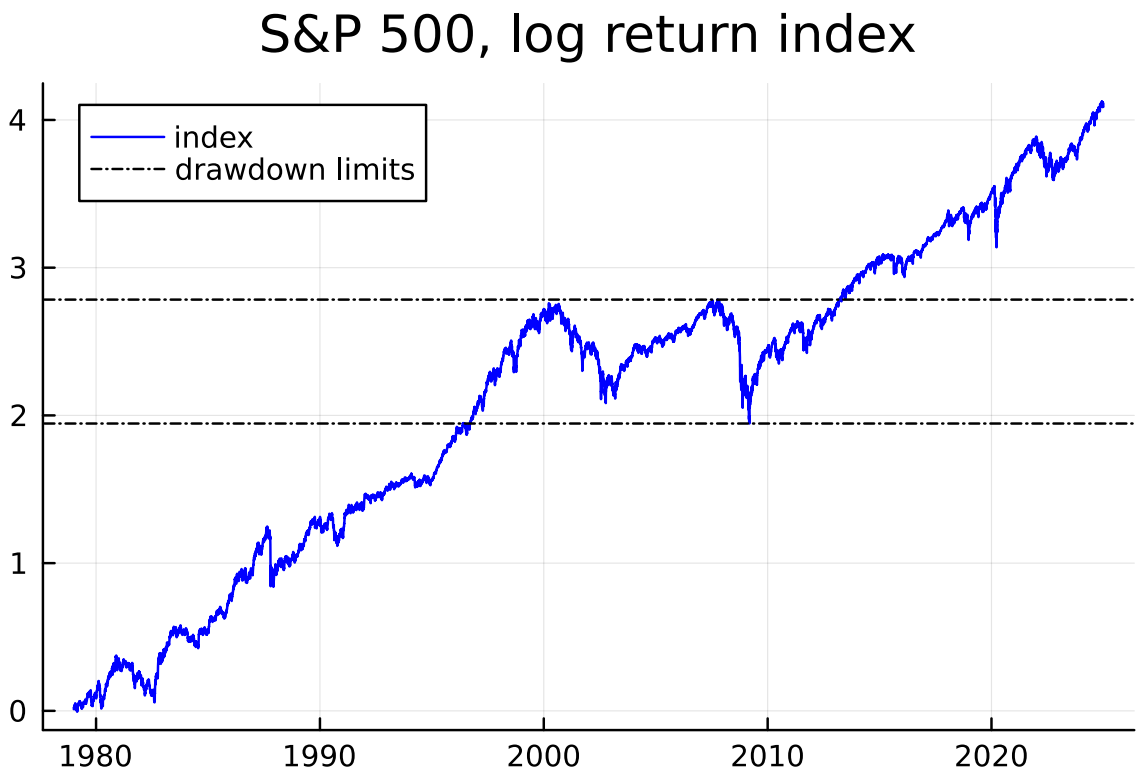
```
(D,P_max) =  Drawdown(P)                          #calculate the relative drawdown

vv1 = argmax(D)                                   #index of day with worst drawdown
vv2 = findfirst(==(P_max[vv1]),P_max)            #index of day when the max was reached

printblue("Maximum relative drawdown: $(round(Int,D[vv1]*100))% from $(dN[vv2]) to $(dN[vv1]) \n")
```

```
Maximum relative drawdown: 57% from 2007-10-09 to 2009-03-09
```

```
p1 = plot( dN,log.(P),
           linecolor = :blue,
           #legend = false,
           label = "index",
           xticks = (xTicksLoc,xTicksLab),
           title = "S&P 500, log return index",
           ylabel= "" )
hline!(log.(P[[vv1,vv2]]),linecolor=:black,line=:dashdot,label="drawdown limits")
display(p1)
```


S&P 500, log return index
index
drawdown limits
4
3
2
1
0
1980
1990
2000
2010
2020

Nonparametric Regressions

This notebook uses non-parametric methods for regression estimates.

Load Packages and Extra Functions

The key functions used in this notebook are in the (local) `FinEcmt_KernelRegression` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS, FinEcmt_KernelRegression
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
include(joinpath(pwd(),"src","FinEcmt_KernelRegression.jl"))
using .FinEcmt_OLS, .FinEcmt_KernelRegression
=#
```

```
using DelimitedFiles, Statistics
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Loading Data

The data consists of daily returns on S&P 500.

```
xx = readdlm("Data/ReSP500Daily.csv",',',skipstart=1)
R  = excise(Float64.(xx[:,2]))       #delete NaNs
xx = nothing
```

Kernel Regression

is a regression of the sort $y_t = b(x_t) + \epsilon_t$ where where regression function $b()$ is estimated by using data in the neighbourhood of a grid point $\tilde{x}$

$\hat{b}(\tilde{x}) = \frac{\sum_{t=1}^{T} w(u_t) y_t}{\sum_{t=1}^{T} w(u_t)}$,

where $u_t = (x_t - \tilde{x})/h$

where $w(u_t)$ is the weight of observation t, defined by a kernel function. This can be implemented as a regression of $\sqrt{w_t} y_t$ on $\sqrt{w_t} 1$, which also gives a valid standard error of the estimates. (The function below uses White's method.)

The function `KernelRegression()` uses one of several available kernels, defaulting to the Gaussian kernel. The function requires the user to input a bandwidth parameter `h`, which controls how quick $w(u_t)$ tapers off as a x_t deviates from $\tilde{x}$.

As an alternative to this code, consider the NonparametricRegression.jl package.

In the application below, we are estimating a non-parametric AR(1) for returns.

```
@doc2 KernelRegression
```

```
KernelRegression(y,x,xGrid,h,vv = :all,DoCovb=true,KernelFun=GaussianKernel)
```

Do kernel regression `y[vv] = b(x[vv])`, evaluated at each point in the `xGrid` vector, using bandwidth `h`. Implemented as weighted least squares (WLS), which also provide heteroskedasticity robust standard errors.

Input

- `y::Vector`: T-vector with data for the dependent variable
- `x::Vector`: T-vector with data for the regressor
- `xGrid::Vector`: Ngrid-vector with grid points where the estimates are done
- `vv::Symbol or Vector`: If `vv = :all`, then all data points are used, otherwise supply indices.
- `DoCovb::Bool`: If true, the standard error of the estimate is also calculated
- `KernelFun::Function`: Function used as kernel.

```
#println(@code_string KernelRegression([1],[1],[1],1))
```

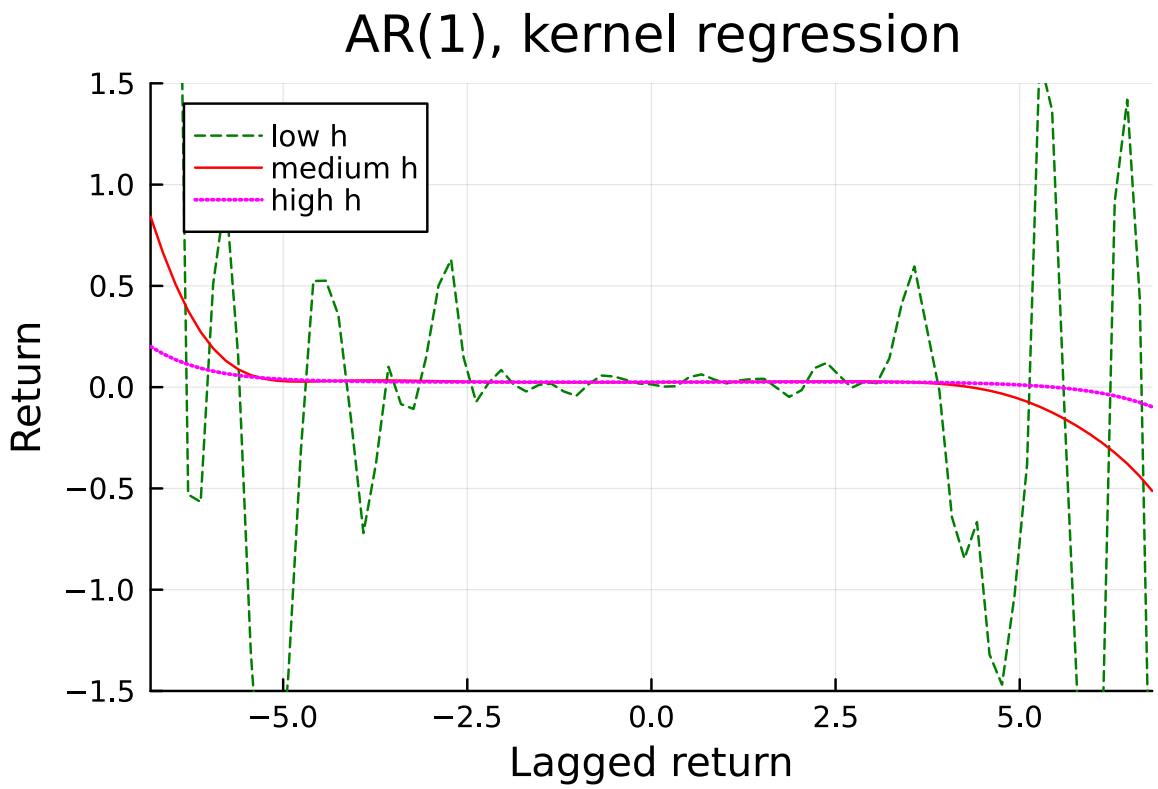
```
(y,x) = (R[2:end],R[1:end-1])               #return, return[t-1]
T     = size(y,1)
println("Sample size: $T")

xGrid = range(-6.8,6.8,81)                 #x values to estimate b(x) at, as in LN
h     = 1.4

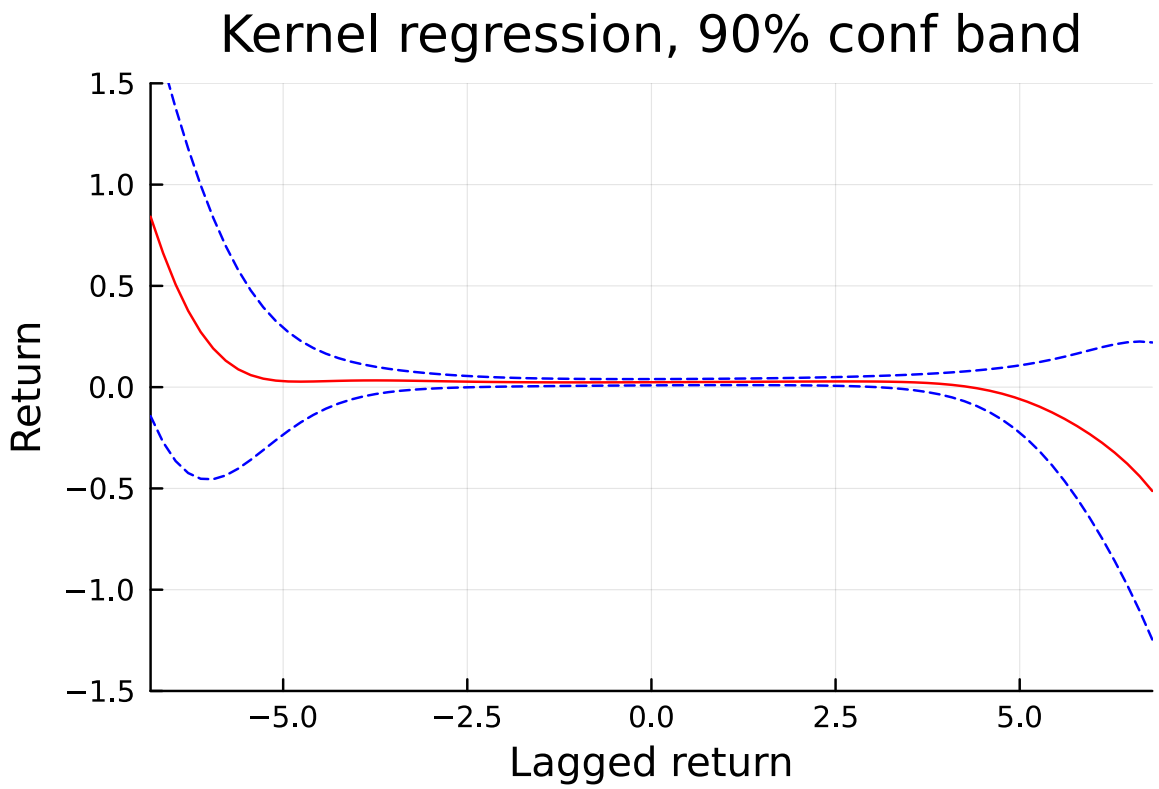
(bHat,StdbHat) = KernelRegression(y,x,xGrid,h)             #baseline choice of h
bHatHih,       = KernelRegression(y,x,xGrid,h*1.5)         #high h
bHatLoh,       = KernelRegression(y,x,xGrid,h/10);         #low h
```

```
Sample size: 11478
```

```
p1 = plot( xGrid,[bHatLoh bHat bHatHih],
           linestyle = [:dash :solid :dot],
           linewidth = [1 1 1.5],
           label = ["low h" "medium h" "high h"],
           color = [:green :red :magenta],
           xlim = extrema(xGrid),
           ylim = (-1.5,1.5),
           title = "AR(1), kernel regression",
           xlabel = "Lagged return",
           ylabel = "Return" )
display(p1)
```




```
p1 = plot( xGrid,[bHat (bHat-1.645*StdbHat) bHat+1.645*StdbHat],
           linecolor = [:red :blue :blue],
           linestyle = [:solid :dash :dash],
           label = "",
           title = "Kernel regression, 90% conf band",
           xlabel = "Lagged return",
           ylabel = "Return",
           xlim = extrema(xGrid),
           ylim = (-1.5,1.5),
           annotation = (-5,-4,text("using medium h",8)))
display(p1)
```



Rule of Thumb Choice of h

for the non-parametric regression.

Run the regression

$y = \alpha + \beta x + \gamma x^2 + \varepsilon$

and use the following rule-of-thumb choice

$h = T^{-1/5}|\gamma|^{-2/5}\sigma_\varepsilon^{2/5}(x_{\max} - x_{\min})^{1/5} \times 0.6.$

In practice, replace $x_{\max} - x_{\min}$ by the difference between the 90th and 10th percentiles of x.

This is implemented in the `hRuleOfThumb()` function (included above).

```
h_crude = hRuleOfThumb(y,x)
println("\nRule-of-thumb value of h: ",h_crude)
```

```
Rule-of-thumb value of h:      0.724
```

Cross-Validation (extra)

to choose the bandwidth h for the kernel regression.

To do a cross-validation (leave-one-out)

(1) Pick a bandwidth h, do the kernel regression but leave out observation t and then record the out-of-sample prediction error $y_t - \hat{b}_{-t}(x_t, h)$. Notice that this is the error for observation t only.

(2) Repeat for all $t = 1 - T$ to calculate the CV

$\text{CV}(h) = \sum_{t=1}^{T} [y_t - \hat{b}_{-t}(x_t, h)]^2 / T$,

(3) Finally, redo for several different values of h, and pick the h value that minimizes $\text{CV}(h)$.

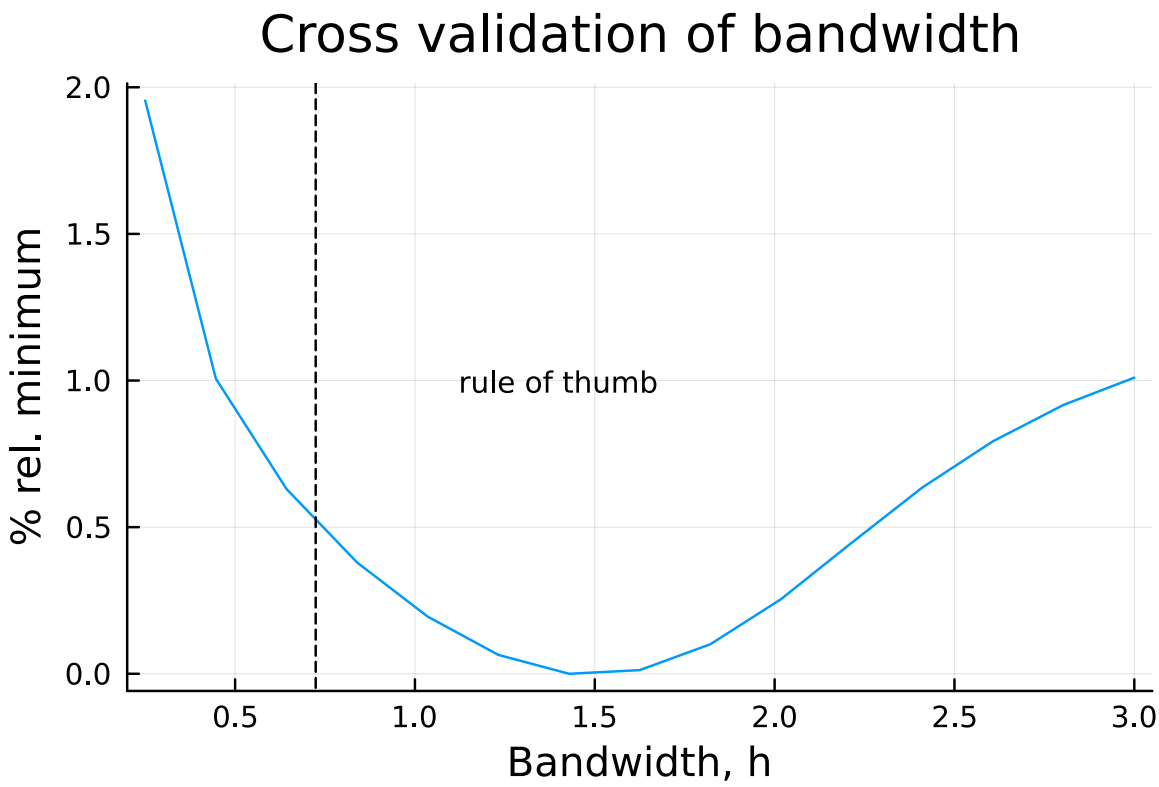
A Remark on the Code

Notice: Cross-validation calculations *take some time*. The code below uses a slightly optimized version. The package also contains a slower brute-force version (`CrossValidateKernelR()`).

```
hM = range(0.25,3,length=15)   #candiate h values

CV = CrossValidateKernelR2(y,x,hM)
CV_rel = (CV/minimum(CV) .- 1)*100;     #% deviation from minimum
```

```
p1 = plot( hM,CV_rel,
           legend = false,
           xlim = (minimum(hM)-0.05,maximum(hM)+0.05),
           #ylim = (0.99,1.025),
           title = "Cross validation of bandwidth",
           xlabel = "Bandwidth, h",
           ylabel = "% rel. minimum",
           annotation = (1.4,0.995,text("rule of thumb",8)) )
vline!([h_crude],linecolor=:black,line=(:dash,1))
display(p1)
```



Local Linear Regression

estimates

$y_t = a + b(x_t - x) + \epsilon_t$

for a grid of x values. This can also be implemented as weighted least squares (which gives both point estimates and valid standard errors). This is implemented in the `LocalLinearRegression()` function.

```
@doc2 LocalLinearRegression
```

```
LocalLinearRegression(y,x,xGrid,h,vv = :all,DoCovb=true,KernelFun=GaussianKernel)
```

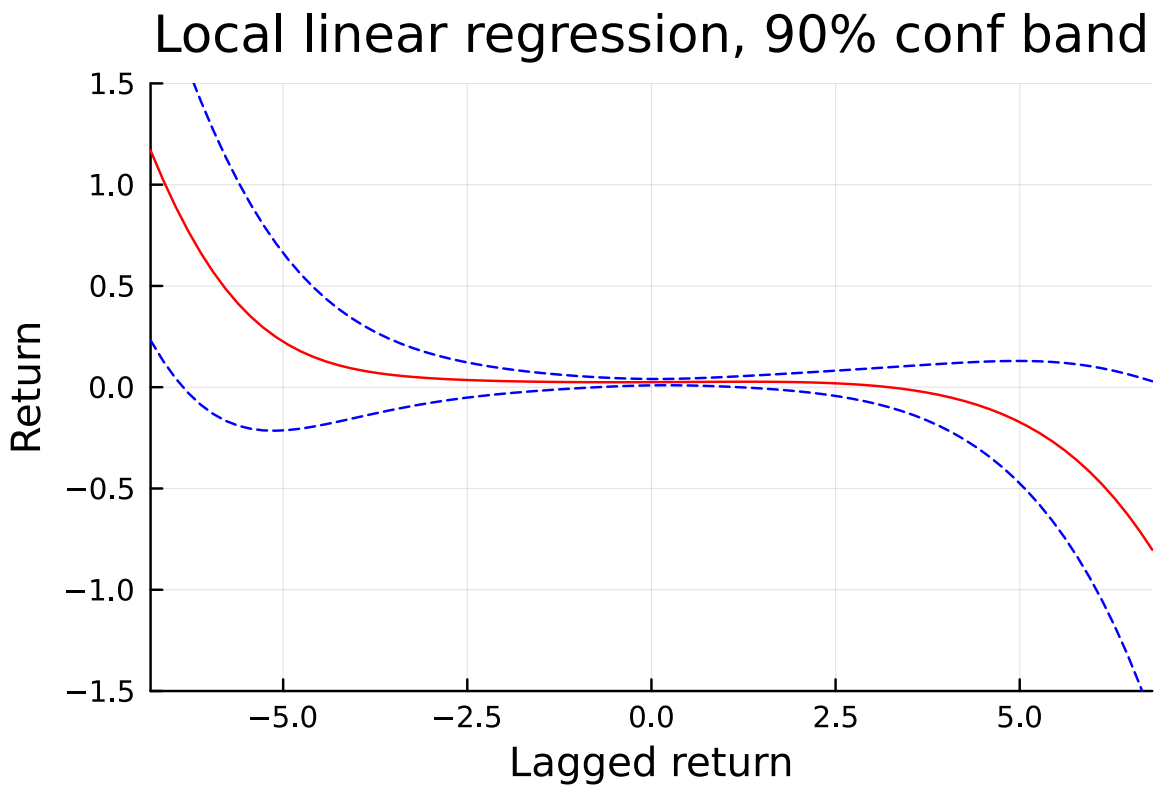
Do local linear regression `y = a + b(x-xGrid[i])`, where both `a` and `b` will differ across `xGrid[i]` values. The estimates of `a` and their standard errors are exported.

See `KernRegrFn()` for further comments

```
#println(@code_string LocalLinearRegression([1],[1],[1],1))
```

```
#xGrid = -10:0.25:10                          #x values to estimate b(x) at
h      = 2.25
(aHat,StdaHat) = LocalLinearRegression(y,x,xGrid,h);          #baseline choice of h

p1 = plot( xGrid,[aHat (aHat-1.645*StdaHat) aHat+1.645*StdaHat],
           linecolor = [:red :blue :blue],
           linestyle = [:solid :dash :dash],
           label = "",
           title = "Local linear regression, 90% conf band",
           xlabel = "Lagged return",
           ylabel = "Return",
           xlim = extrema(xGrid),
           ylim = (-1.5,1.5) )
display(p1)
```



Quantile Regressions

This notebook illustrates quantile regressions.

As an alternative, consider the QuantileRegressions.jl package.

Load Packages and Extra Functions

The key function used in this notebook `QuantRegrIRLS()` is from the (local) `FinEcmt_OLS` module.

```
MyModulePath = joinpath(pwd(),"src")
!in(MyModulePath,LOAD_PATH) && push!(LOAD_PATH,MyModulePath)
using FinEcmt_OLS
```

```
#=
include(joinpath(pwd(),"src","FinEcmt_OLS.jl"))
using .FinEcmt_OLS
=#
```

```
using DelimitedFiles, Statistics
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Loading Data

```
xx  = readdlm("Data/SmallCapDaily.csv",',',skipstart=1)
Re  = Float64.(xx[:,2])        #equity market excess returns

y = Re[2:end]                  #dependent variable
x = Re[1:end-1]                #regressor in an AR(1)
```

```
T   = size(y,1)
println("Sample size: $T")
```

```
Sample size: 11597
```

A Function for Quantile Regressions

The `QuantRegrIRLS()` function from the `FinEcmt_OLS` estimates a quantile regression. It uses a simple iterative (re-)weighted least squares (IRLS) approach. It can be shown that a linear programming approach gives almost identical results for this data set. However, it seems that the iterative approach is quicker in really large data sets. The function returns several different calculations of the standard errors: see the code below and the lecture notes for details.

The subsequent cells calculate and show the results. The calculations take some time.

A Remark on the Code

- The arrays in the iteration are pre-allocated and then the new results overwrite the old ones, in an attempt to decrease the memory usage.

```
@doc2 QuantRegrIRLS
```

```
QuantRegrIRLS(y,x,q=0.5;prec=1e-8,epsu=1e-6,maxiter=1000)
```

Estimate a quantile regression for quantile `q`. The outputs are the point estimates and three different variance-covariance matrices of the estimates.

Input

- `y::Vector`: T vector, dependent variable
- `x::VecOrMat`: TXK, regressors (including any constant)
- `q::Number`: quantile to estimate at, 0<q<1
- `prec::Float64`: convergence criterion, 1e-8
- `epsu::Float64`: lower bound on 1/weight, 1e-6
- `maxiter::Int`: maximum number of iterations, 1000

Output

- `theta::Vector`: K vector, estimated coefficients
- `vcv::Matrix`: KxK, traditional covariance matrix
- `vcv2::Matrix`: KxK, Powell (1991) covariance matrix
- `vcv3::Matrix`: KxK, Powell (1991) covariance matrix, uniform

Remarks

1. `while maximum(abs,b - b_old) > prec ...end` creates a loop that continues as long as the new and previous estimates differ more than `prec`. However, once the number of iterations exceed `maxiter` then the execution is stopped.
2. `u .= max.(u,epsu)` limits how small `u` can become which makes the algorithm more stable (recall: the next command is `x./u`).

```
#using CodeTracking
#println(@code_string QuantRegrIRLS([1],[1],0.05))
```

```
xGrid = quantile(x,0.01:0.01:0.99)              #quantiles of the regressor, for plots
xGrid = [ones(size(xGrid)) xGrid]

qM = [0.95,0.75,0.5,0.25,0.05]                      #quantiles to estimate at

cx = [ones(T) x]                                    #[constant regressors]

bM    = fill(NaN,length(qM),2)                      #to store regression coeffs
qPred = fill(NaN,length(qM),size(xGrid,1))          #to store predicted values
for (i,q) in enumerate(qM)
    #local b_q
    b_q        = QuantRegrIRLS(y,cx,q)[1]        #output just the point estimates
    bM[i,:]    = b_q
    qPred[i,:] = xGrid*b_q
end

printblue("quantile regression coefs:\n")
printmat(bM;colNames=["c","slope"],rowNames=string.(qM),cell00="quantile")

printred("\nThe function QuantRegrIRLS also outputs different variance-covariance matrices. Compare th
```

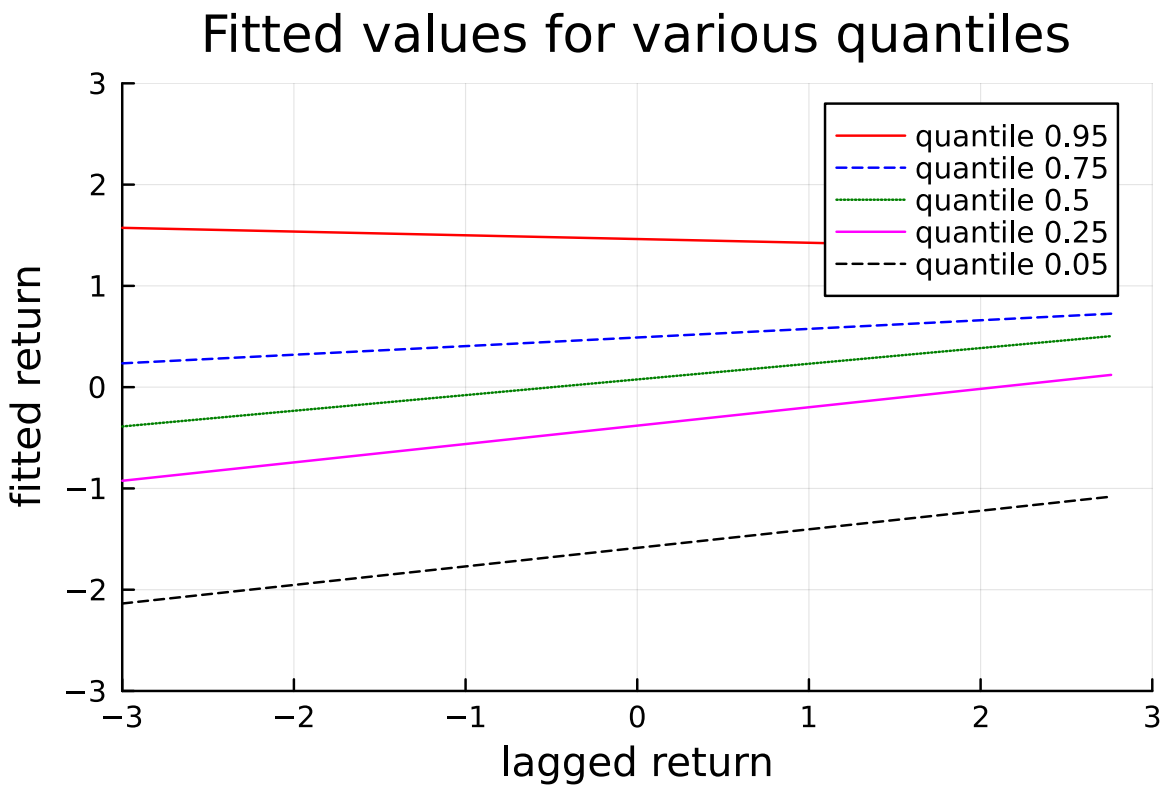
```
quantile regression coefs:
```

```
quantile          c      slope
0.95          1.463     -0.037
0.75          0.490      0.085
0.5           0.076      0.155
0.25         -0.380      0.182
0.05         -1.587      0.183
```

The function QuantRegrIRLS also outputs different variance-covariance matrices. Compare them.

```
lab = permutedims(["quantile $(qM[i])" for i=1:length(qM)])

p1 = plot( xGrid[:,2],qPred',
           color = [:red :blue :green :magenta :black],
           linestyle = [:solid :dash :dot :solid :dash],
           ylims = (-3,3),
           xlims = (-3,3),
           label = lab,
           xlabel = "lagged return",
           ylabel = "fitted return",
           title = "Fitted values for various quantiles" )
display(p1)
```


Fitted values for various quantiles
quantile 0.95
quantile 0.75
quantile 0.5
quantile 0.25
quantile 0.05
fitted return
lagged return
3
2
1
0
−1
−2
−3
−3
−2
−1
0
1
2
3